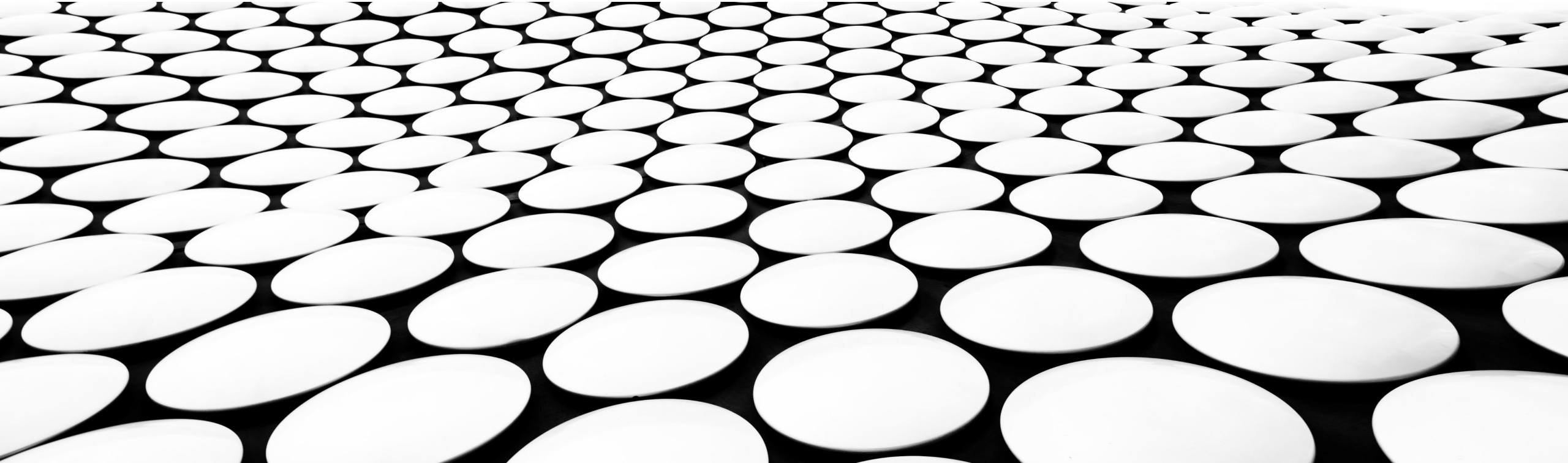
Introduction to Machine Learning

part 2



References

Parts of this lecture are based on:

1. Slides from the Harvard Course CS109A Introduction to Data Science by Pavlos Protopapas, Kevin Rader and Chris Tanner, available at https://github.com/Harvard-IACS/2019-CS109A

2. James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. An introduction to statistical learning. Vol. 112. New York: springer, 2013.

3. Cielen, Davy, Arno Meysman, and Mohamed Ali. Introducing data science: big data, machine learning, and more, using Python tools. Manning Publications Co., 2016.

Agenda

- Polynomial Regression
- Overfitting
- Regularization
- Decision Trees

Polynomial Regression

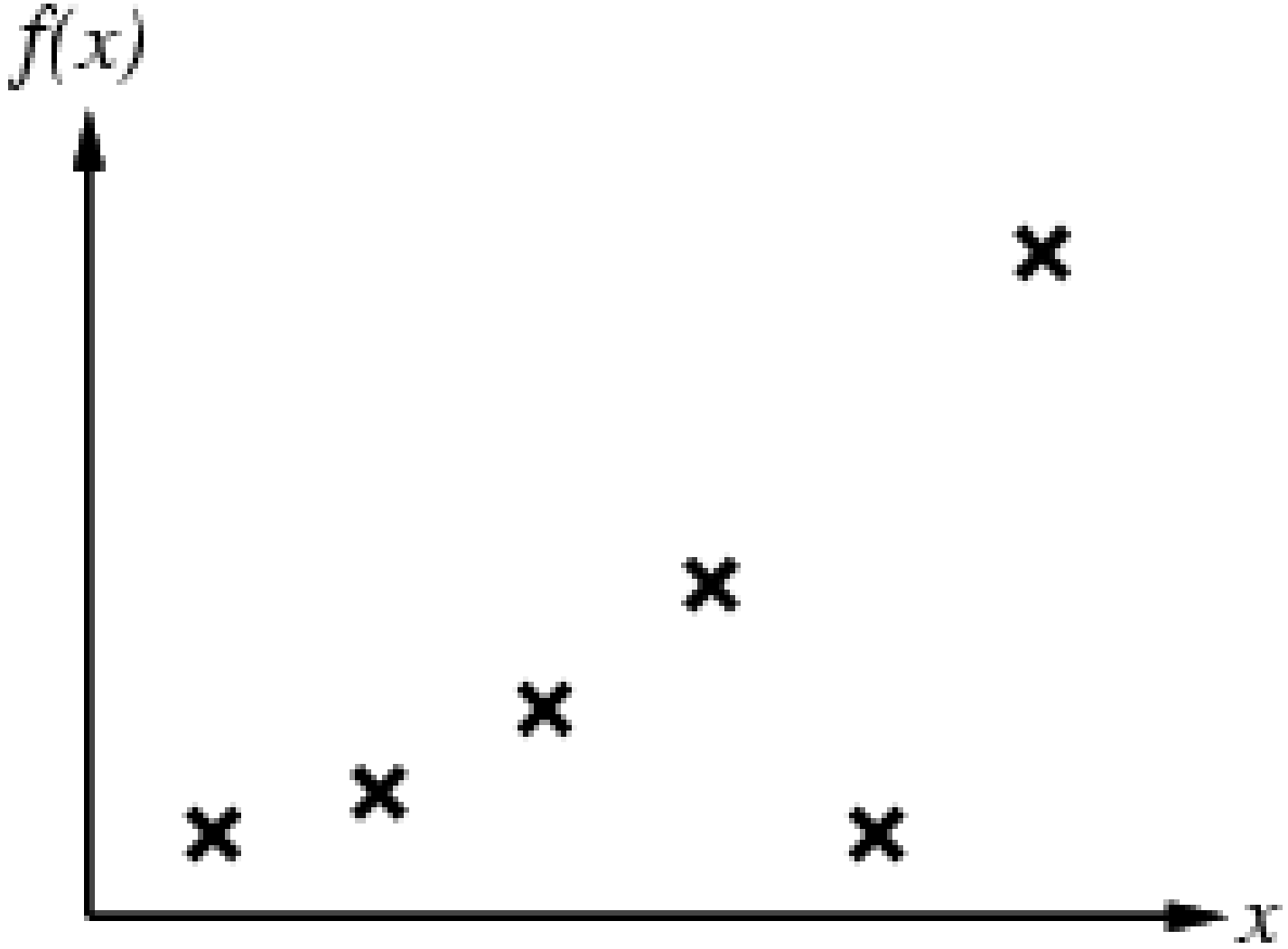
- The simplest non-linear model we can consider, for a response *Y* and a predictor *X*, is a polynomial model of degree *M*,

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \ldots + \beta_M x^M + \epsilon.$$

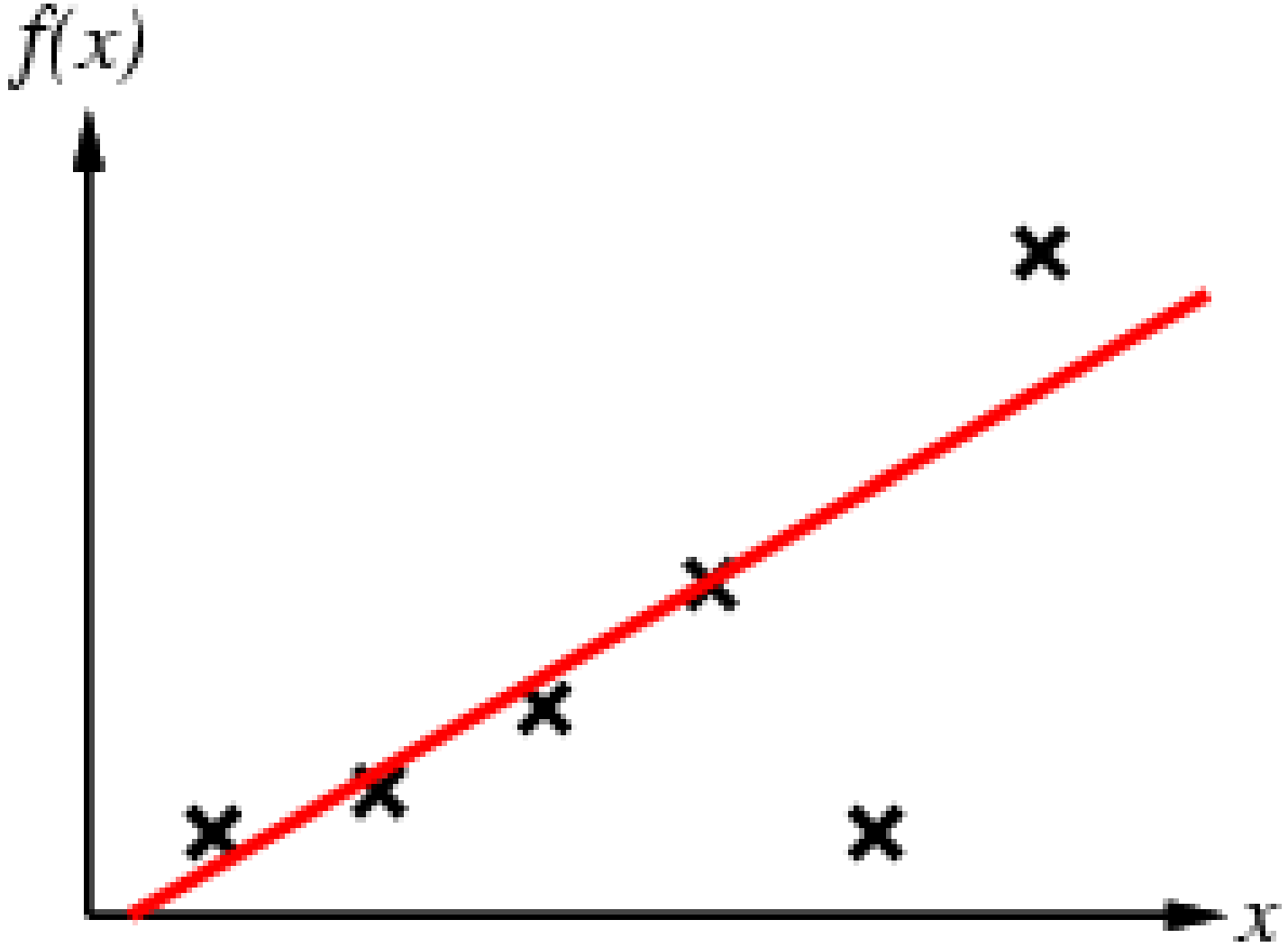
- Just as in the case of linear regression with cross terms, polynomial regression is a special case of linear regression - we treat each x^m as a separate predictor. Thus, we can write

$$\mathbf{Y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}, \qquad \mathbf{X} = \begin{pmatrix} 1 & x_1^1 & \ldots & x_1^M \\ 1 & x_2^1 & \ldots & x_2^M \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \ldots & x_n^M \end{pmatrix}, \qquad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_M \end{pmatrix}.$$

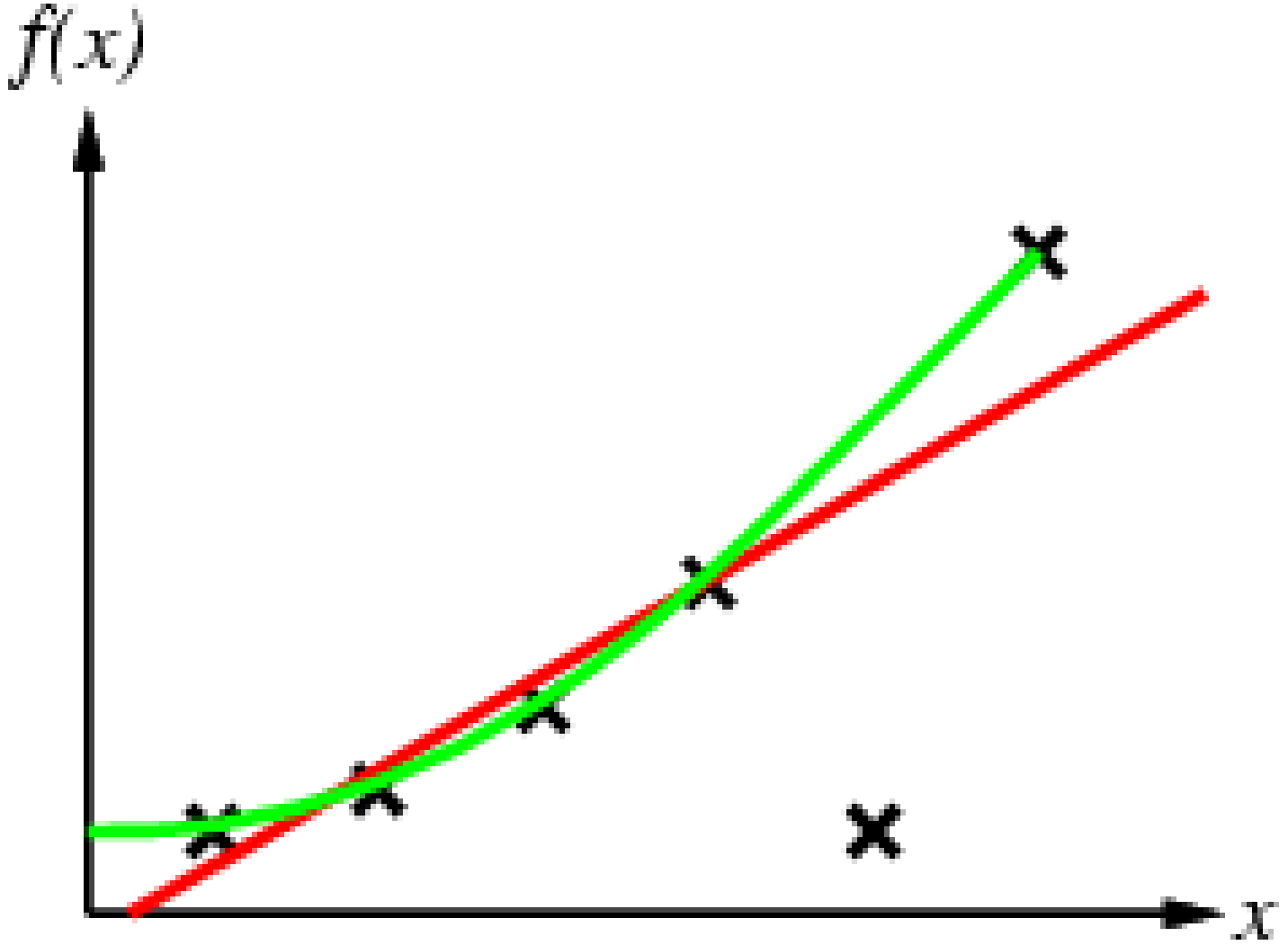
Curve fitting



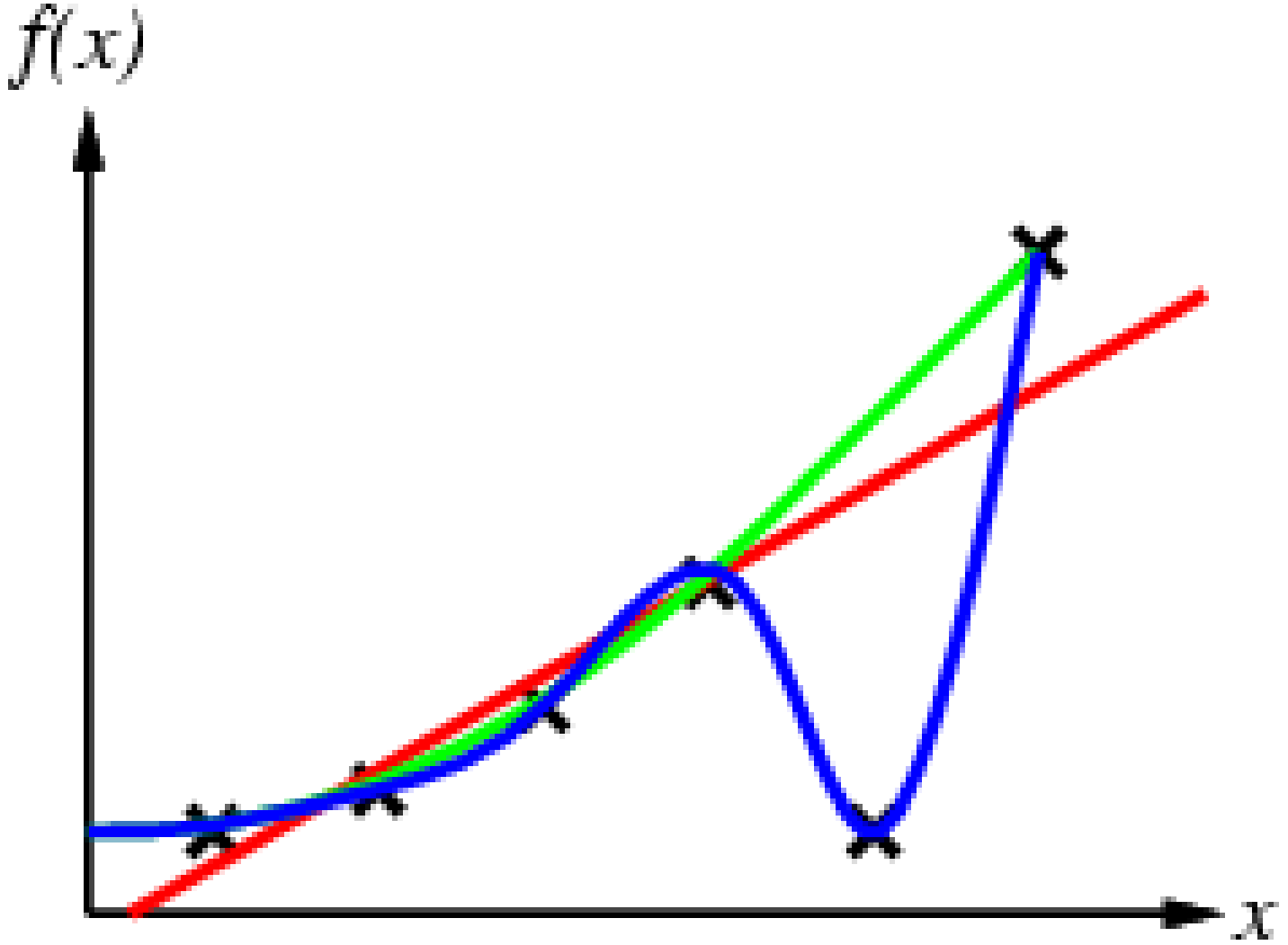
Curve fitting



Curve fitting



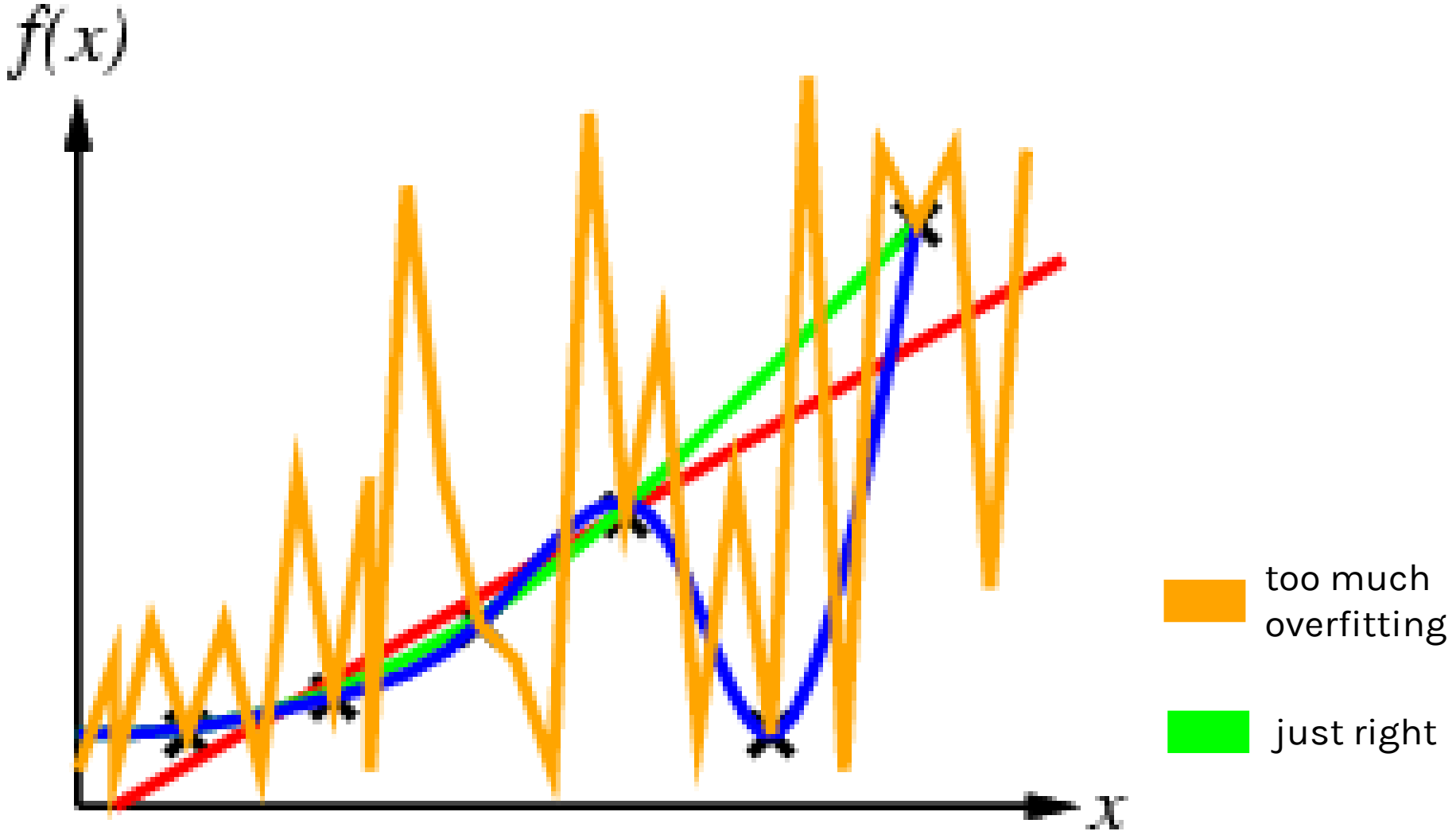
Curve fitting



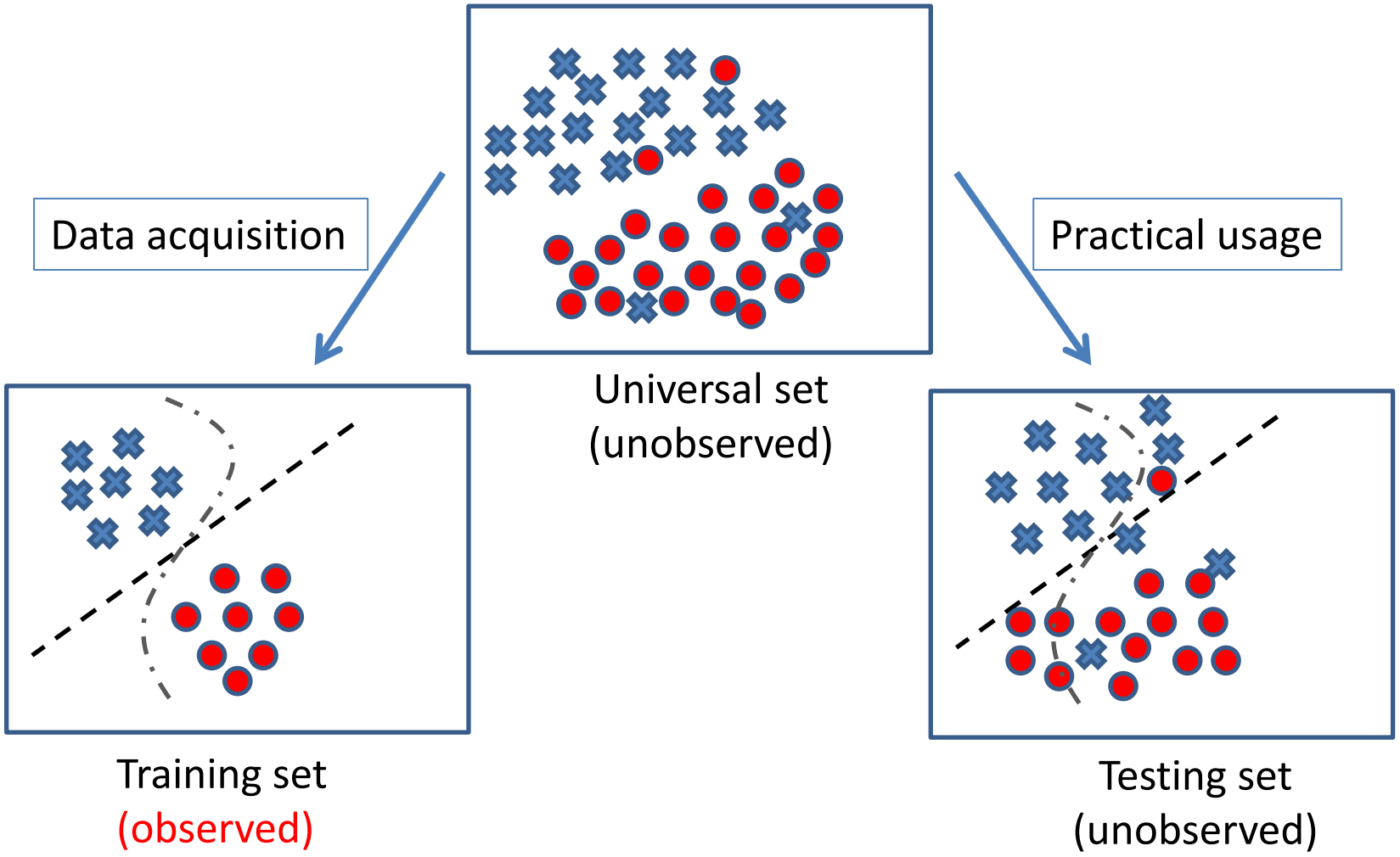
Curve fitting

- **Ockham's razor**:

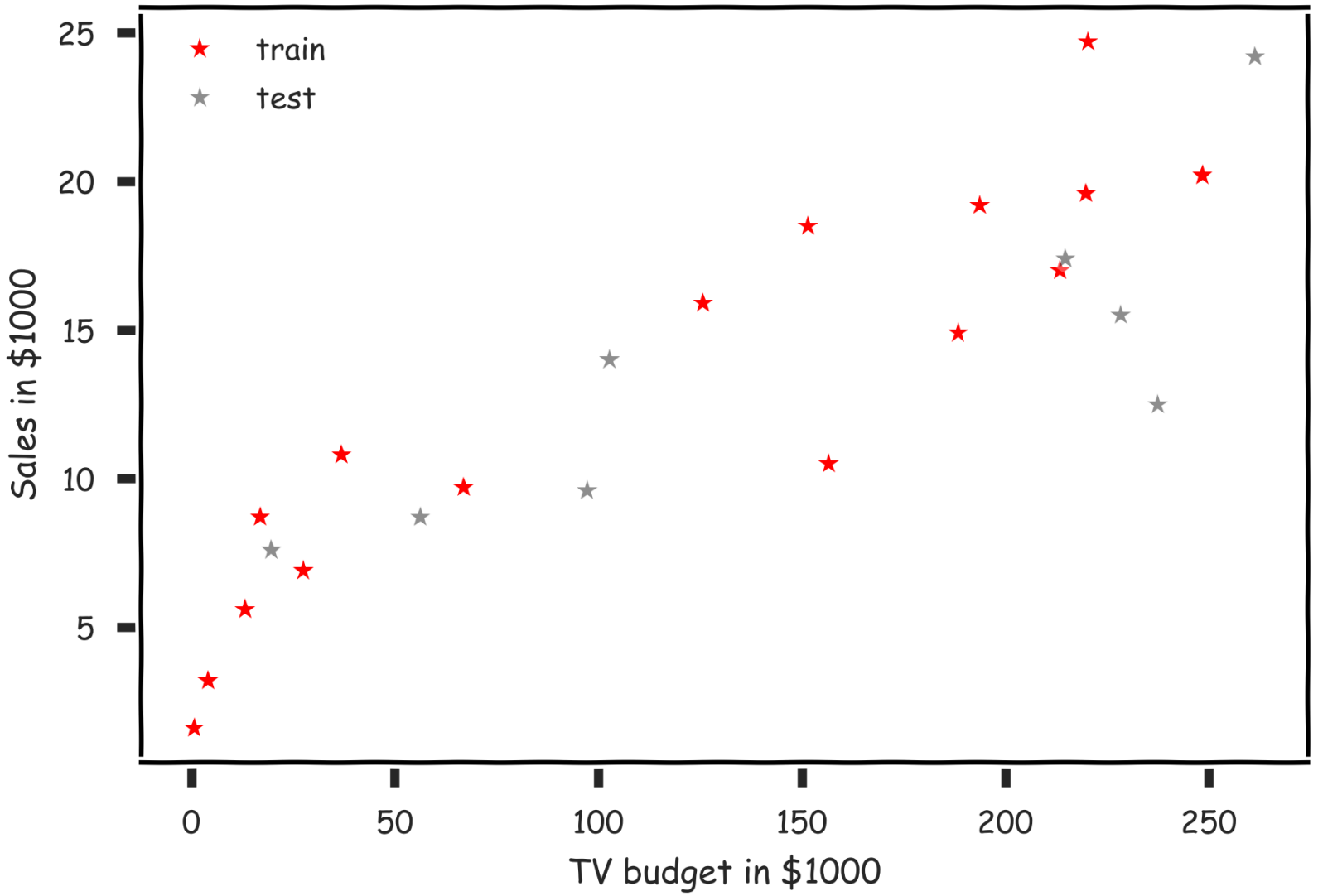
  Prefering the simplest hypothesis that is consistant with the data.



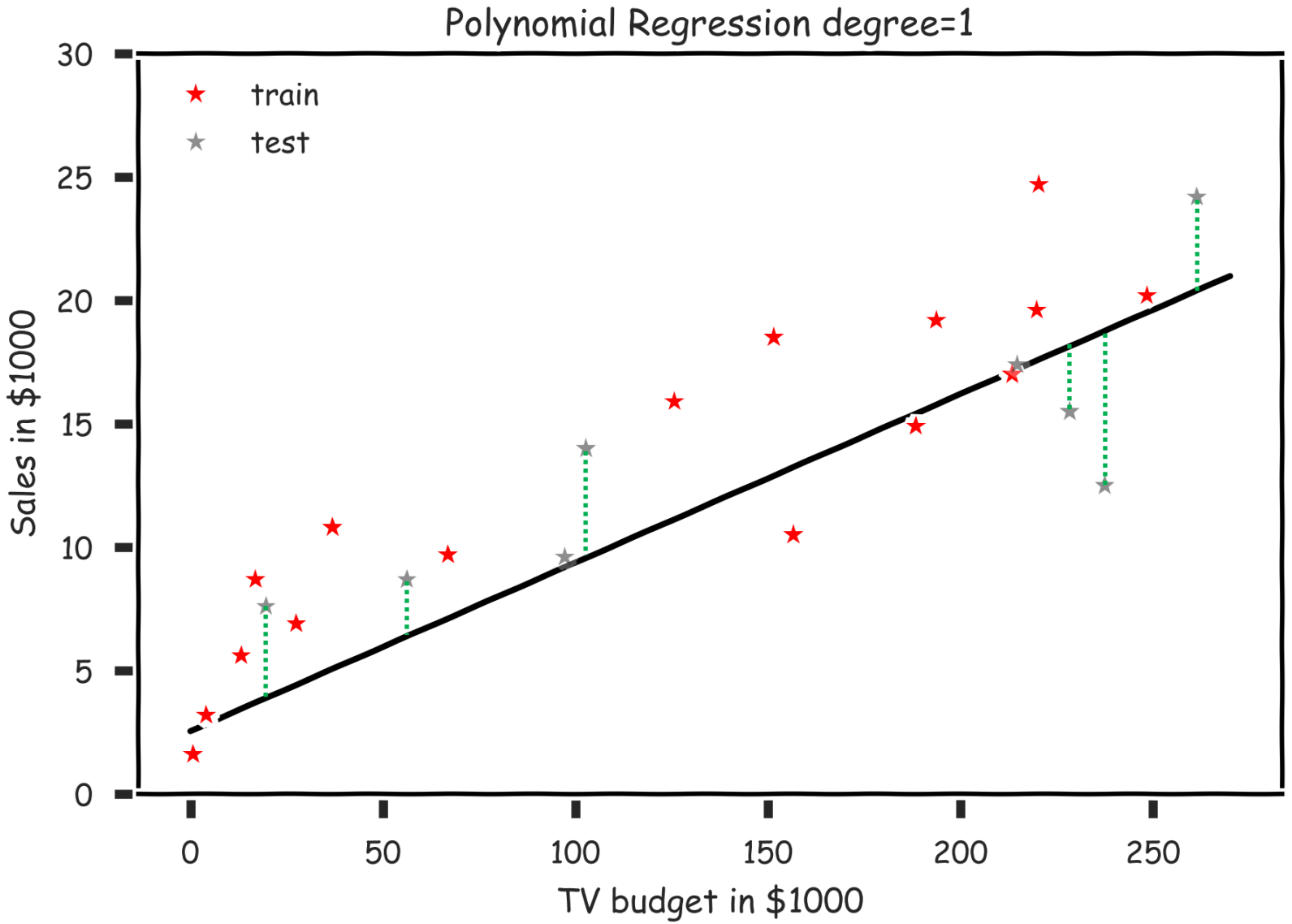
Training and testing



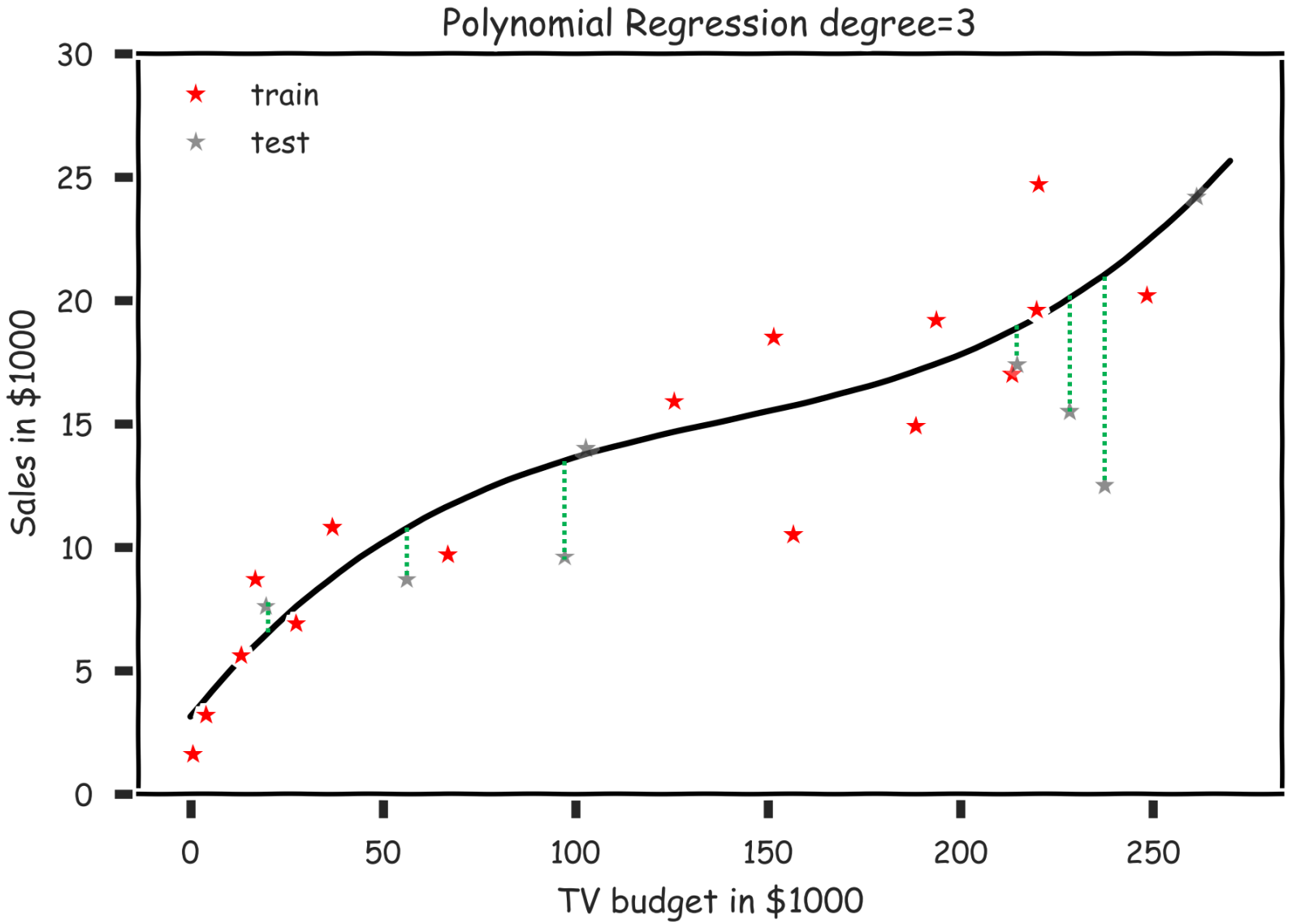
Polynomial Regression (cont)



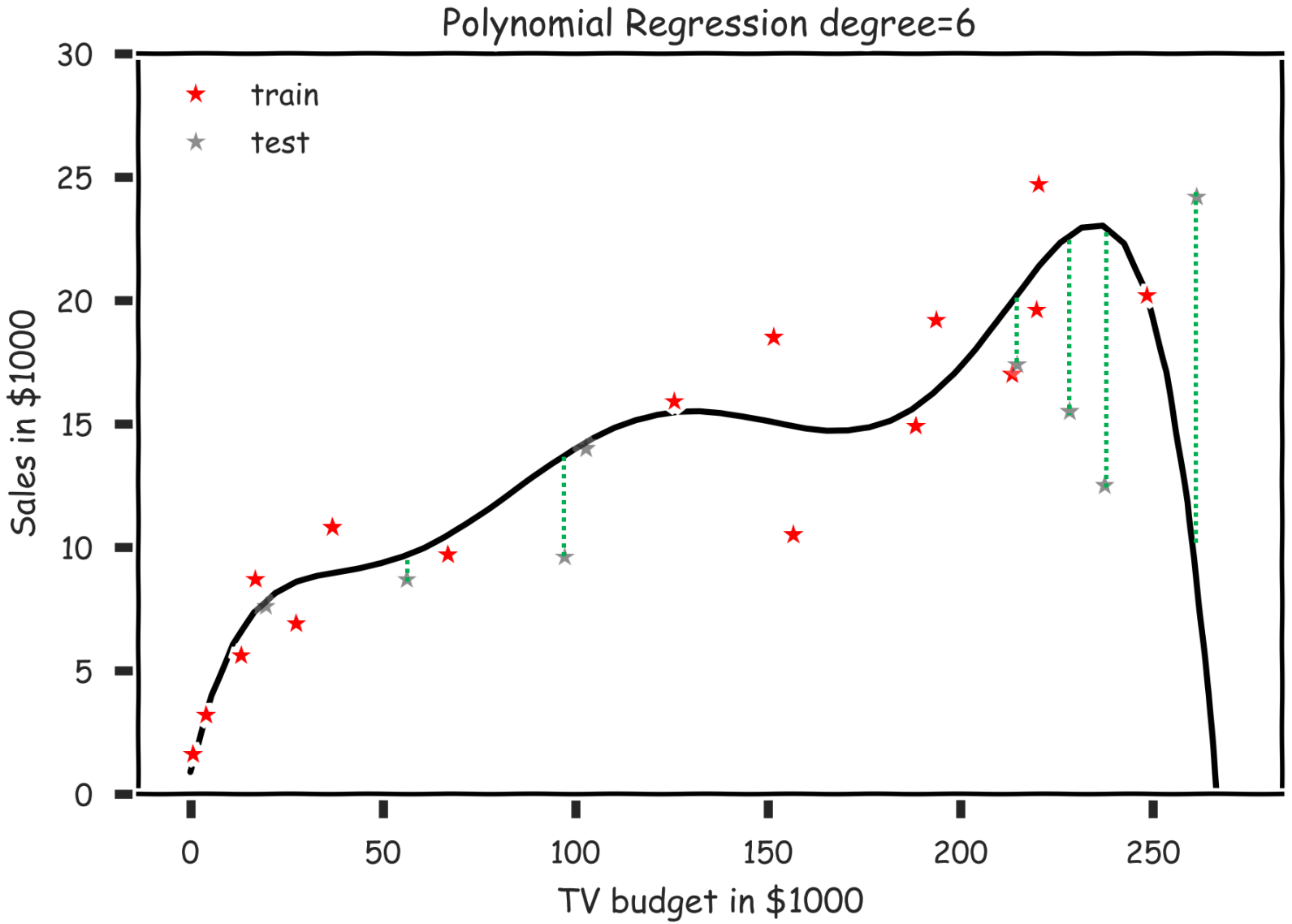
Polynomial Regression (cont)



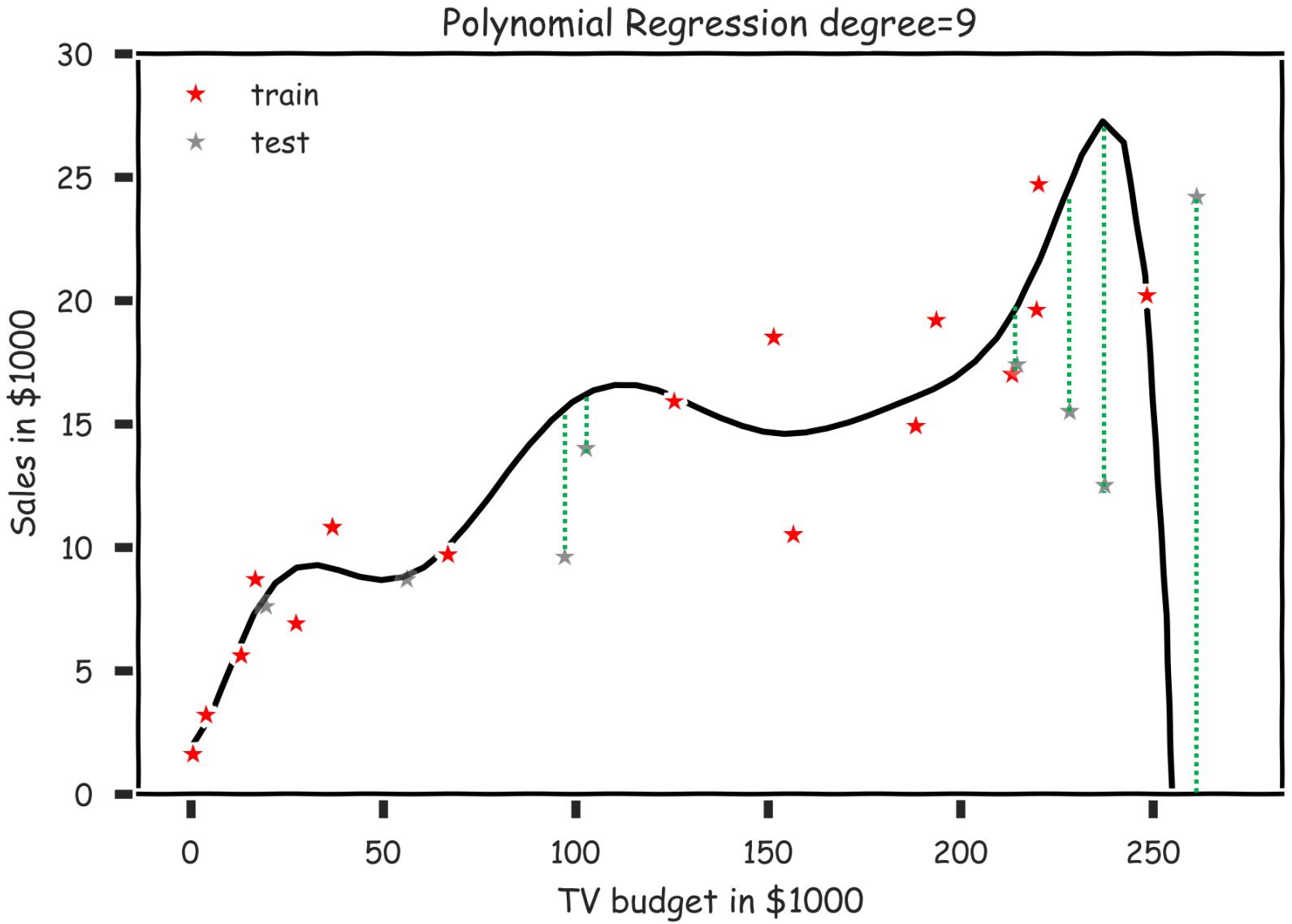
Polynomial Regression (cont)



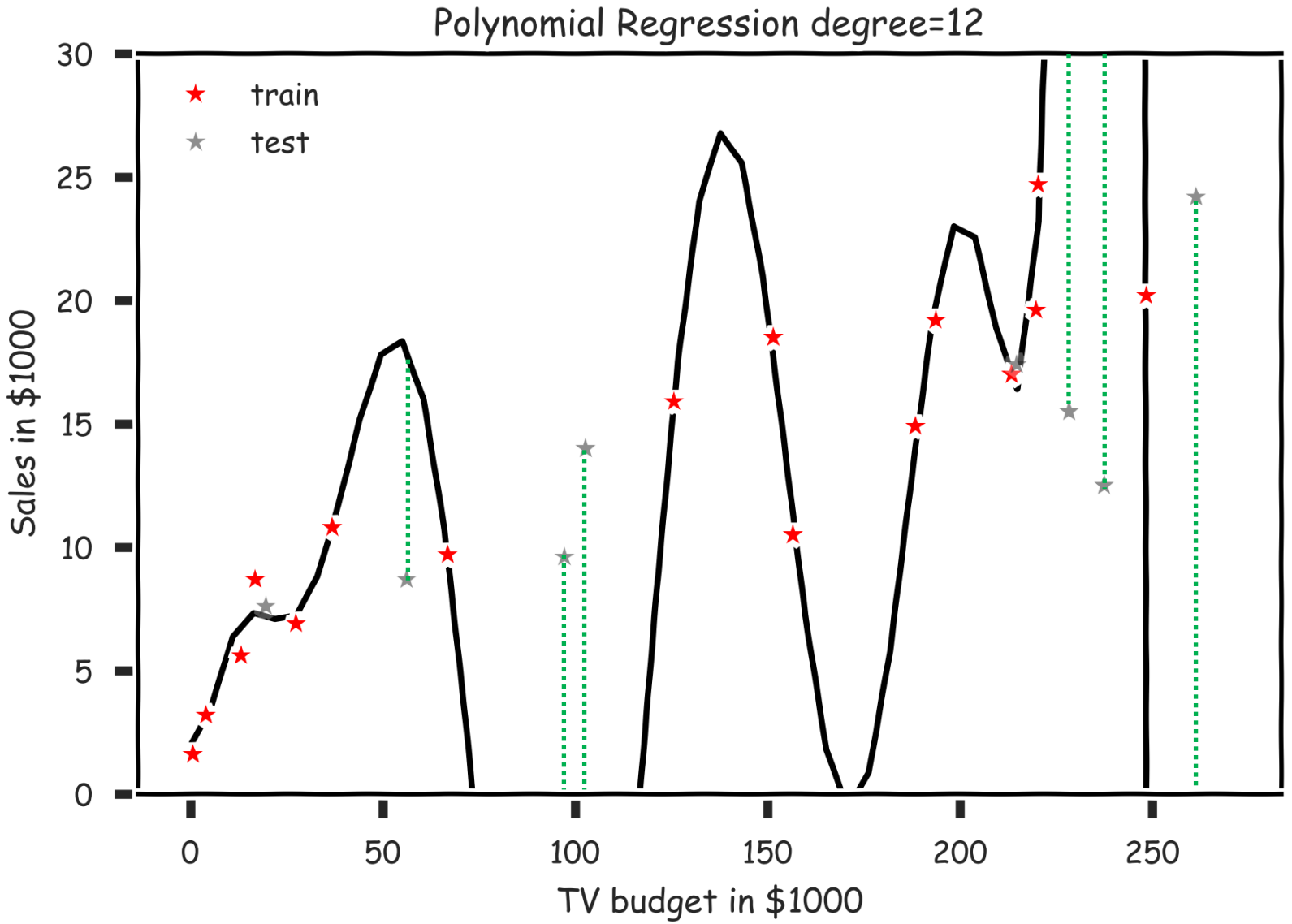
Polynomial Regression (cont)



Polynomial Regression (cont)

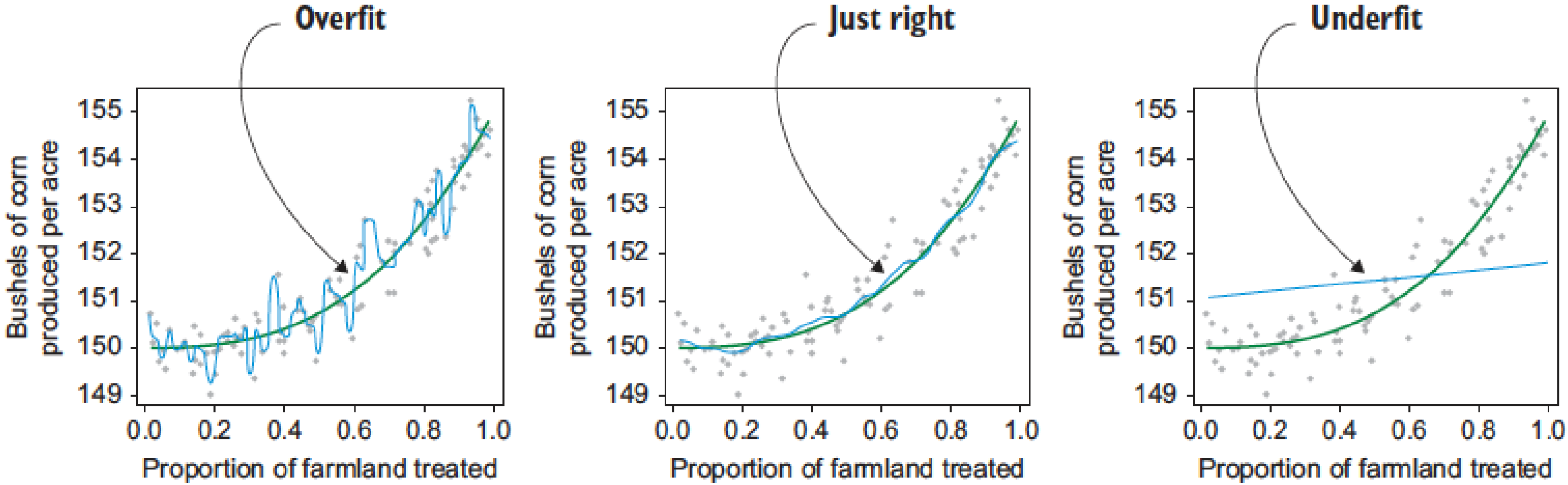


Polynomial Regression (cont)



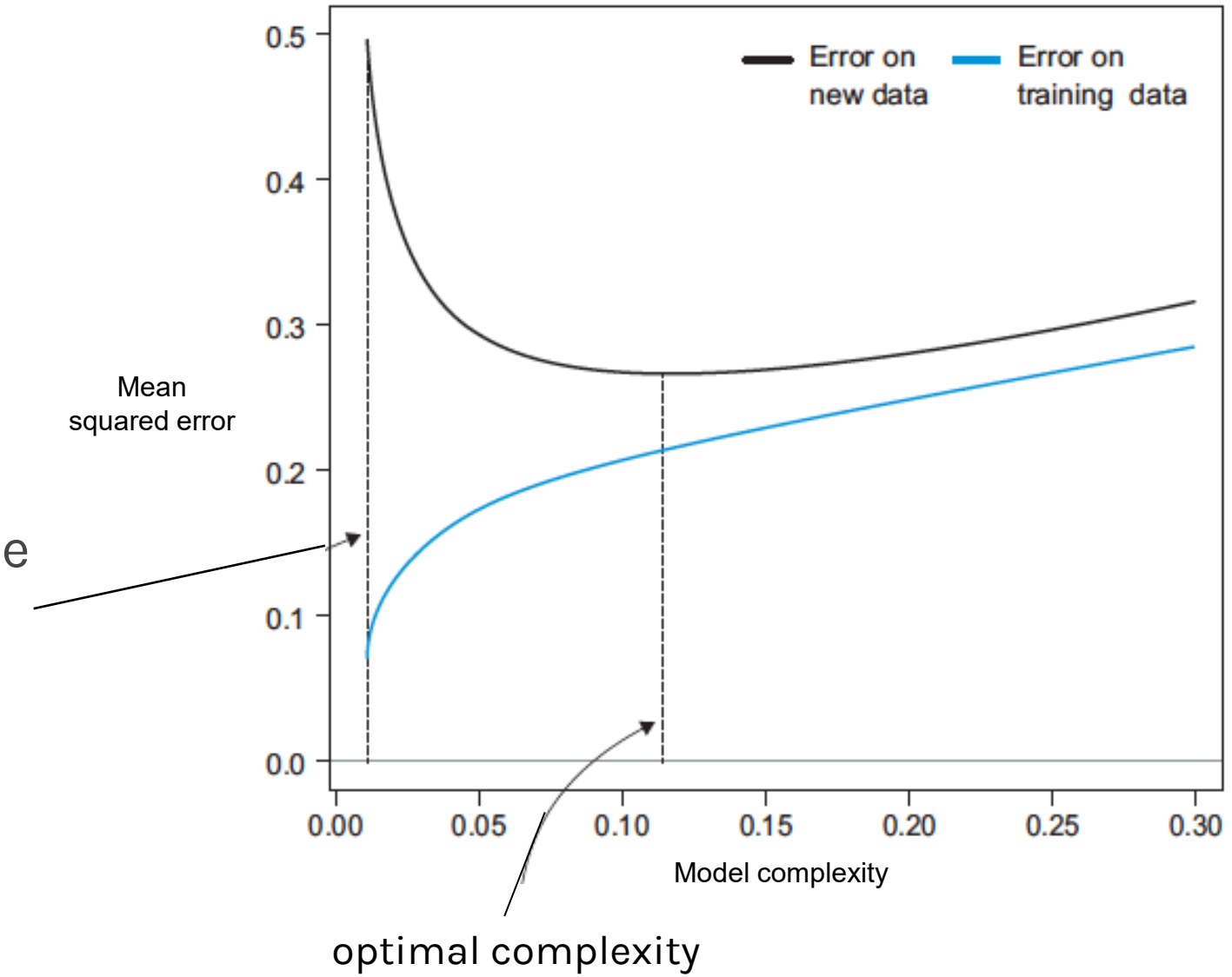
Overfitting

- Albert **Einstein:** Everything should be made as **simple** as possible, **but not simpler**.
- **overfitting** is "the production of an analysis that corresponds too closely or exactly to a particular set of data, and may therefore fail to fit additional data or predict future observations reliably"



Overfitting: Tune the complexity

- The training set error is an overly optimistic measure of the performance of the model for new data, particularly for simple models.
- Training set error should not be used as a surrogate for the prediction error on new data.

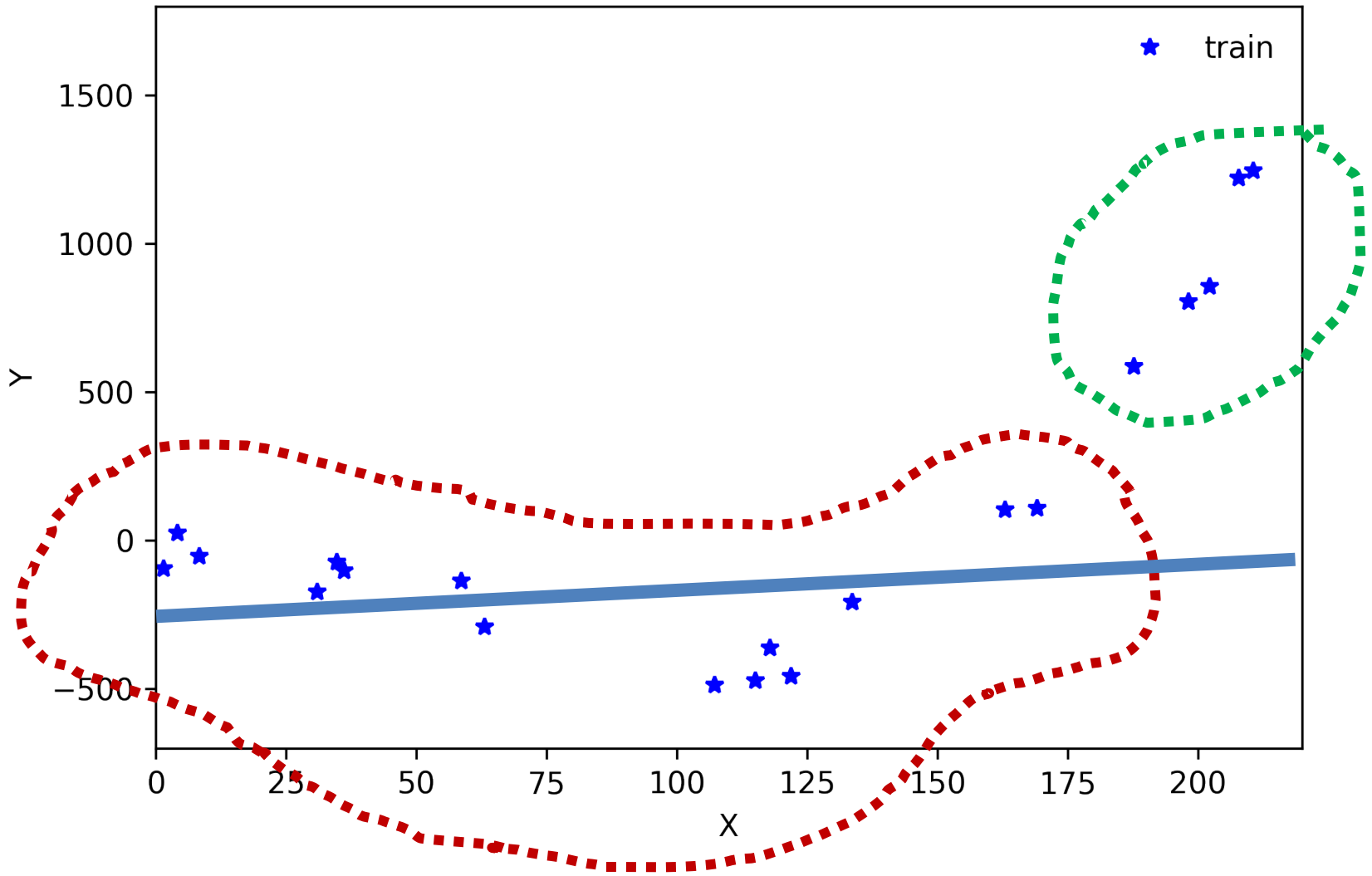


Model Selection

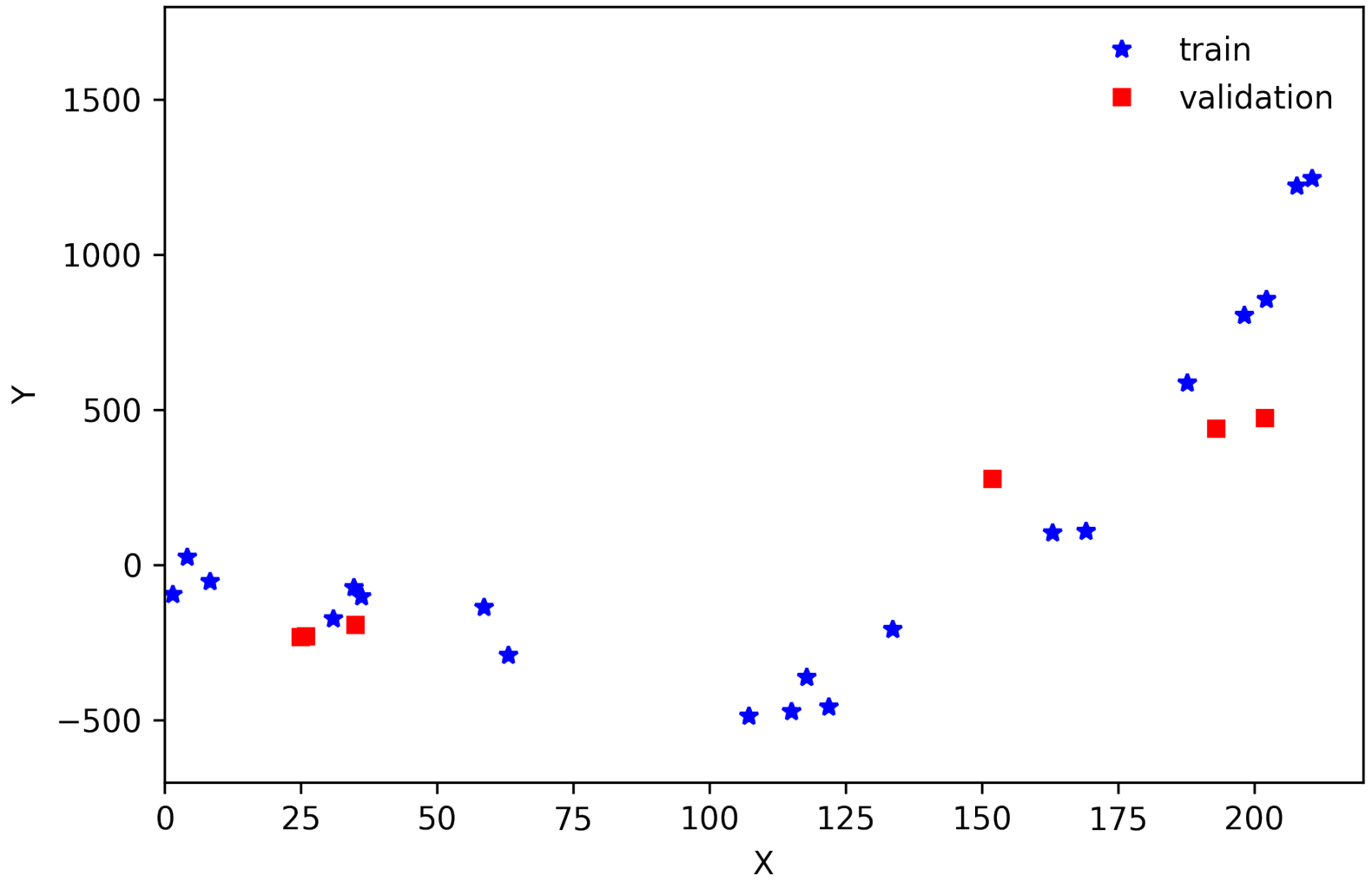
- **Model selection** is the application of a principled method to determine the complexity of the model, e.g. choosing a subset of predictors, choosing the degree of the polynomial model etc.

- A strong motivation for performing model selection is **to avoid overfitting**, which we saw can happen when:
  - there are too many predictors:
    - the feature space has high dimensionality
    - the polynomial degree is too high
    - too many cross terms are considered
  - the coefficients values are too **extreme (we have not seen this yet)**

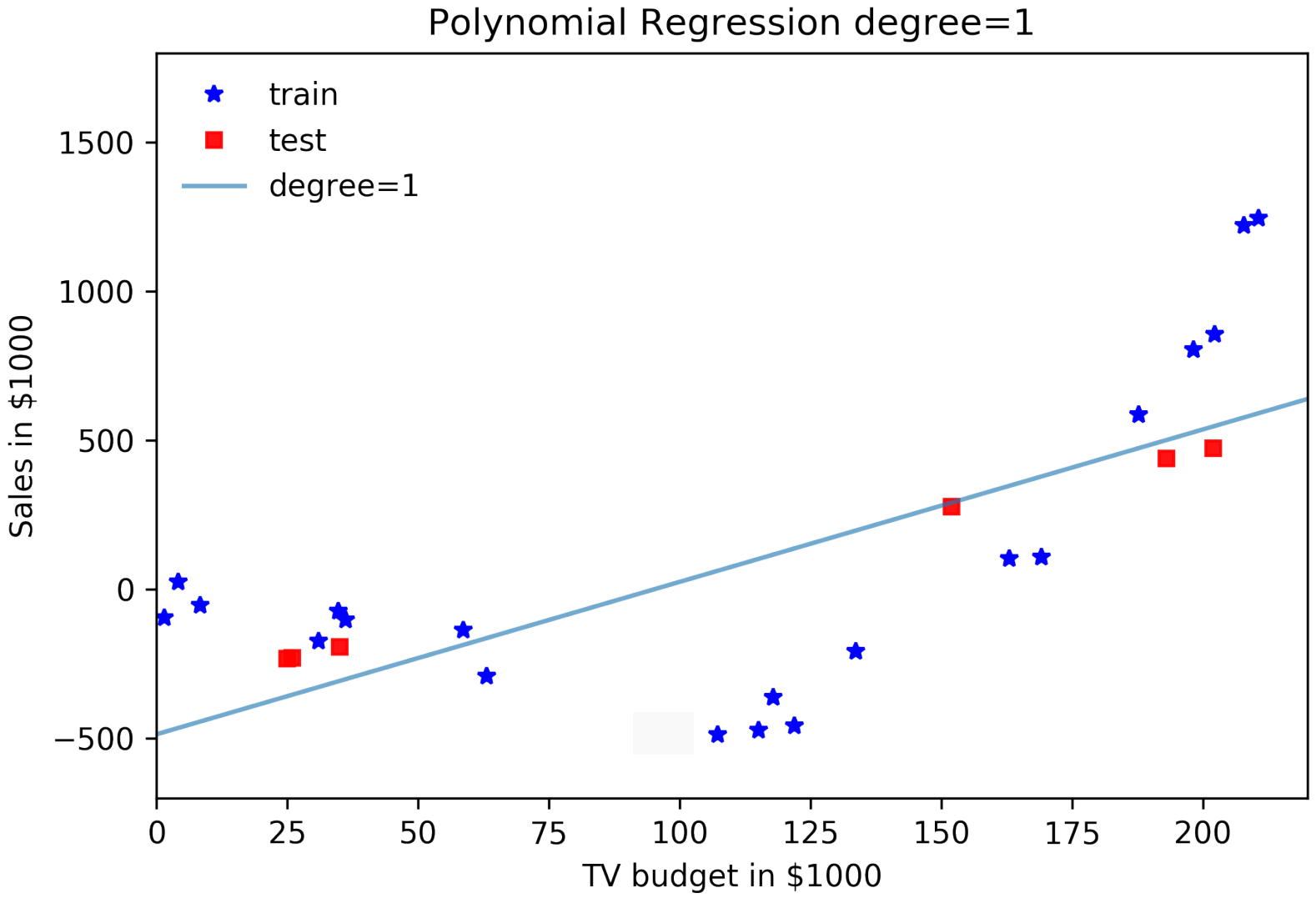
Cross Validation



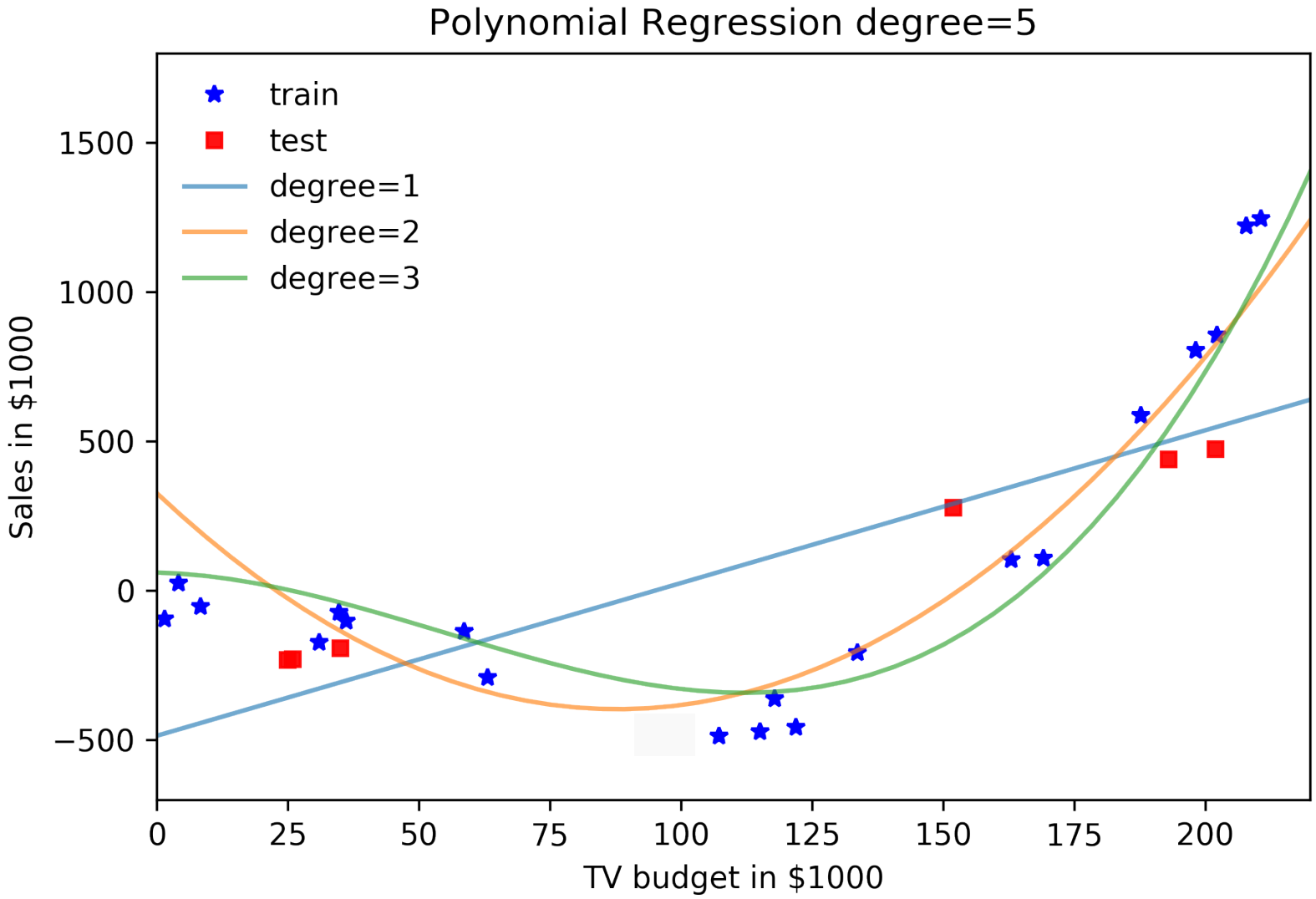
Cross Validation



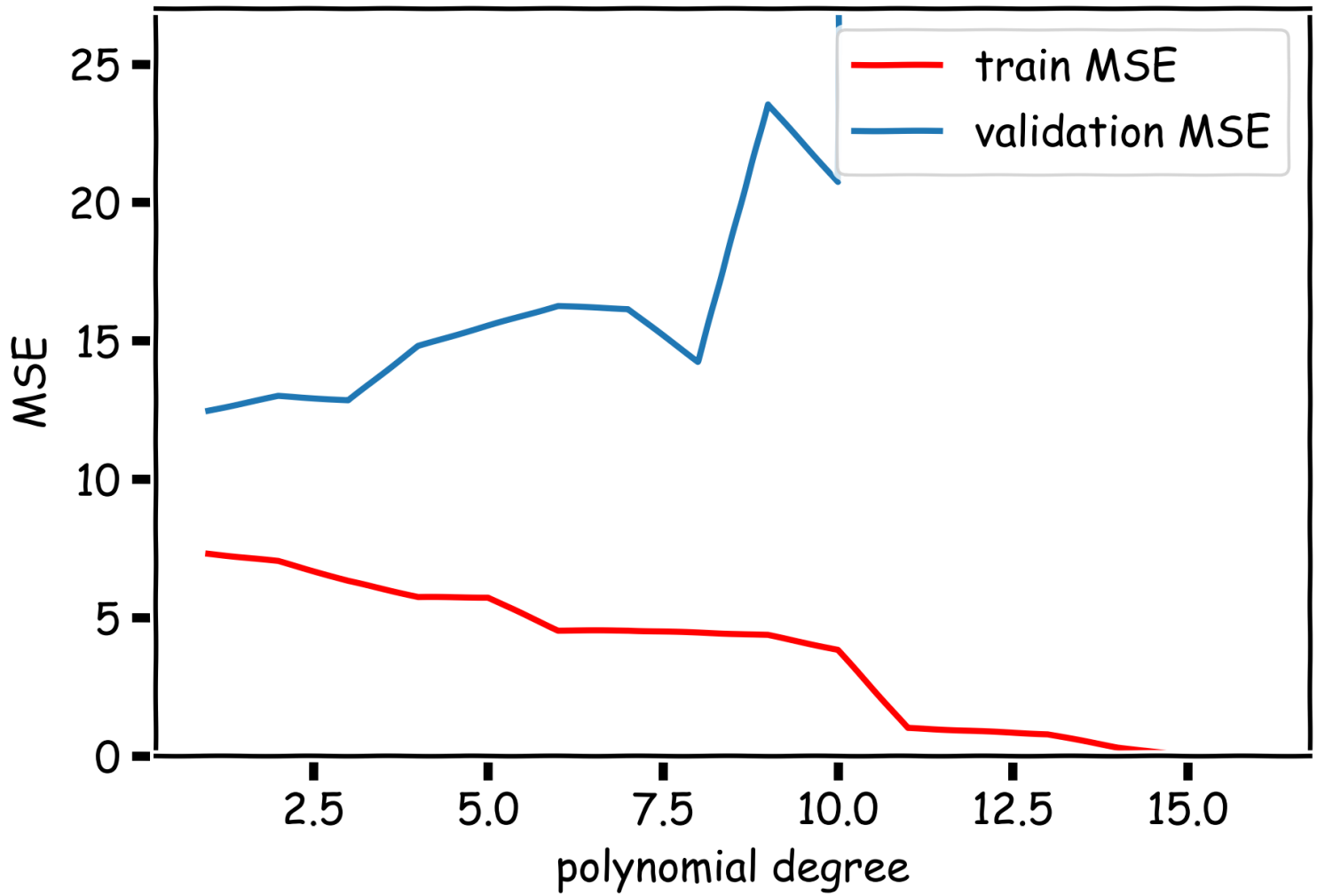
Cross Validation



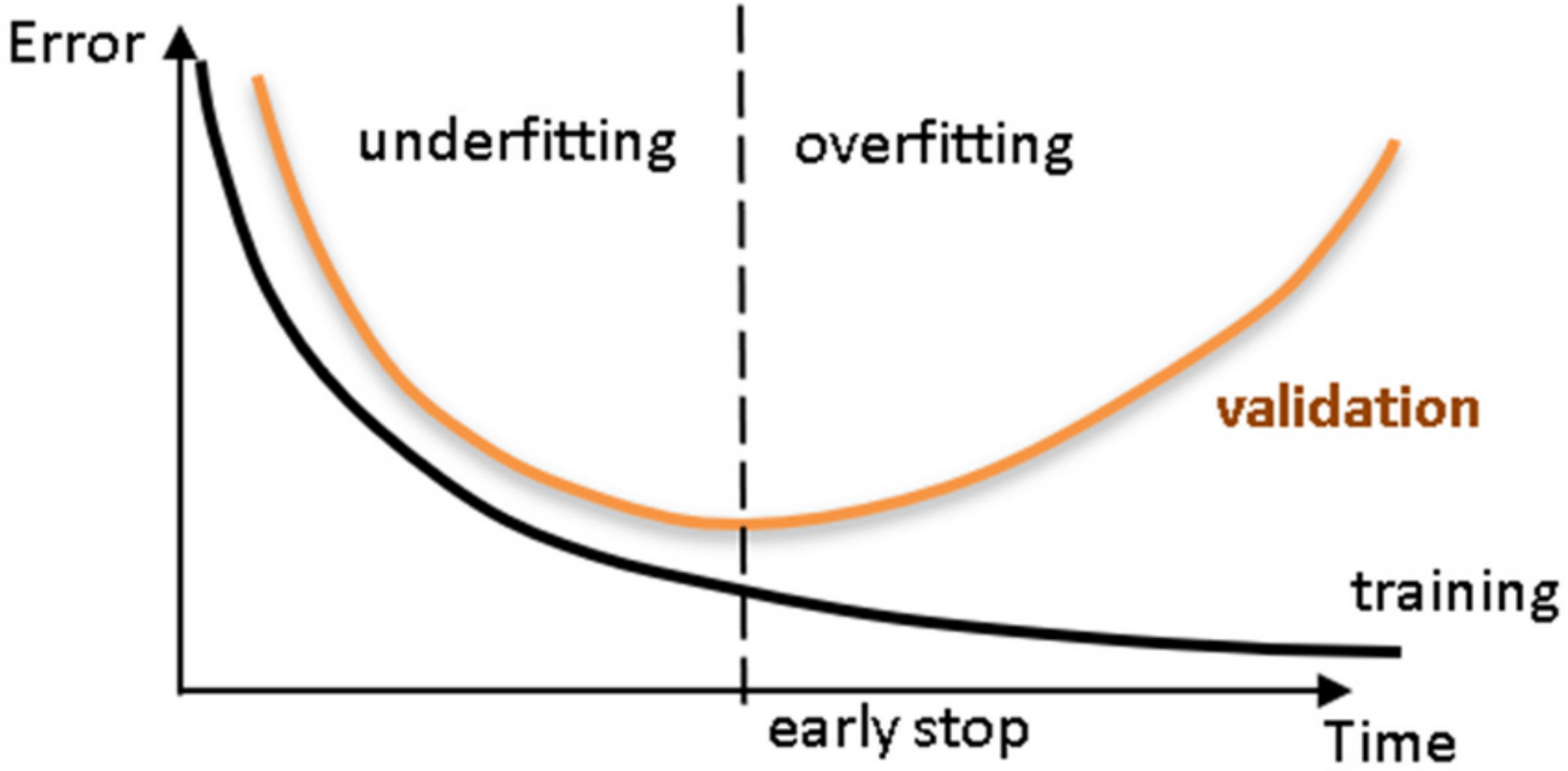
Cross Validation



Validation


train MSE
validation MSE
MSE
25
20
15
10
5
0
2.5
5.0
7.5
10.0
12.5
15.0
polynomial degree

Detection of overfitting

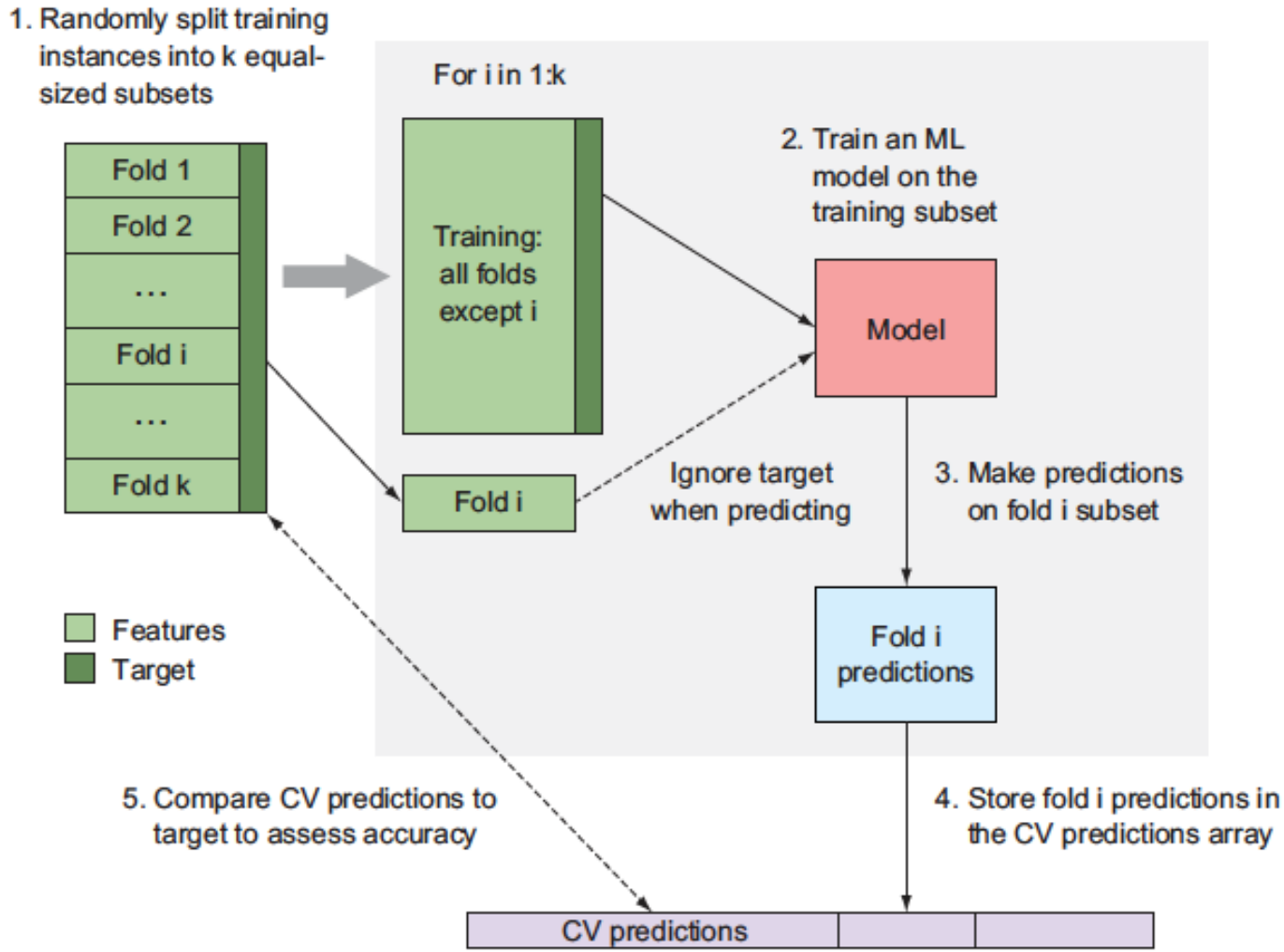


Cross Validation: Motivation

- Using a single validation set to select amongst multiple models can be problematic - **there is the possibility of overfitting to the validation set**.
- One solution to the problems raised by using a single validation set is to evaluate each model on **multiple** validation sets and average the validation performance.
- One can randomly split the training set into training and validation multiple times.
  - **But randomly** creating these sets can create the scenario where important features of the data never appear in our random draws.

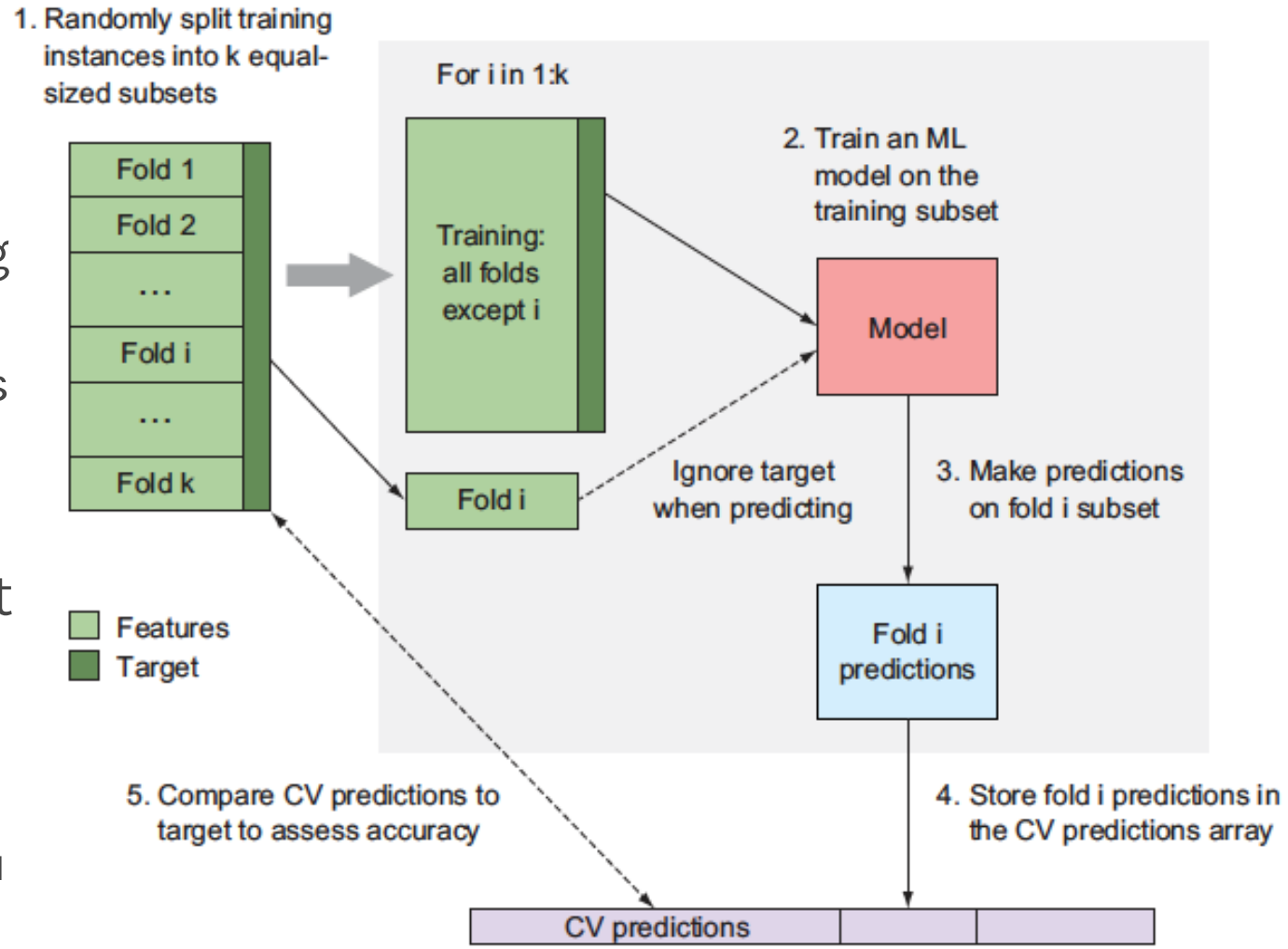
K-fold cross validation

- **K-fold cross validation** is one way to improve over the holdout method.
- The data set is divided into *k* subsets, and the holdout method is repeated *k* times.
- Each time, one of the *k* subsets is used as the test set and the other *k-1* subsets are put together to form a training set. Then the average error across all *k* trials is computed.

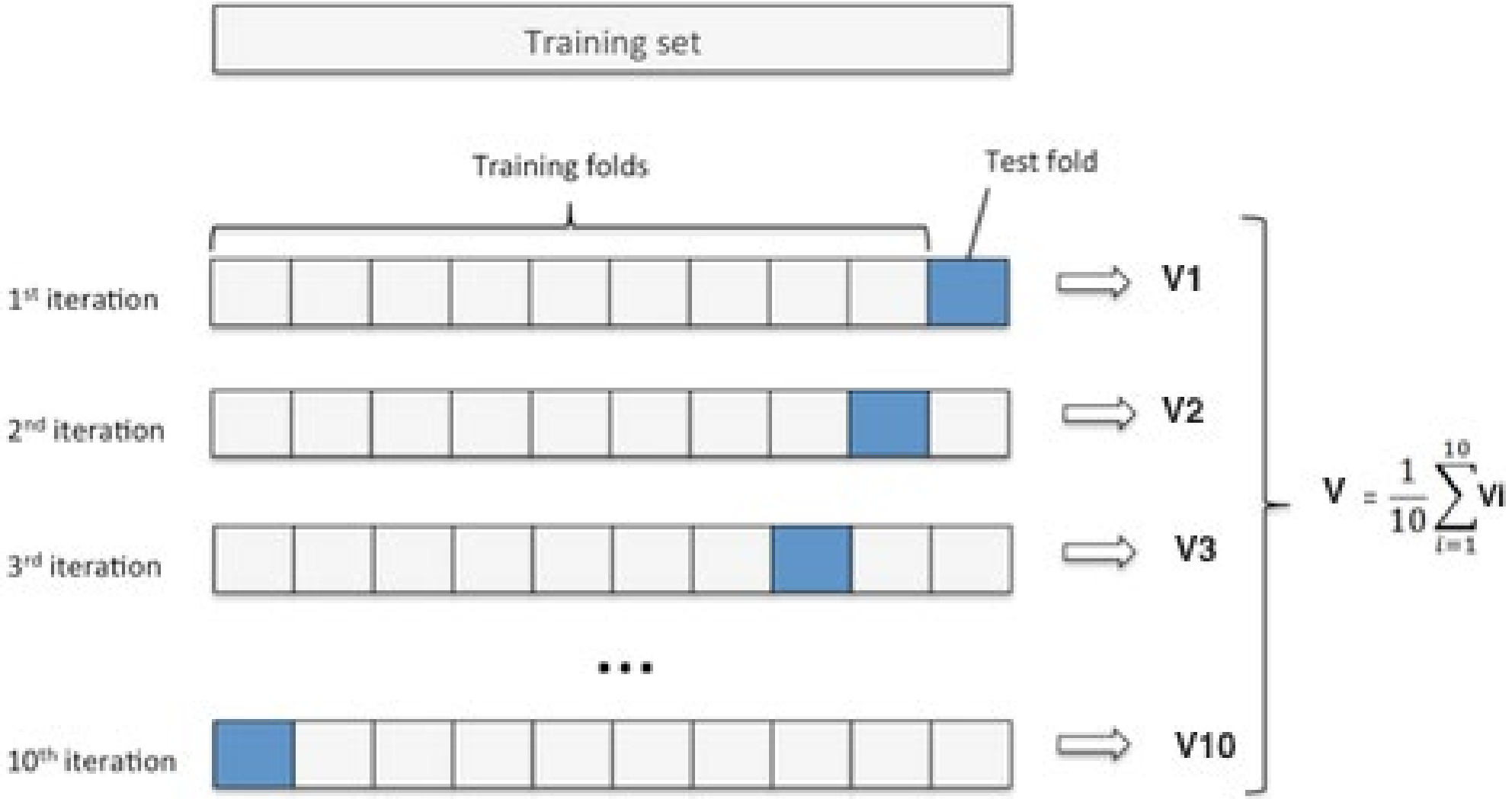


K-fold cross validation

- The advantage of this method is that it matters less how the data gets divided.
  - Every data point gets to be in a test set exactly once, and gets to be in a training set *k-1* times.
  - The variance of the resulting estimate is reduced as *k* is increased.
- A variant of this method is to randomly divide the data into a test and training set *k* different times.
  - The advantage of doing this is that you can independently choose how large each test set is and how many trials you average over.



Cross Validation



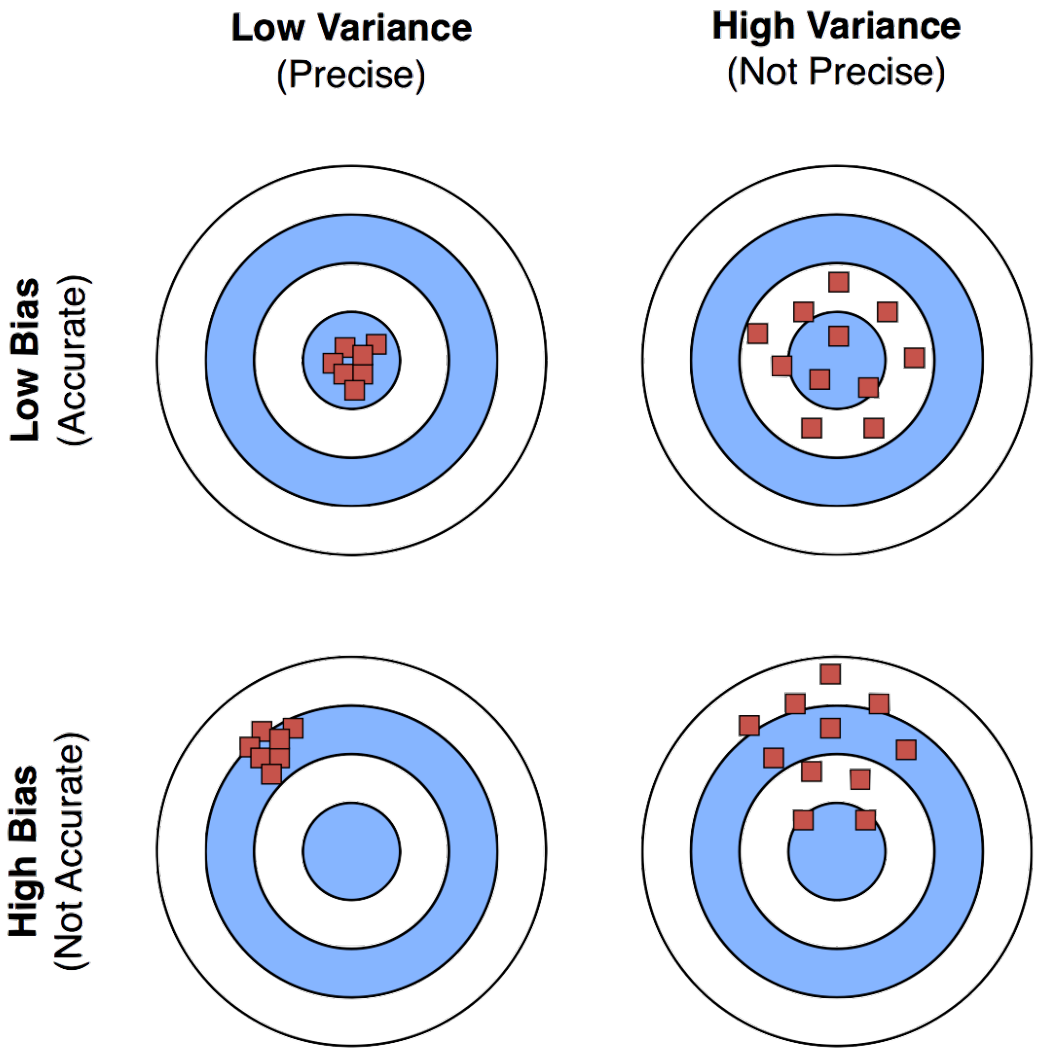
K-Fold Cross Validation

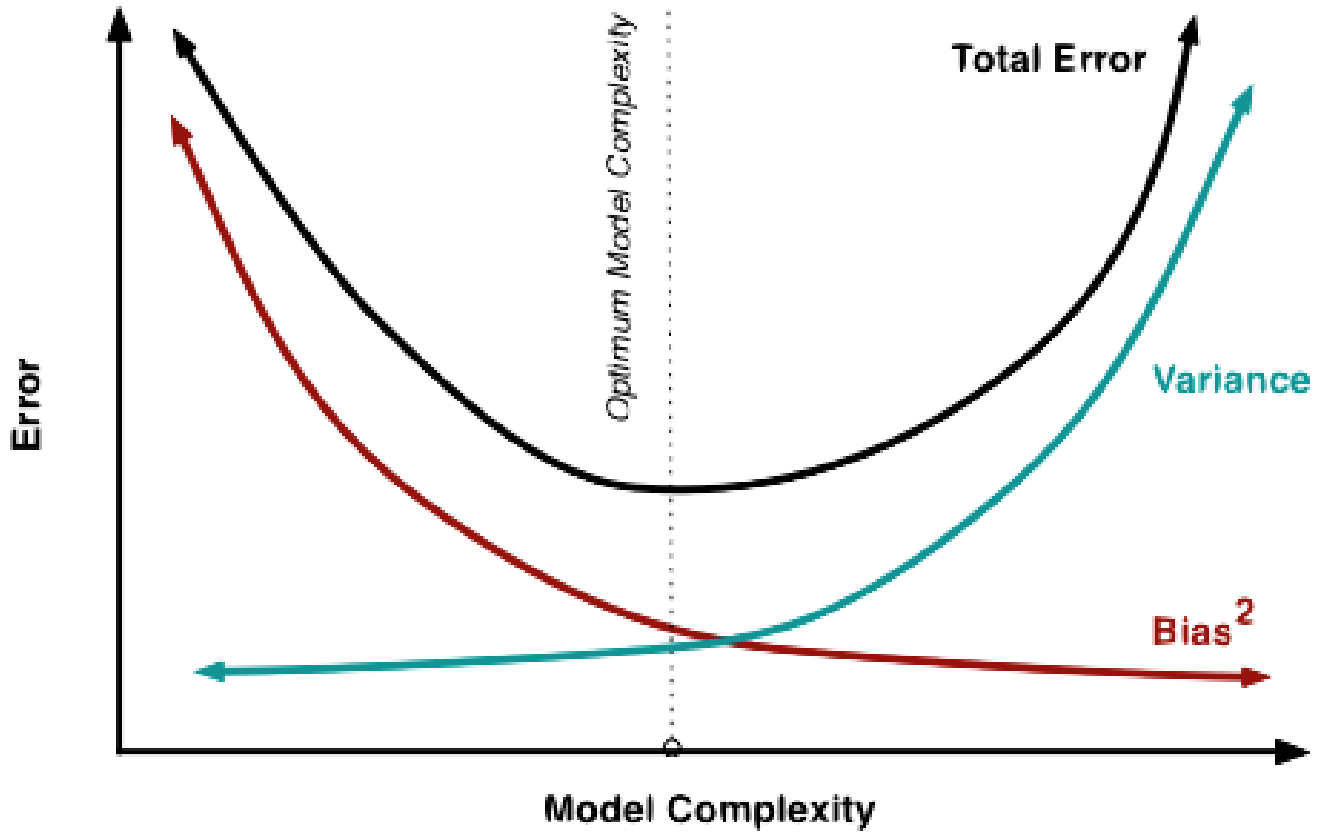
- Given a data set $\{X_1, \dots, X_n\}$, where each $\{X_1, \dots, X_n\}$ contains *J* features.
- To ensure that every observation in the dataset is included in at least one training set and at least one validation set we use the **K-fold validation**:
  - split the data into *K* uniformly sized chunks, $\{C_1, \dots, C_K\}$
  - we create *K* number of training/validation splits, using one of the *K* chunks for validation and the rest for training.
- We fit the model on each training set, denoted $\hat{f}_{C_{-i}}$, and evaluate it on the corresponding validation set, $\hat{f}_{C_{-i}}(C_i)$. The ***cross validation is the performance*** of the model averaged across all validation sets:

$$CV(\text{Model}) = \frac{1}{K}\sum_{i=1}^{K} L(\hat{f}_{C_{-i}}(C_i))$$

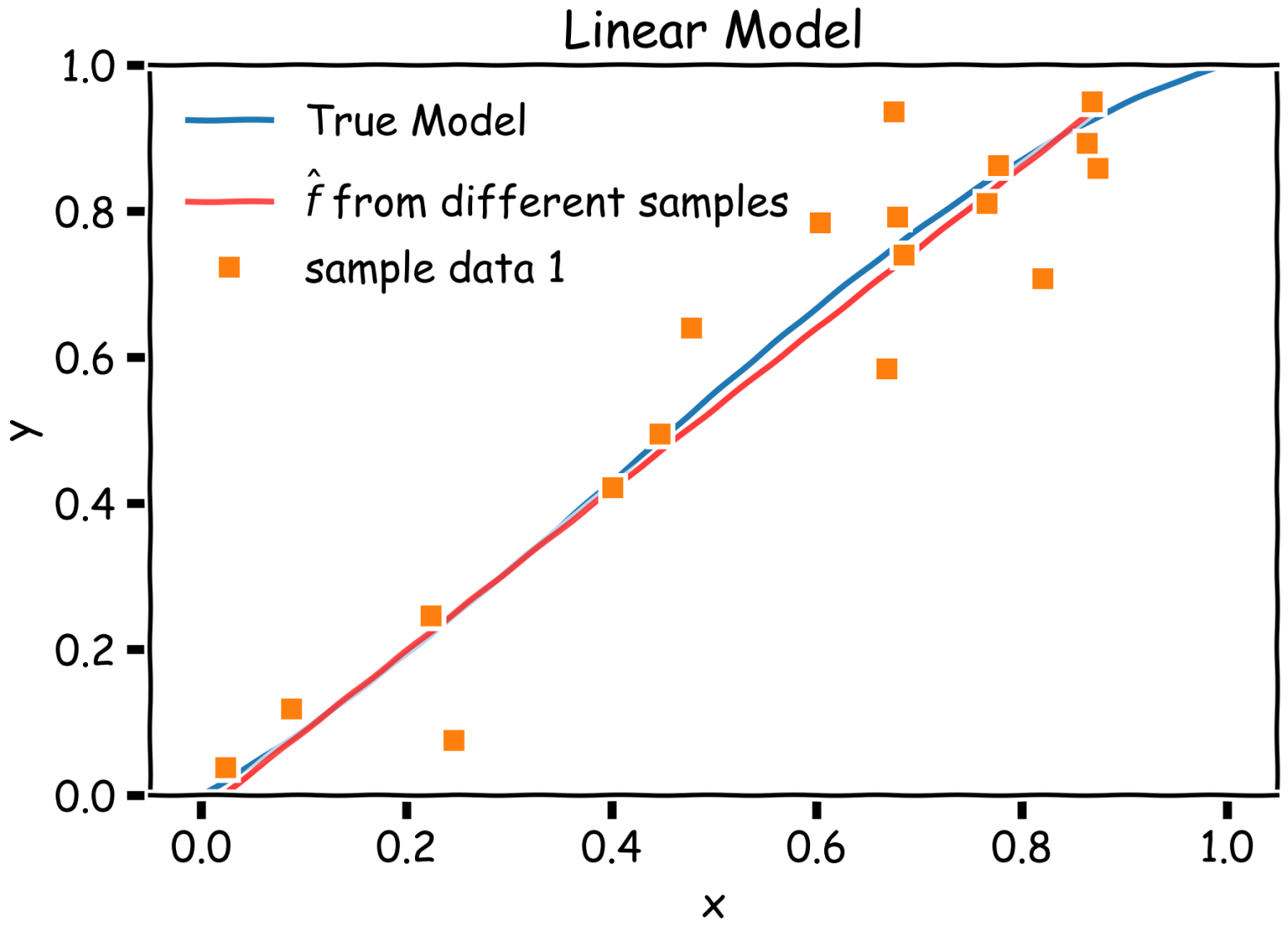
- where *L* is a loss function

Bias vs Variance

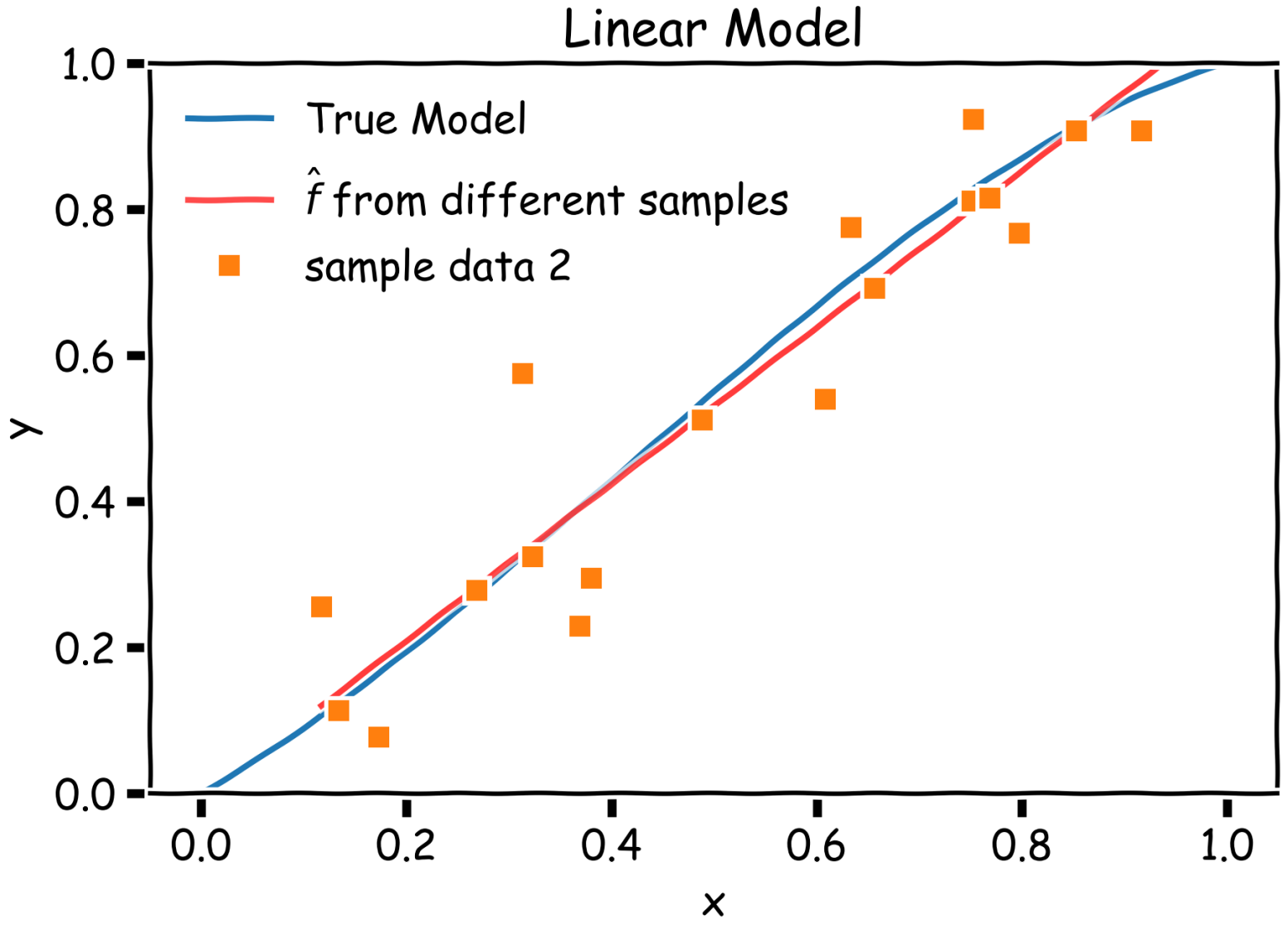




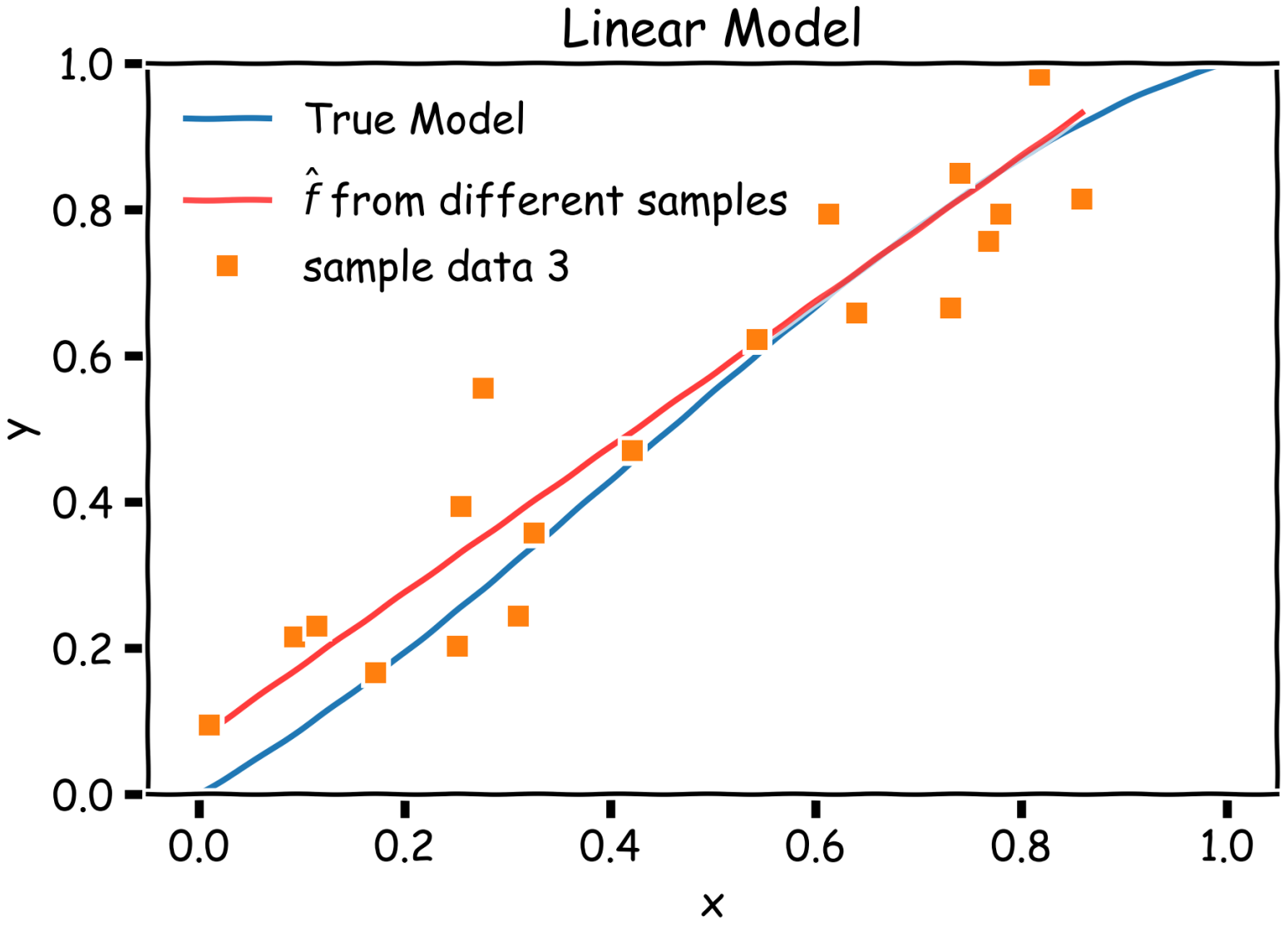
Bias vs Variance



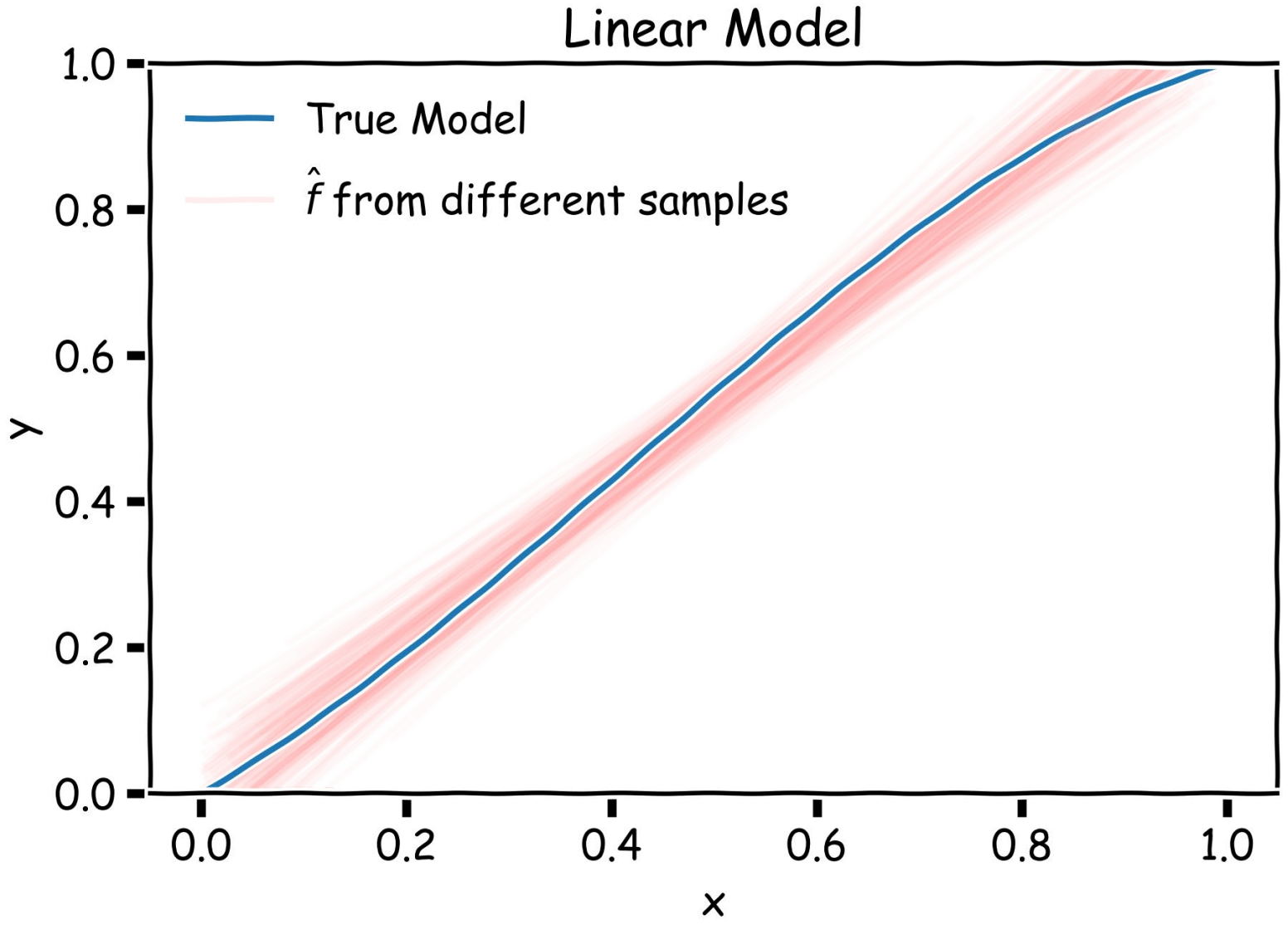
Bias vs Variance



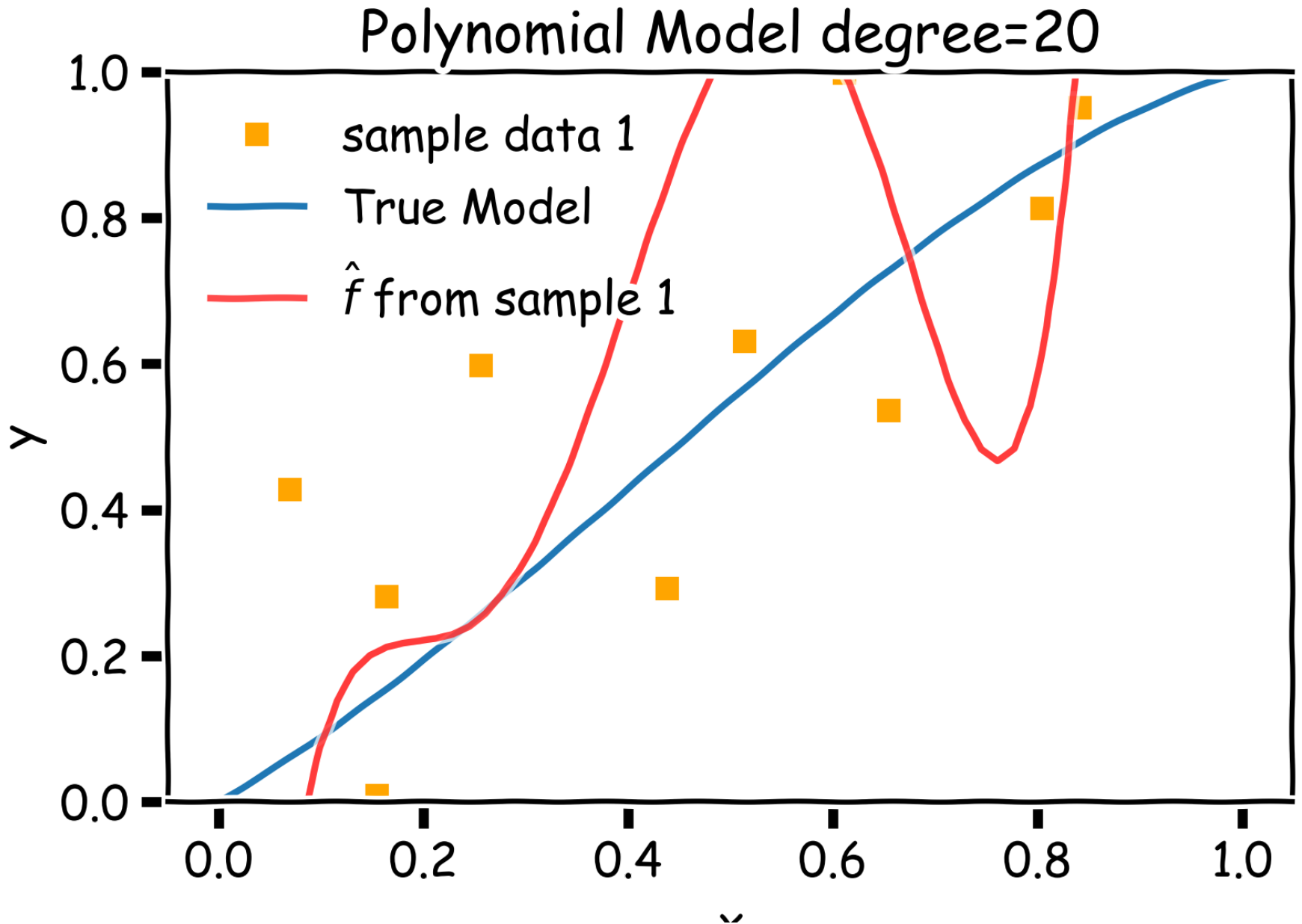
Bias vs Variance



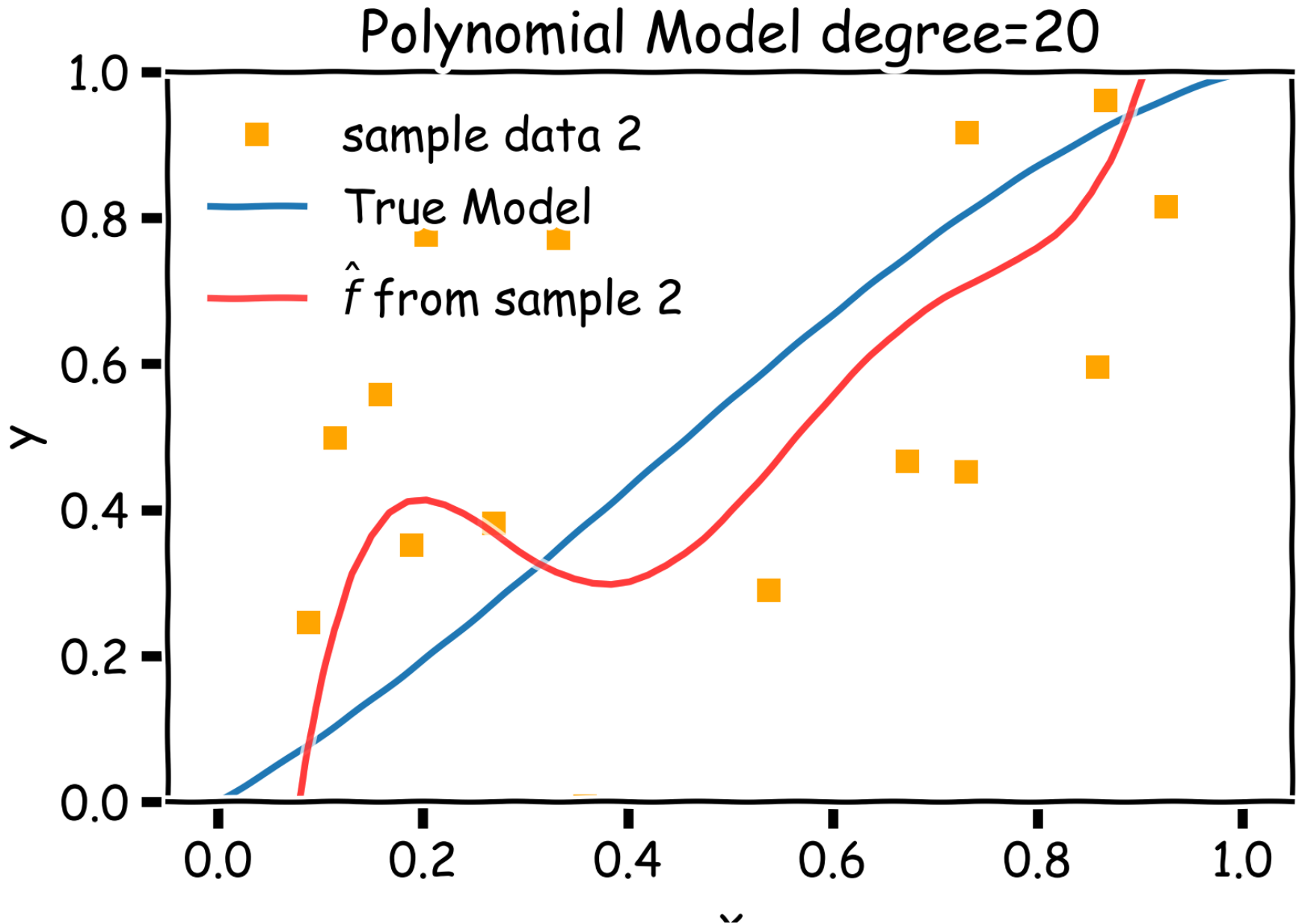
Linear models: … after 2000 simulations



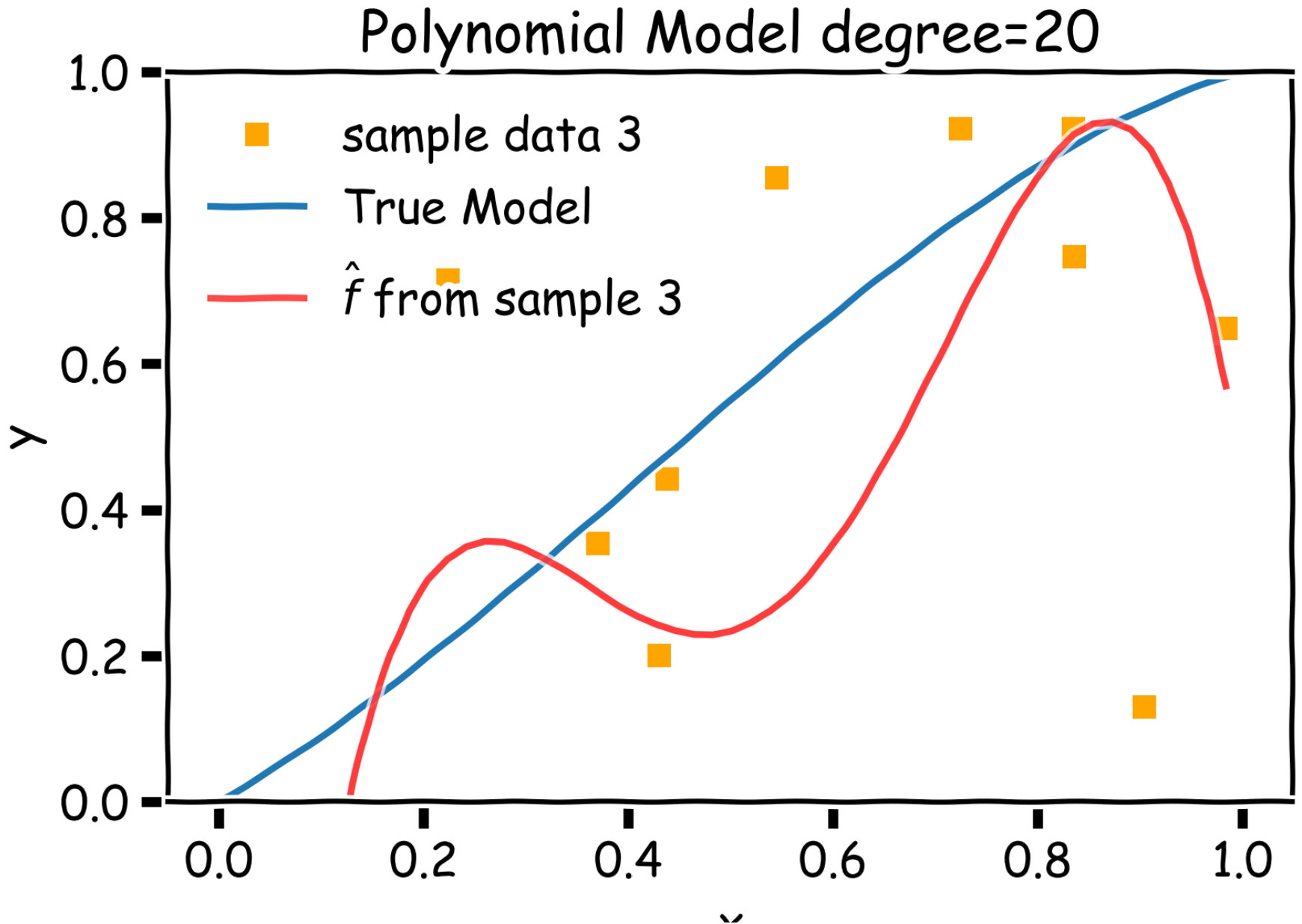
Bias vs Variance



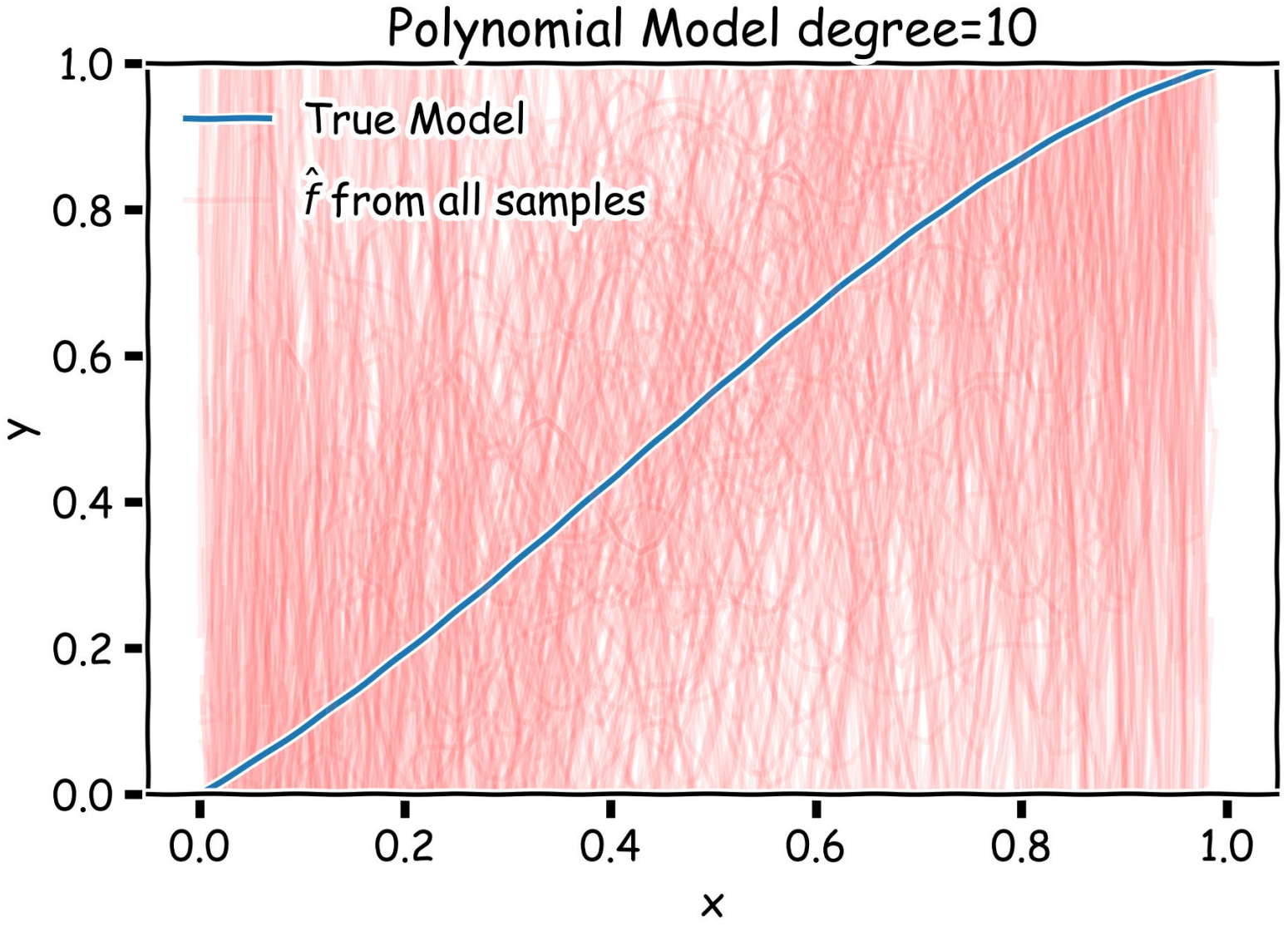
Bias vs Variance



Bias vs Variance

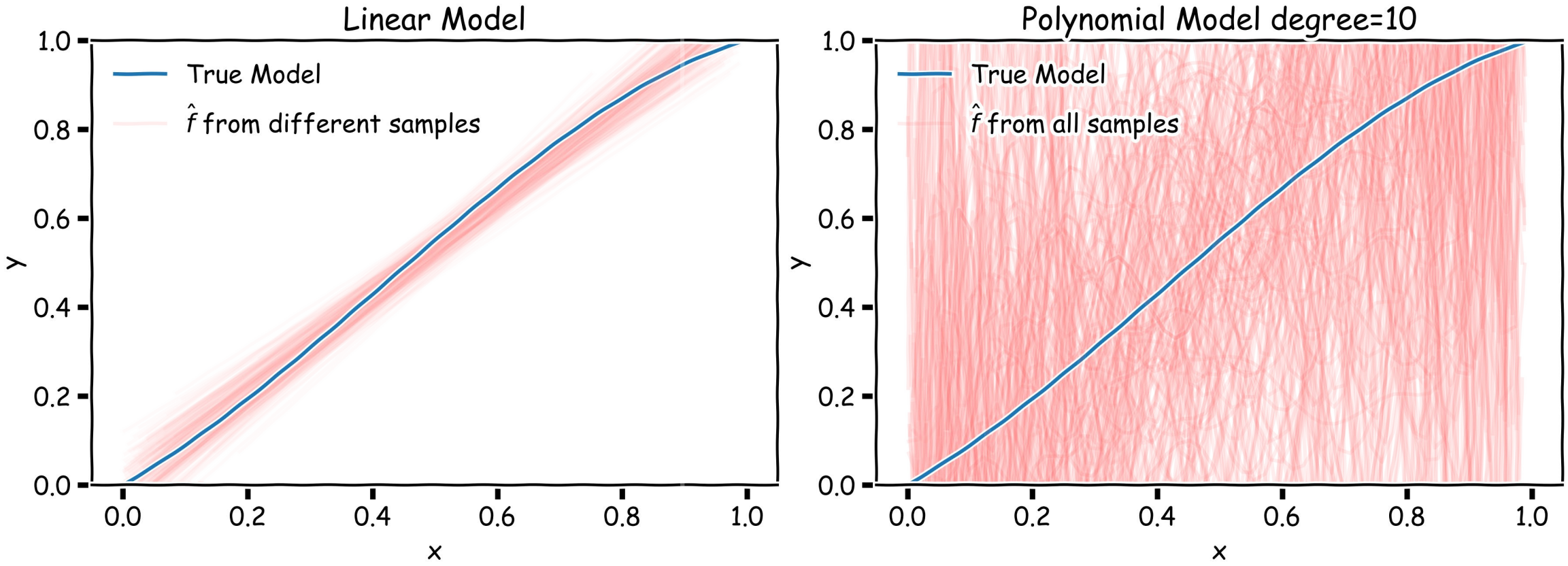


Poly 10-degree models: … after 2000 simulations



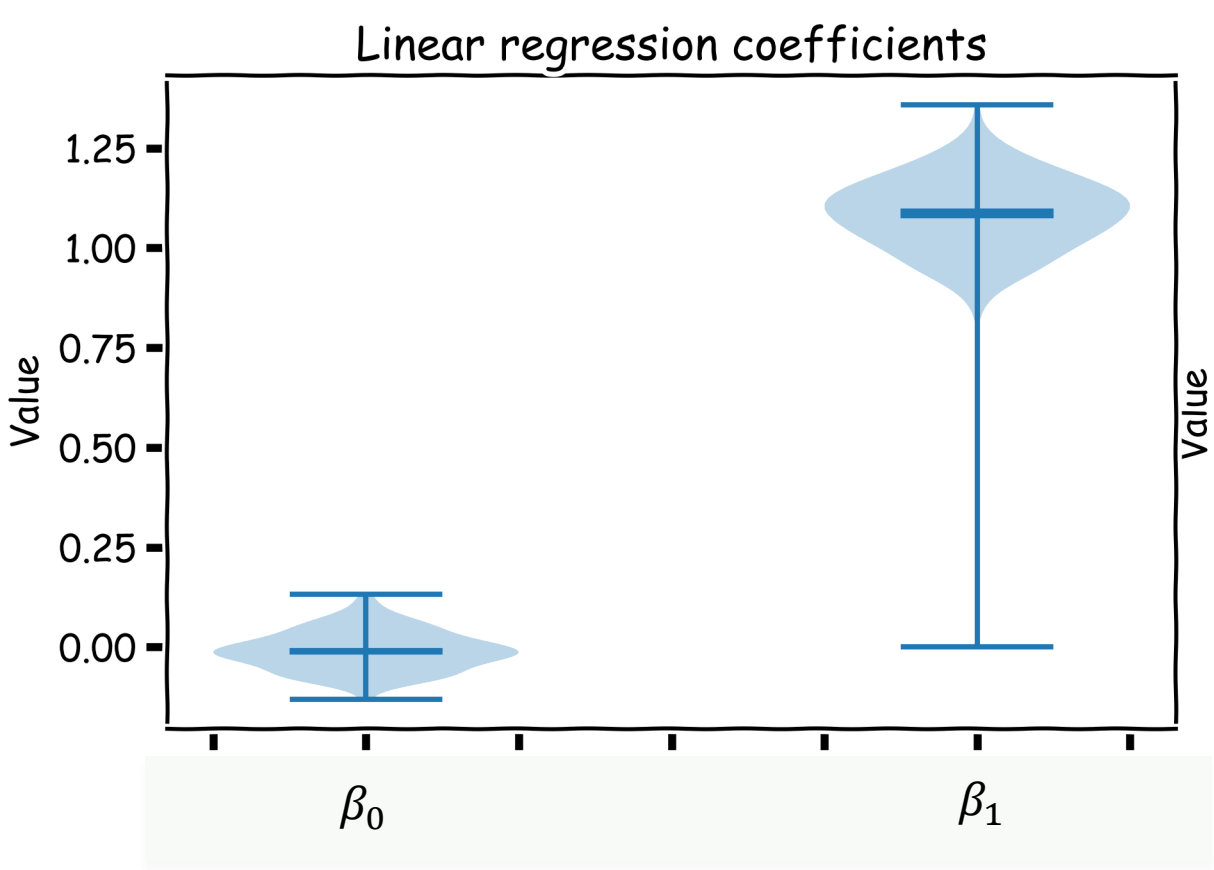
Bias vs Variance

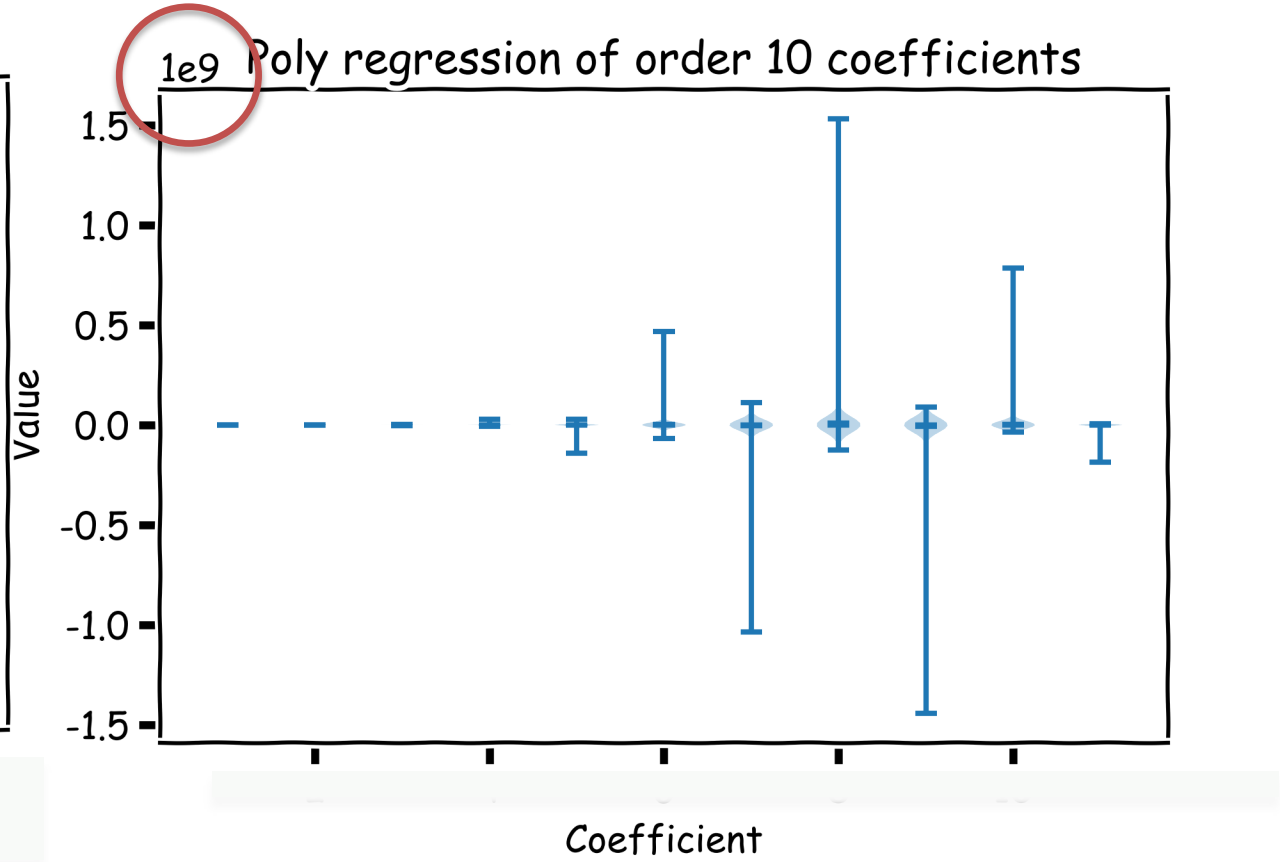
- **Left**: 2000 best-fit straight lines using linear regression
- **Right**: 2000 best-fit models using degree 10 polynomial



Bias vs Variance

- Left: Coefficients of linear regression
- Right: Coefficients of poly regression of order 10





Regularization: An Overview

- The idea of regularization revolves around modifying the loss function L; in particular, we add a regularization term that **penalizes some specified properties of the model parameters**

$$L_{reg}(\beta) = L(\beta) + \lambda R(\beta),$$

- where λ is a scalar that gives the weight (or importance) of the regularization term.
- Fitting the model using the modified loss function L_{reg} would result in model parameters with desirable properties (specified by R).

LASSO Regression

- Since we wish to discourage extreme values in model parameter, we need to choose a regularization term that penalizes parameter magnitudes. For our loss function, we will again use MSE.
- Together our regularized loss function is:

$$L_{LASSO}(\beta) = \frac{1}{n}\sum_{i=1}^{n}|y_i - \boldsymbol{\beta}^\top \boldsymbol{x}_i|^2 + \lambda \sum_{j=1}^{J}|\beta_j|.$$

- Note that $\sum_{j=1}^{J}|\beta_j|$ is the l_1 norm of the vector $\boldsymbol{\beta}$

$$\sum_{j=1}^{J}|\beta_j| = \|\boldsymbol{\beta}\|_1$$

LASSO Regression

- Hence, we often say that L_{LASSO} is the loss function for l_1 regularization.
- Finding the model parameters β_{LASSO} that minimize the l_1 regularized loss function is called ***LASSO regression***.

In [ ]:
```
from sklearn.linear_model import Lasso
```

In [22]:
```
lasso_regression = Lasso(alpha=1.0, fit_intercept=True)
lasso_regression.fit(np.vstack((X_train, X_val)), np.hstack((y_train, y_val)))

print('Lasso regression model:\n {} + {}^T . x'.format(lasso_regression.intercept_, lasso_regression.coe
```

```
Lasso regression model:
 10.424895873901445 + [ 0.24482603  3.48164594  1.84836859 -0.06864603 -0.         -0.
 -0.02249766 -0.          0.          0.          0.          ]^T . x
```

In [23]:
```
print('Train R^2: {}, test R^2: {}'.format(lasso_regression.score(np.vstack((X_train, X_val)),
                                                                  np.hstack((y_train, y_val))),
                                           lasso_regression.score(X_test, y_test)))
```

```
Train R^2: 0.48154992527975765, test R^2: 0.6846451270316087
```

Ridge Regression

- Alternatively, we can choose a regularization term that penalizes the squares of the parameter magnitudes. Then, our regularized loss function is:

$$L_{Ridge}(\beta) = \frac{1}{n}\sum_{i=1}^{n} |y_i - \boldsymbol{\beta}^\top \boldsymbol{x}_i|^2 + \lambda \sum_{j=1}^{J} \beta_j^2.$$

- Note that $\sum_{j=1}^{J} |\beta_j|^2$ is the $\boldsymbol{l_2}$ norm of the vector $\boldsymbol{\beta}$

$$\sum_{j=1}^{J} \beta_j^2 = \|\boldsymbol{\beta}\|_2^2$$

Ridge Regression

- Hence, we often say that L_{ridge} is the loss function for l_2 regularization.
- Finding the model parameters β_{ridge} that minimize the l_2 regularized loss function is called *ridge regression.*

In [ ]:
```
from sklearn.linear_model import Ridge
```

In [20]:
```
X_train = train[all_predictors].values
X_val = validation[all_predictors].values
X_test = test[all_predictors].values

ridge_regression = Ridge(alpha=1.0, fit_intercept=True)
ridge_regression.fit(np.vstack((X_train, X_val)), np.hstack((y_train, y_val)))

print('Ridge regression model:\n {} + {}^T . x'.format(ridge_regression.intercept_, ridge_regression.coe
```

```
Ridge regression model:
 -525.7662550875951 + [ 0.24007312  8.42566029  2.04098593 -0.04449172 -0.01227935  0.41902475
 -0.50397312 -4.47065168  4.99834262  0.          0.          0.29892679]^T . x
```

In [21]:
```
print('Train R^2: {}, test R^2: {}'.format(ridge_regression.score(np.vstack((X_train, X_val)),
                                                                  np.hstack((y_train, y_val))),
                                           ridge_regression.score(X_test, y_test)))
```

```
Train R^2: 0.5319764744847737, test R^2: 0.7881798111697319
```

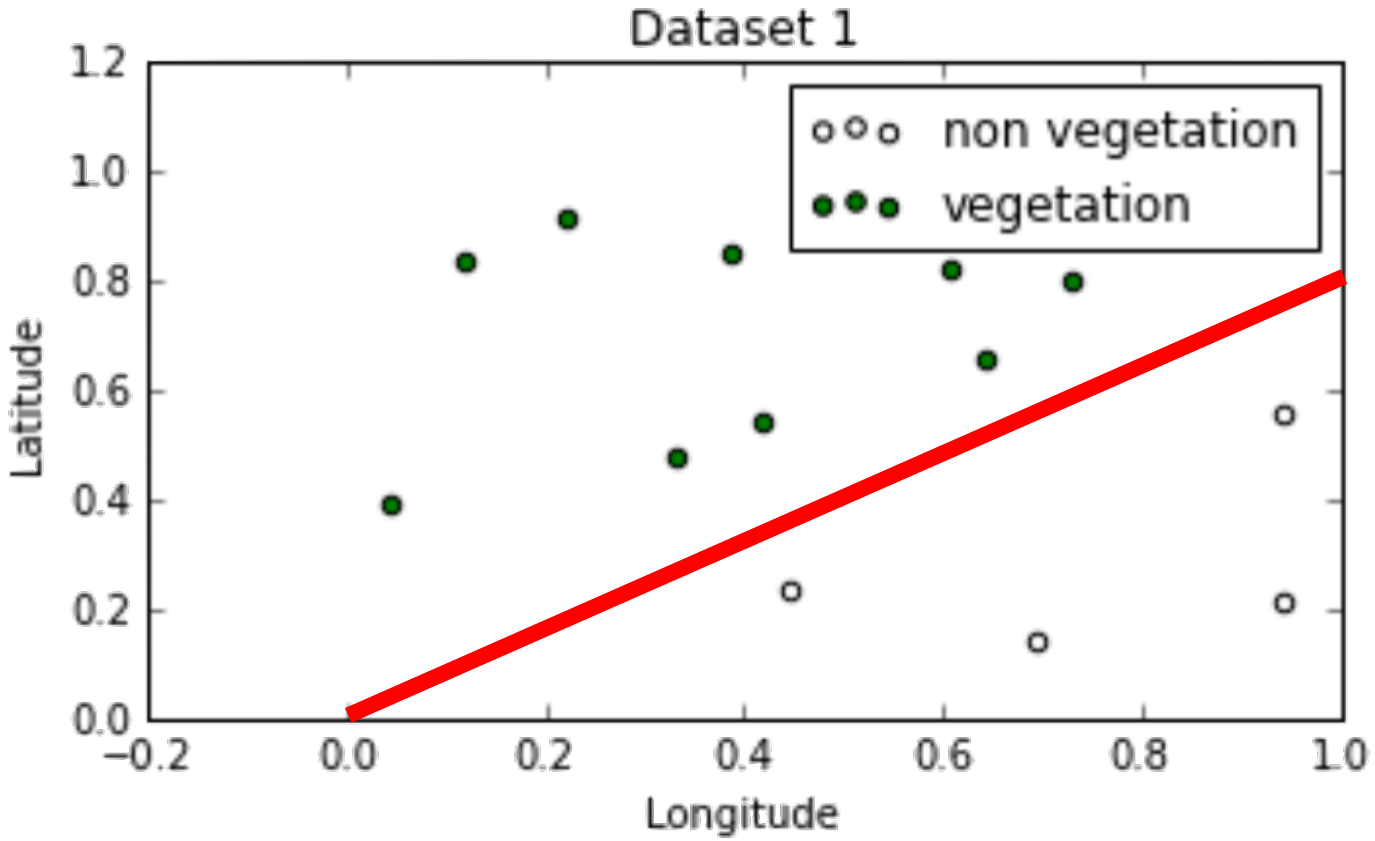
Choosing λ

- In both ridge and LASSO regression, we see that the larger our choice of the **regularization parameter** λ, the more heavily we penalize large values in $\boldsymbol{\beta}$,
  - If λ is close to zero, we recover the MSE, i.e. ridge and LASSO regression is just ordinary regression.
  - If λ is sufficiently large, the MSE term in the regularized loss function will be insignificant and the regularization term will force $\boldsymbol{\beta}_{\text{ridge}}$ and $\boldsymbol{\beta}_{\text{LASSO}}$ to be close to zero.
- To avoid ad-hoc choices, we should select λ using cross-validation.

$$L_{LASSO}(\beta) = \frac{1}{n}\sum_{i=1}^{n} |y_i - \boldsymbol{\beta}^\top \boldsymbol{x}_i|^2 + \lambda \sum_{j=1}^{J} |\beta_j|. \qquad L_{Ridge}(\beta) = \frac{1}{n}\sum_{i=1}^{n} |y_i - \boldsymbol{\beta}^\top \boldsymbol{x}_i|^2 + \lambda \sum_{j=1}^{J} \beta_j^2.$$

Geometry of Data

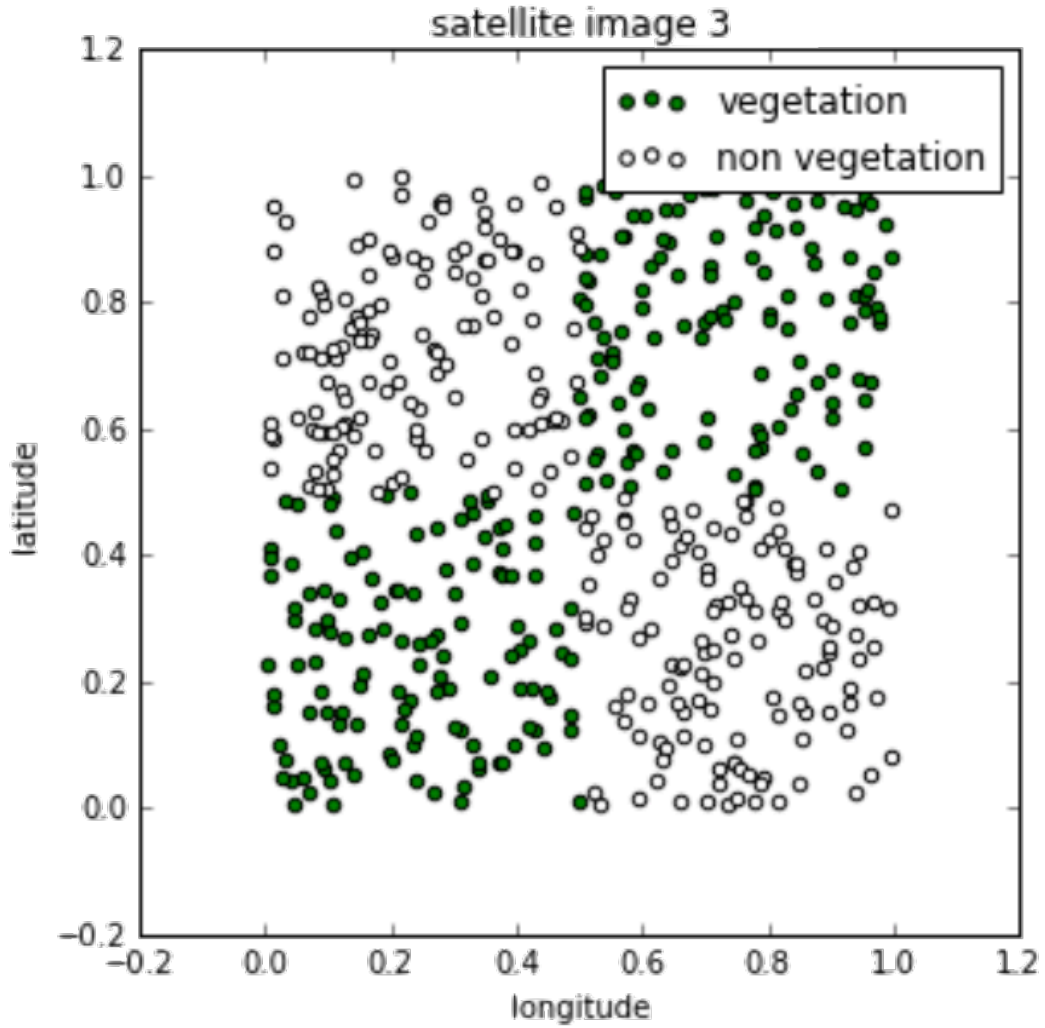
- Recall that, **logistic regression** for building classification boundaries works best when:
  - the classes are well-separated in the feature space
  - have a nice geometry to the classification boundary)

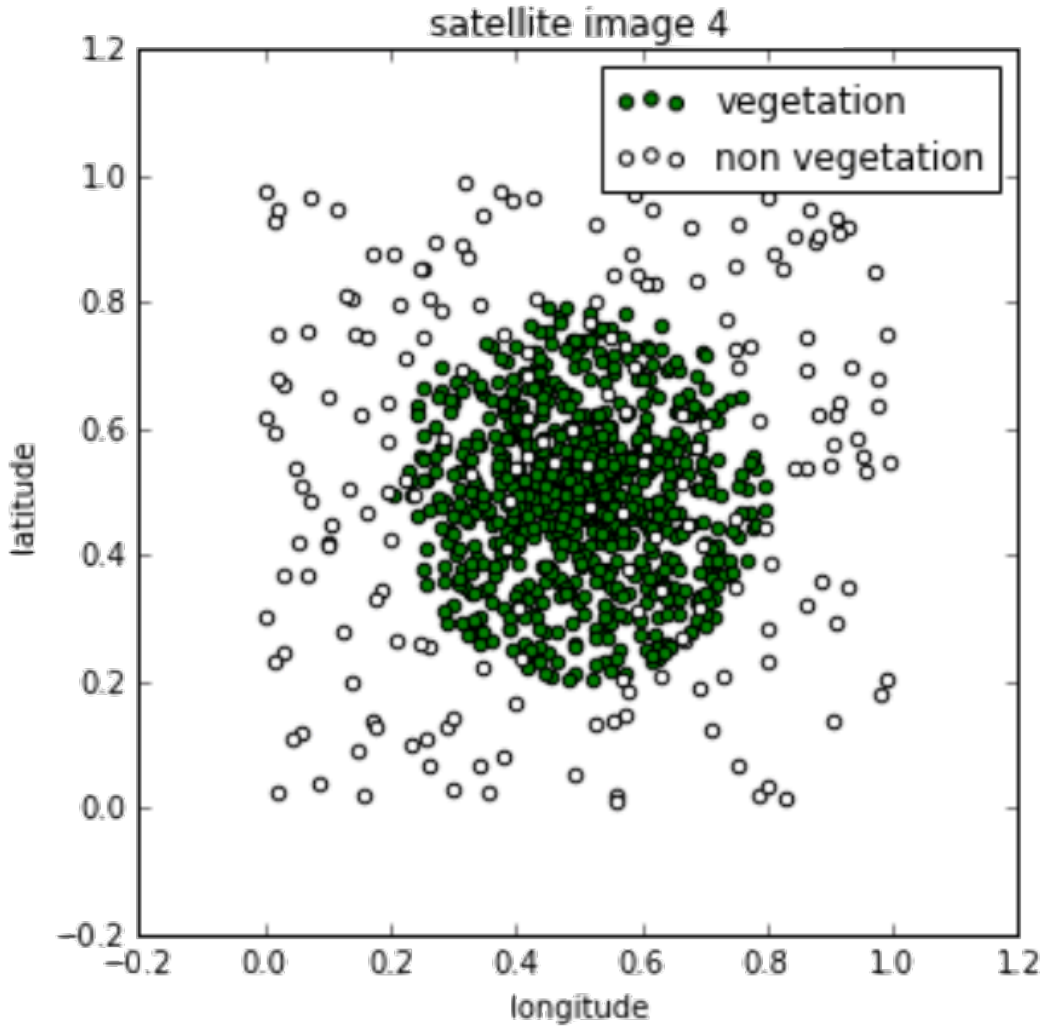


$$-0.8x_1 + x_2 = 0 \implies$$
$$x_2 = 0.8x_1 \Rightarrow$$
$$Lat = 0.8\ Lon$$

Geometry of Data

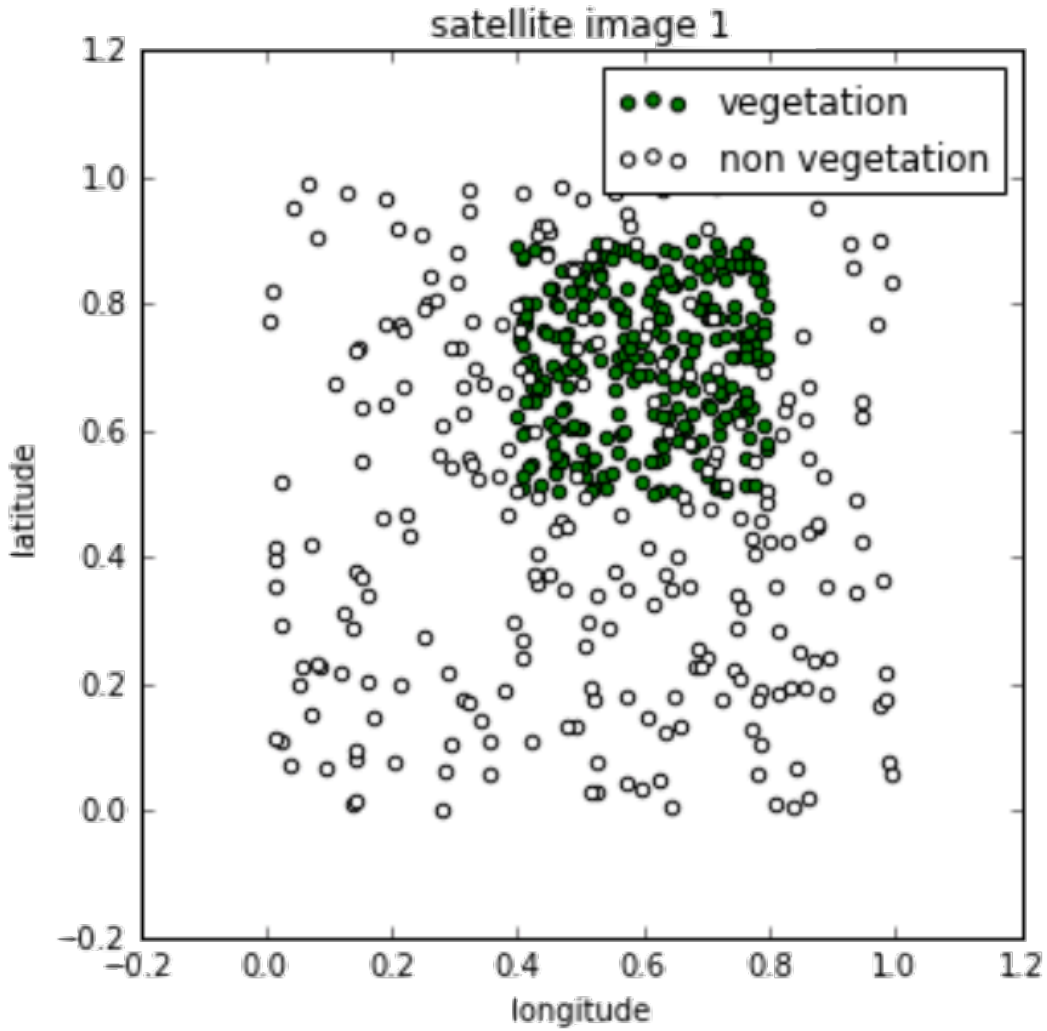
- How about these?

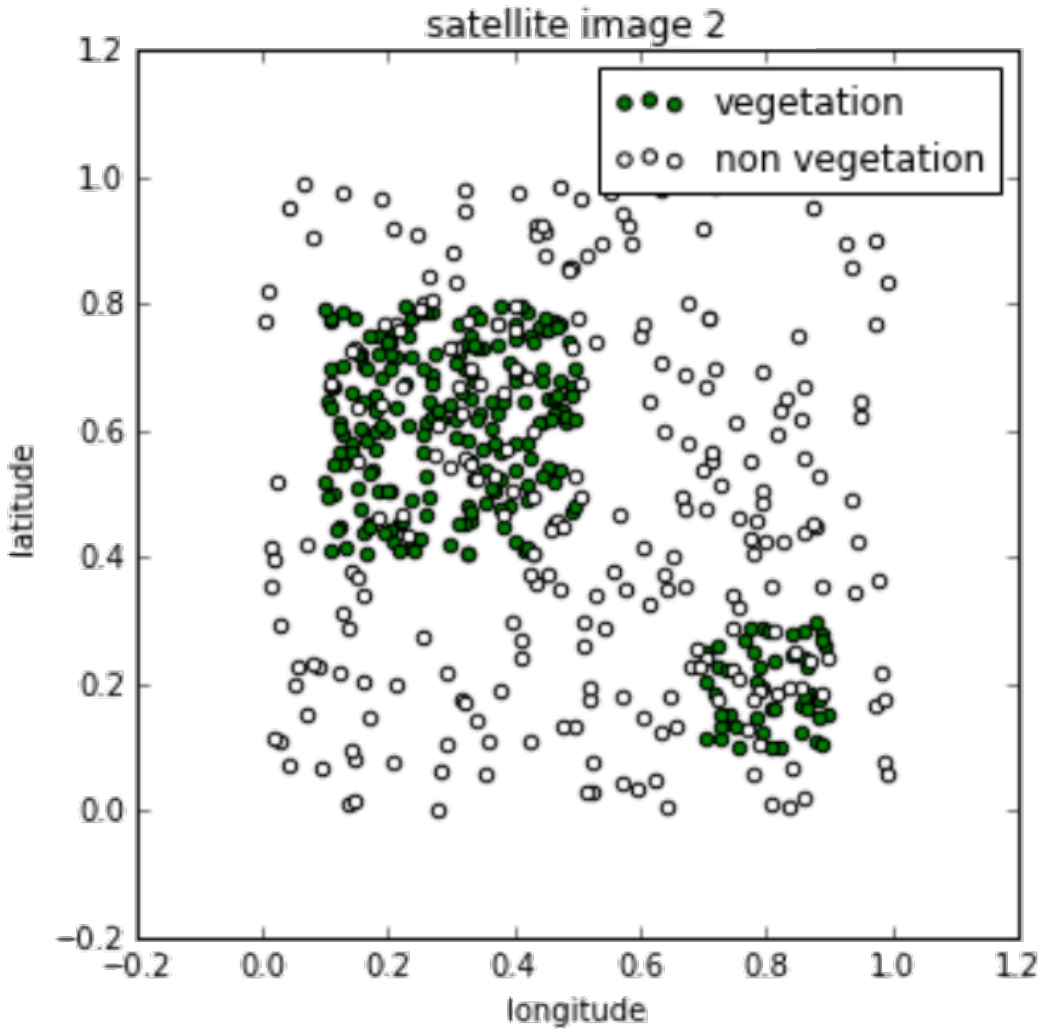




Geometry of Data

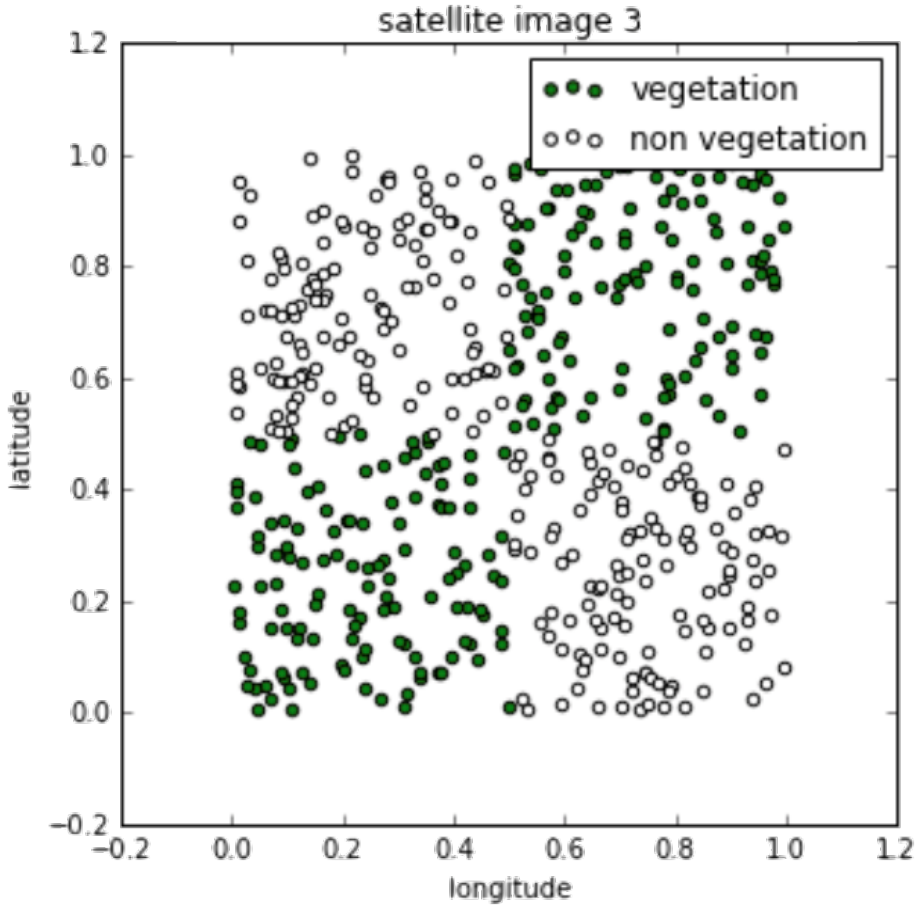
- Or these?





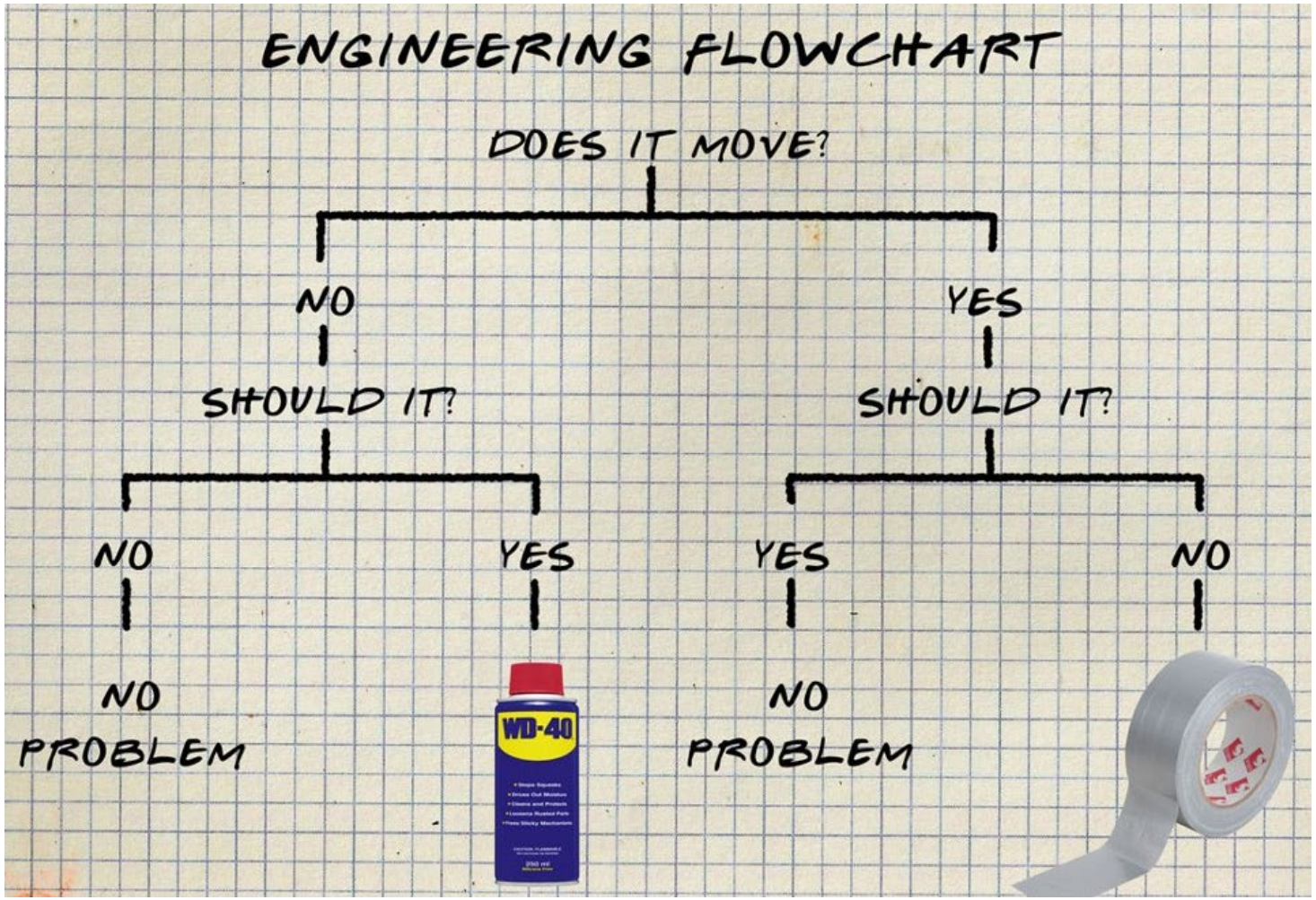
Geometry of Data

- Notice that in all of the datasets the classes are still well-separated in the feature space, but ***the decision boundaries cannot easily be described by single equations***:



Interpretable Models

- People in everyday life have long been using interpretable models for differentiating between classes of objects and phenomena:



Decision Trees

- It turns out that the simple flow charts in our examples can be formulated as mathematical models for classification and these models have the properties we desire; they are:

1. interpretable by humans
2. have sufficiently complex decision boundaries
3. the decision boundaries are locally linear, each component of the decision boundary is simple to describe mathematically.

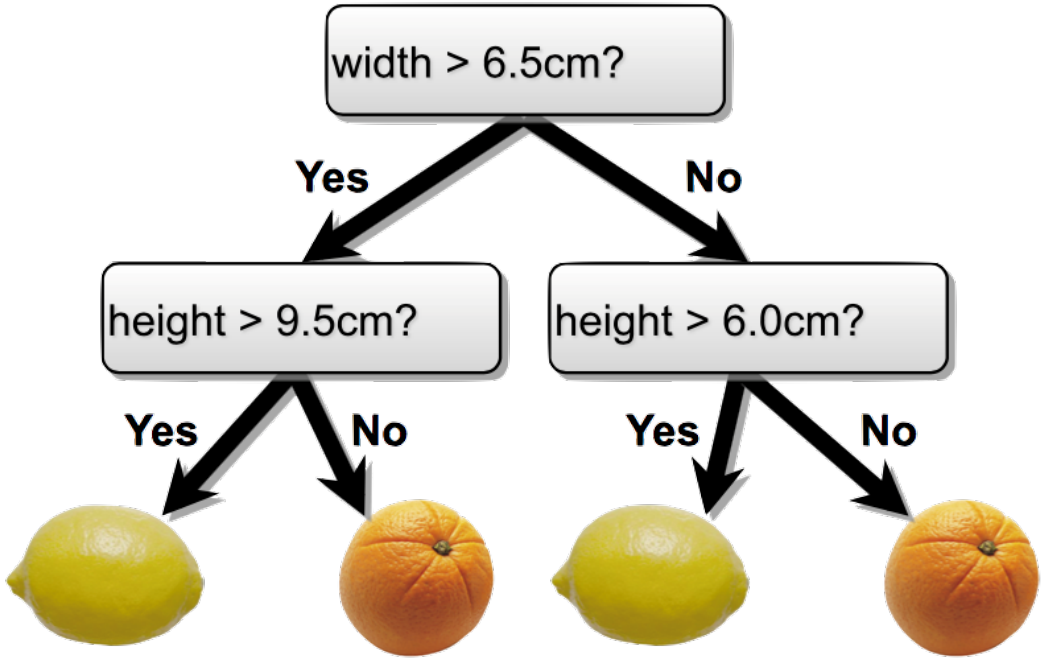
The Geometry of Flow Charts

- Flow charts whose graph is a tree (connected and no cycles) represents a model called a ***decision tree***.
- Formally, a ***decision tree model*** is one in which the final outcome of the model is based on a series of comparisons of the values of predictors against threshold values.
- In a graphical representation (flow chart),
  - the **internal nodes** of the tree represent **attribute testing**.
  - **branching** in the next level is determined by **attribute value (yes/no)**.
  - terminal **leaf nodes** represent **class assignments**.

The Geometry of Flow Charts

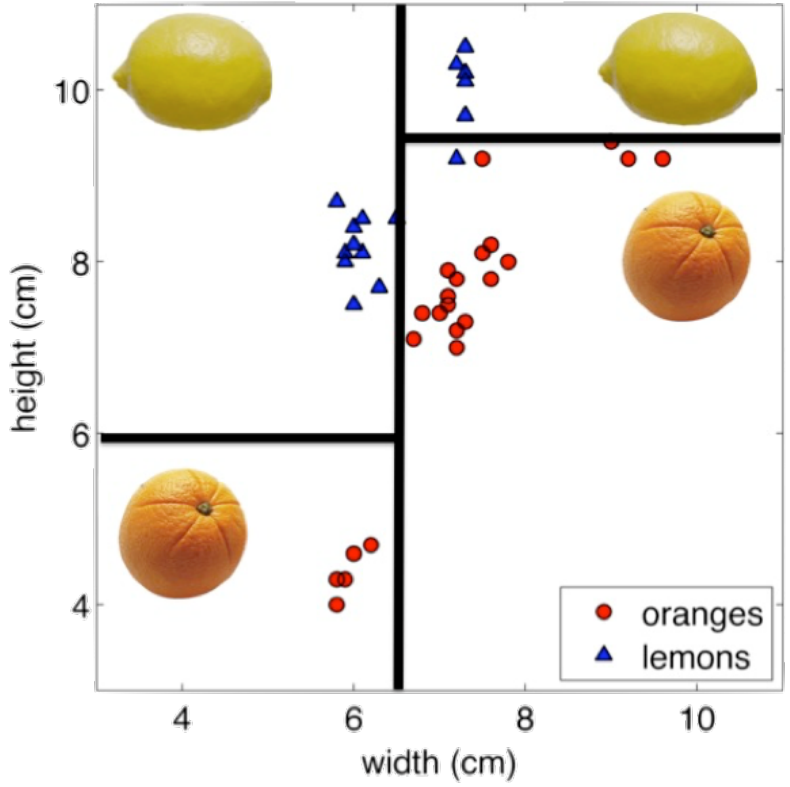
- Flow charts whose graph is a tree (connected and no cycles) represents a model called a ***decision tree***.

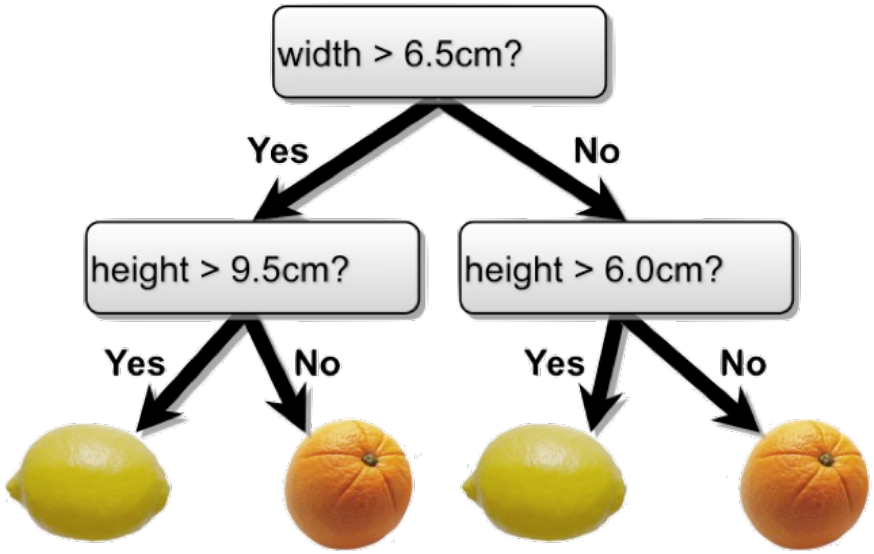
- Formally, a ***decision tree model*** is one in which the final outcome of the model is based on a series of comparisons of the values of predictors against threshold values.



The Geometry of Flow Charts

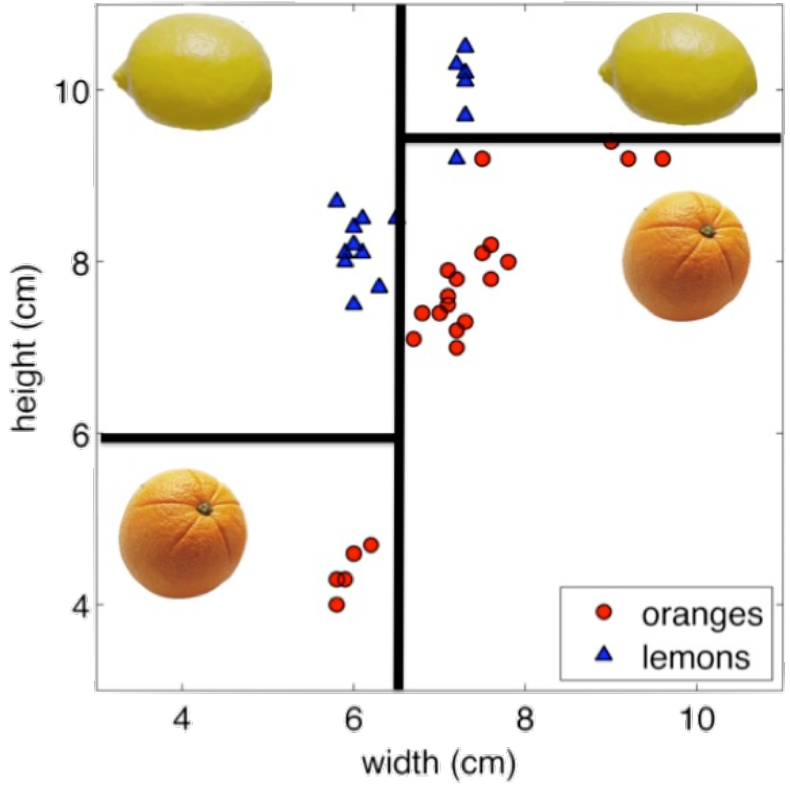
- Every flow chart tree corresponds to a partition of the feature space by **axis aligned lines or (hyper) planes**. Conversely, every such partition can be written as a flow chart tree.

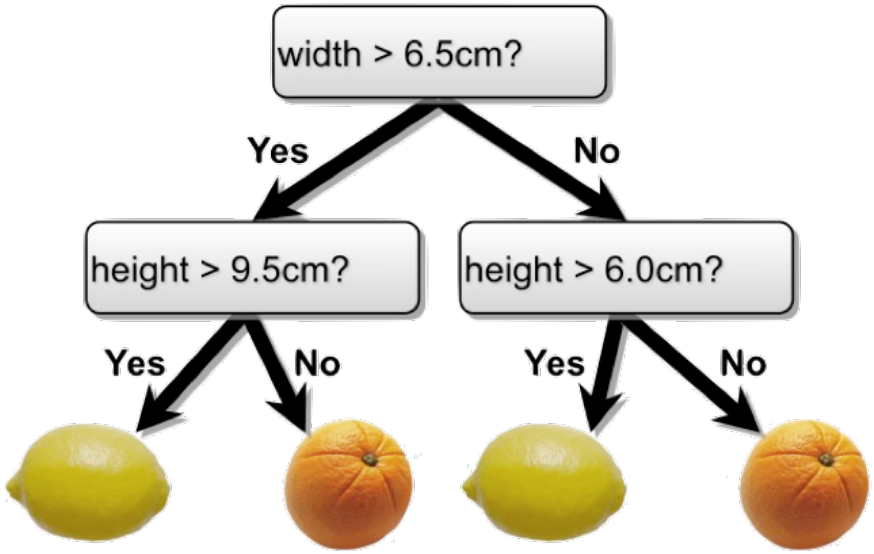




The Geometry of Flow Charts

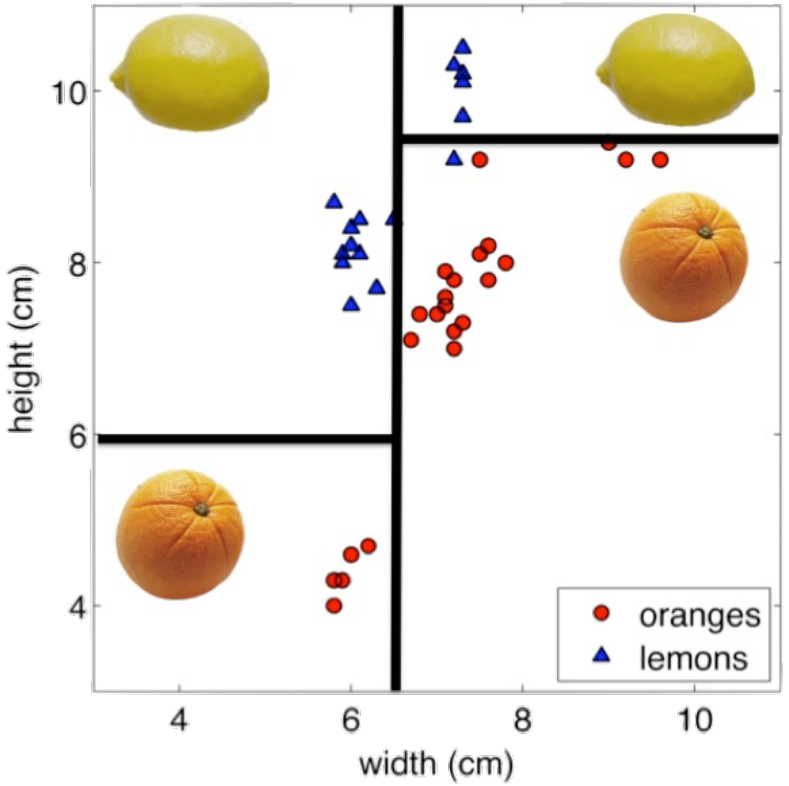
- Each comparison and branching represents splitting a region in the feature space on **a single feature**. Typically, at each iteration, we split once along one dimension (one predictor).





Learning the Model

- Given a training set, *learning* a decision tree model for binary classification means:
  - producing an ***optimal*** partition of the feature space with axis-aligned linear boundaries (very interpretable!),
  - each region is predicted to have a class label based on the **largest class** of the training points in that region (Bayes' classifier) when performing prediction.



Learning the Model

- Learning the smallest ‘optimal’ decision tree for any given set of data is NP complete for numerous simple definitions of ‘optimal’. Instead, we will seek a reasonably model using a greedy algorithm.

    1. Start with an empty decision tree (undivided feature space)

    2. Choose the ‘optimal’ predictor on which to split and choose the ‘optimal’ threshold value for splitting.

    3. Recurse on each new node until ***stopping condition*** is met

- Now, we need only define our splitting criterion and stopping condition.

Numerical vs Categorical Attributes

- Note that the ‘compare and branch’ method by which we defined classification tree works well for numerical features.

- However, if a feature is categorical (with more than two possible values), comparisons like *`feature < threshold`* does not make sense.

- How can we handle this?

- A simple solution is to encode the values of a categorical feature using numbers and treat this feature like a numerical variable. This is indeed what some computational libraries (e.g. `sklearn`) do, however, this method has drawbacks.

Numerical vs Categorical Attributes

Example

Supposed the feature we want to split on is **color**, and the values are: Red, Blue and Yellow. If we encode the categories numerically as:

Red = 0, Blue = 1, Yellow = 2

Then the possible non-trivial splits on **color** are

{{Red}, {Blue, Yellow}} {{Red, Blue},{Yellow}}

But if we encode the categories numerically as:

Red = 2, Blue = 0, Yellow = 1

The possible splits are

{{Blue}, {Yellow, Red}} {{Blue,Yellow}, {Red}}

Depending on the encoding, the splits we can optimize over can be different!

Numerical vs Categorical Attributes

- In practice, the effect of our choice of naive encoding of categorical variables are often negligible - models resulting from different choices of encoding will perform comparably.

- In cases where you might worry about encoding, there is a more sophisticated way to numerically encode the values of categorical variables so that one can optimize over all possible partitions of the values of the variable.

- This more principled encoding scheme is computationally more expensive but is implemented in a number of computational libraries (e.g. `R's randomForest`).

Example of building a Decision Tree from discrete features

Problem: decide whether to wait for a table at a restaurant based on the following attributes:

1. Alternate: is there an alternative restaurant nearby?
2. Bar: is there a comfortable bar area to wait in?
3. Fri/Sat: is today Friday or Saturday?
4. Hungry: are we hungry?
5. Patrons: number of patrons in the restaurant (None, Some, Full)
6. Price: price range ($, $$, $$$)
7. Raining: is it raining outside?
8. Reservation: have we made a reservation?
9. Type: kind of restaurant (French, Italian, Thai, Burger)
10. WaitEstimate: estimated waiting time (0-10, 10-30, 30-60, >60)

Example of building a Decision Tree from discrete features

- Representing samples with attribute values that can be discrete, in ranges, or Boolean
- Classification of the samples as positive (T) or negative (F)

Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

ID3 Algorithm – works with discrete values only

ID3 Algorithm:

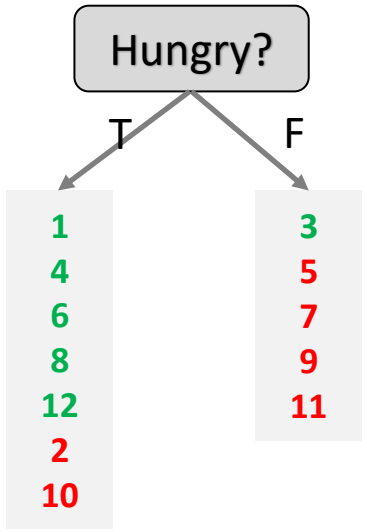
1. If all samples of the training set S are positive examples of the concept K, then the decision (sub)tree consists only of one leaf labeled with K.

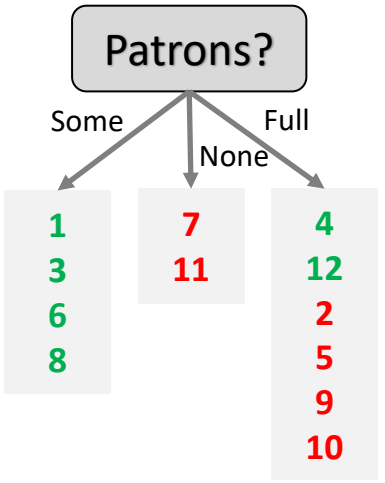
2. Otherwise, choose a test that will split the training set S into two or more subsets S_i, depending on the result of the test (according to the values of the attribute).

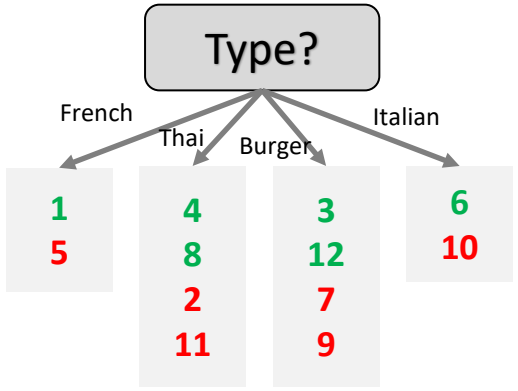
    - The test becomes a root of the (sub)tree and each result of the test builds a subtree, calling this procedure recursively for each S_i.

ID3 Algorithm – works with discrete values only

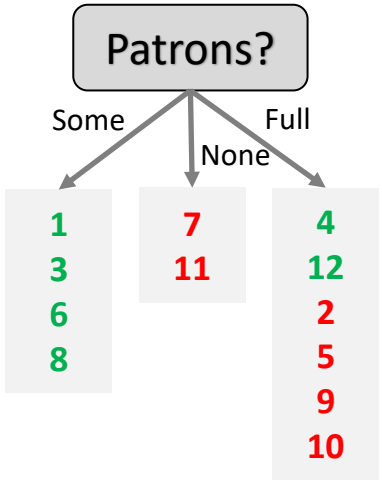
Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_1	T	F	F	T	Some	$$$	F	T	French	0–10	T
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_3	F	T	F	F	Some	$	F	F	Burger	0–10	T
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_6	F	T	F	T	Some	$$	T	T	Italian	0–10	T
X_7	F	T	F	F	None	$	T	F	Burger	0–10	F
X_8	F	F	F	T	Some	$$	T	T	Thai	0–10	T
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{11}	F	F	F	F	None	$	F	F	Thai	0–10	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T

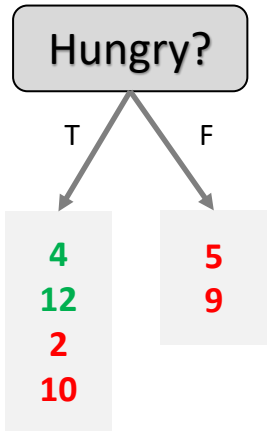


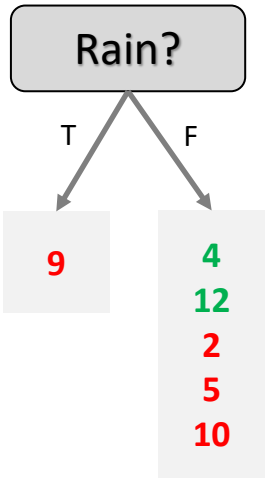




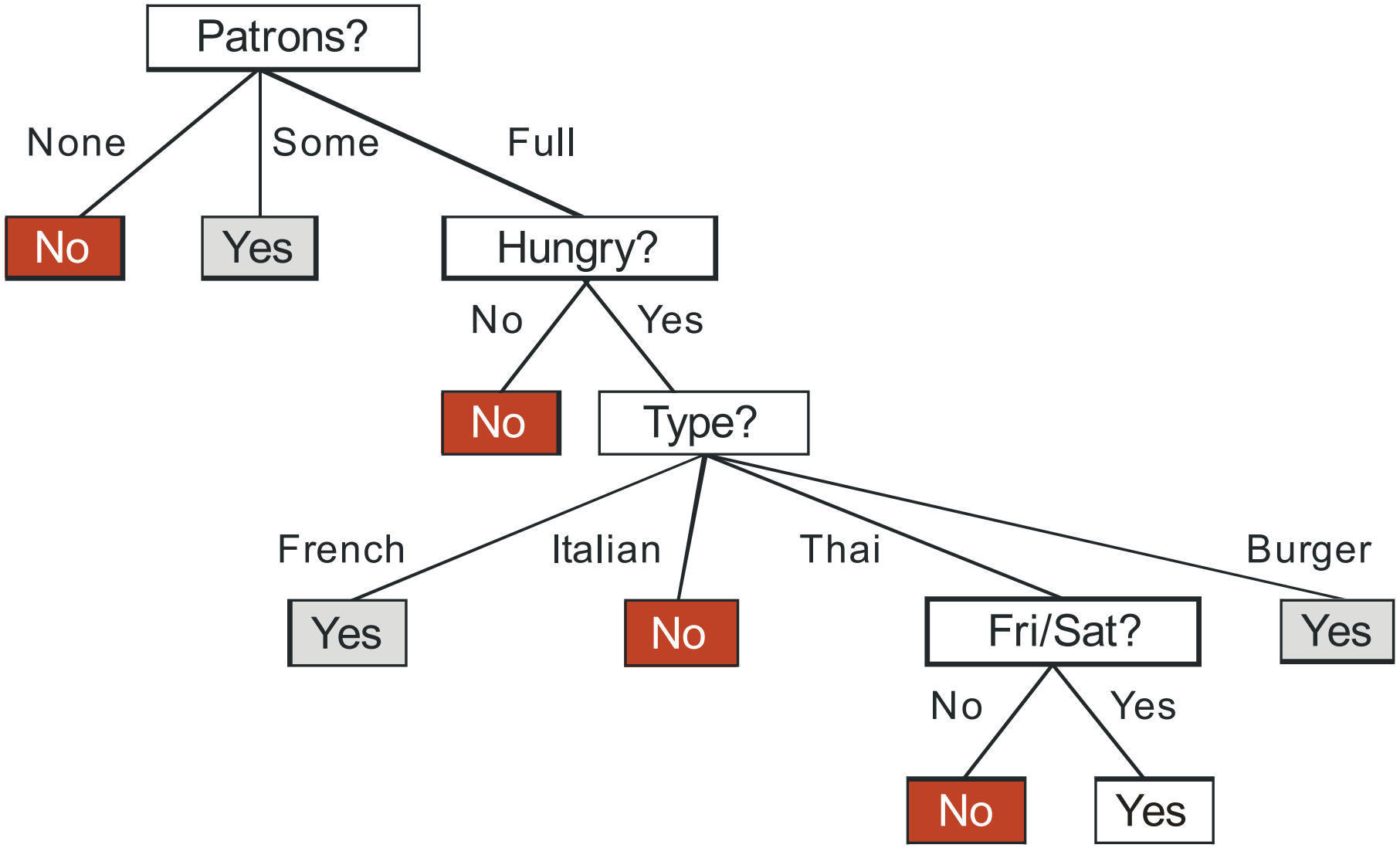
Example	Attributes										Target
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*Wait*
X_2	T	F	F	T	Full	$	F	F	Thai	30–60	F
X_4	T	F	T	T	Full	$	F	F	Thai	10–30	T
X_5	T	F	T	F	Full	$$$	F	T	French	>60	F
X_9	F	T	T	F	Full	$	T	F	Burger	>60	F
X_{10}	T	T	T	T	Full	$$$	F	T	Italian	10–30	F
X_{12}	T	T	T	T	Full	$	F	F	Burger	30–60	T







ID3 Algorithm – works with discrete values only



Splitting criteria

- While there is no ‘correct’ way to define an optimal split, there are some commonsensical guidelines for every splitting criterion:
  - The regions in the feature space should grow progressively purer with the number of splits. That is, we should see each region ‘specialize’ towards a single class.
  - The fitness metric of a split should take a differentiable form (making optimization possible).
  - We shouldn’t end up with empty regions - regions containing no training points.

Classification Error

- Suppose we have J number of predictors and K classes.
- Suppose we select the jth predictor and split a region containing N number of training points along the threshold $t_j \in \mathbb{R}$.
- We can assess the quality of this split by measuring the **classification error** made by each newly created region, R_1, R_2:

$$\text{Error}(i|j, t_j) = 1 - \max_k p(k|R_i)$$

- where $p(k|R_i)$ is the proportion of training points in R_i that are labeled class k.

Gini Index

- Suppose we have J number of predictors, N number of training points and K classes.
- Suppose we select the jth predictor and split a region containing N number of training points along the threshold $t_j \in \mathbb{R}$.
- We can assess the quality of this split by measuring the purity of each newly created region, R_1, R_2. This metric is called the **Gini Index**:

$$\text{Gini}(i|j, t_j) = 1 - \sum_k p(k|R_i)^2$$

Information Theory

- The last metric for evaluating the quality of a split is motivated by metrics of uncertainty in information theory.
- Ideally, our decision tree should split the feature space into regions such that each region represents a single class. In practice, the training points in each region is distributed over multiple classes, e.g.:

	Class 1	Class 2
R_1	1	6
R_2	5	6

- However, though both imperfect, R_1 is clearly sending a stronger 'signal' for a single class (Class 2) than R_2.

Information Theory

- One way to quantify the strength of a signal in a particular region is to analyze the distribution of classes within the region. We compute the **entropy** of this distribution.
- For a random variable with a discrete distribution, the entropy is computed by:

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$

- Higher entropy means the distribution is uniform-like (flat histogram) and thus values sampled from it are ‘less predictable’ (all possible values are equally probable).
- Lower entropy means the distribution has more defined peaks and valleys and thus values sampled from it are ‘more predictable’ (values around the peaks are more probable).

Entropy

- Suppose we have J number of predictors, N number of training points and K classes.
- Suppose we select the jth predictor and split a region containing N number of training points along the threshold $t_j \in \mathbb{R}$.
- We can assess the quality of this split by measuring the entropy of the class distribution in each newly created region, R_1, R_2:
- $$\min_{j,t_j} \left\{ \frac{N_1}{N} \text{Entropy}(1|j,t_j) + \frac{N_2}{N} \text{Entropy}(2|j,t_j) \right\}$$

These are entropies within subsets after the split

- **Note:** we are actually computing the conditional entropy of the distribution of training points amongst the K classes given that the point is in region i.

Summary of splitting criteria

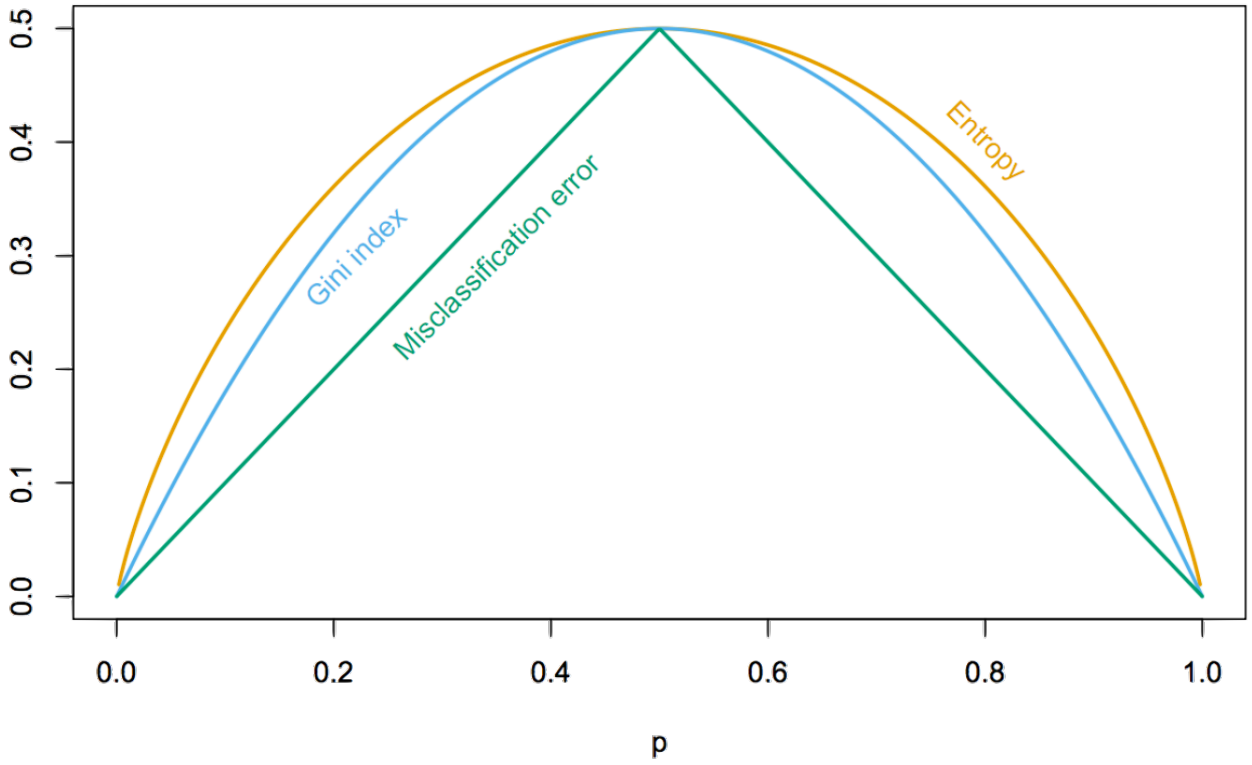
Example			
	Class 1	Class 2	Error$(i \mid j, t_j)$
R_1	0	6	$1 - \max\{6/6, 0/6\} = 0$
R_2	5	8	$1 - \max\{5/13, 8/13\} = 5/13 \approx 0.38$

Example			
	Class 1	Class 2	GINI(i | j, t_j)
R_1	0	6	$1 - (6/6)^2 + (0/6)^2 = 0$
R_2	5	8	$1 - (5/13)^2 + (8/13)^2 = 80/169 \approx 0.47$

Example			
	Class 1	Class 2	Entropy(i | j, t_j)
R_1	0	6	$-(\frac{6}{6}\log_2\frac{6}{6} + \frac{0}{6}\log_2\frac{0}{6}) = 0$
R_2	5	8	$-(\frac{5}{13}\log_2\frac{5}{13} + \frac{8}{13}\log_2\frac{8}{13}) \approx 1.38$

Comparison of Splitting Criteria

- Recall our intuitive guidelines for splitting criteria, which of the three criteria fits our guideline the best?
- We have the following comparison of the value of the three criteria at different levels of purity (from 0 to 1) in a single region (for binary outcomes).



Comparison of Splitting Criteria

- Recall our intuitive guidelines for splitting criteria, which of the three criteria fits our guideline the best?
- **To note that entropy penalizes impurity the most is not to say that it is the best splitting criteria.** For one, a model with purer leaf nodes on a training set may not perform better on the testing test.
- Another factor to consider is the size of the tree (i.e. model complexity) each criteria tends to promote.
- To compare different decision tree models, we need to first discuss *stopping conditions*.

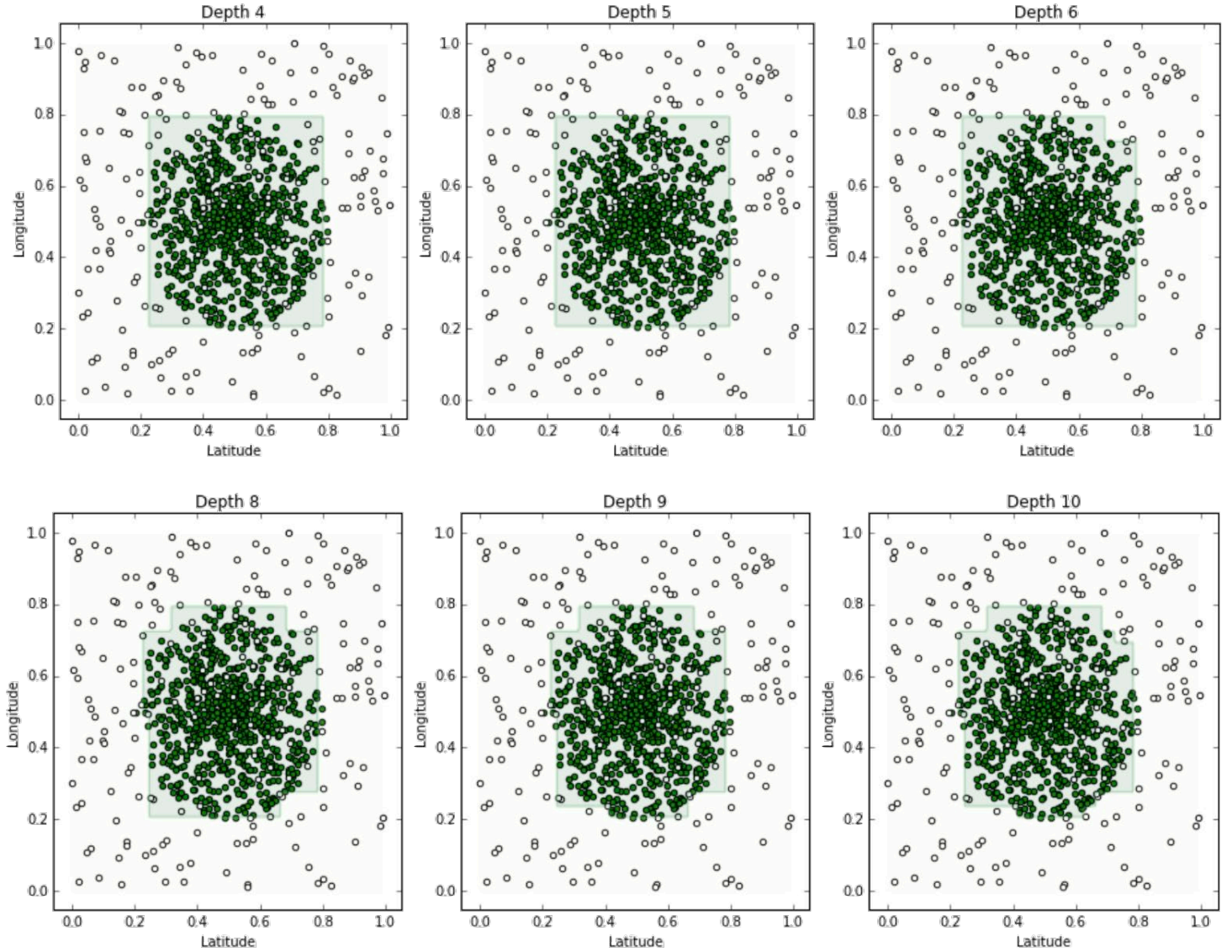
Stopping Criteria

- If we don't terminate the decision tree learning algorithm manually, the tree will continue to grow until each region defined by the model possibly contains exactly one training point (and the model attains 100% training accuracy).

  – To prevent this from happening, we can simply **stop the algorithm** at a particular depth.

  – **But how do we determine the appropriate depth?**

Variance vs Bias



Variance vs Bias

- We make some observations about our models:
  - **(High Bias)** A tree of depth 4 is not a good fit for the training data - it's unable to capture the nonlinear boundary separating the two classes.
  - **(Low Bias)** With an extremely high depth, we can obtain a model that correctly classifies all points on the boundary (by zig-zagging around each point).
  - **(Low Variance)** The tree of depth 4 is robust to slight perturbations in the training data - the square carved out by the model is stable if you move the boundary points a bit.
  - **(High Variance)** Trees of high depth are sensitive to perturbations in the training data, especially to changes in the boundary points.
- Not surprisingly, complex trees have low bias (able to capture more complex geometry in the data) but high variance (can overfit). Complex trees are also harder to interpret and more computationally expensive to train.

Stopping Conditions

- Common simple stopping conditions:
  - Don’t split a region if all instances in the region belong to the same class
  - Don’t split a region if the number of instances in the sub-region will fall below pre-defined threshold (**min_samples_leaf**)
  - Don’t split a region if the total number of leaves in the tree will exceed pre-defined threshold

- The appropriate thresholds can be determined by evaluating the model on a held-out data set or, better yet, via cross-validation.

Stopping Conditions

- More restrictive stopping conditions:
  - Don’t split a region if the class distribution of the training points inside the region are independent of the predictors.
  - Compute the gain in purity, information or reduction in entropy of splitting a region R into R_1 and R_2:

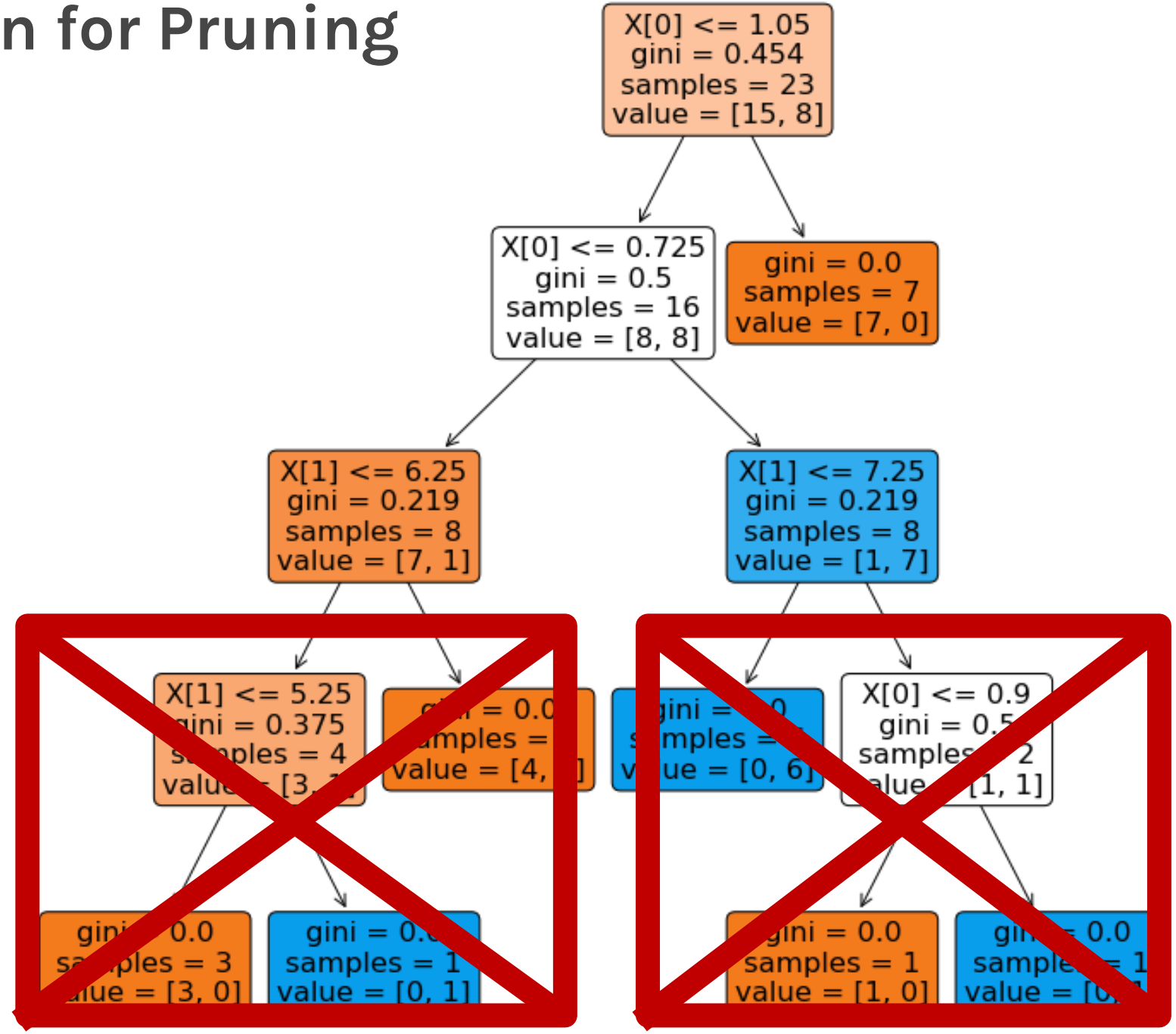
$$Gain(R) = \Delta(R) = m(R) - \frac{N_1}{N} m(R_1) - \frac{N_2}{N} m(R_2)$$

  - where m is a metric like the Gini Index or entropy. Don’t split if the gain is less than some pre-defined threshold (**min_impurity_decrease**).

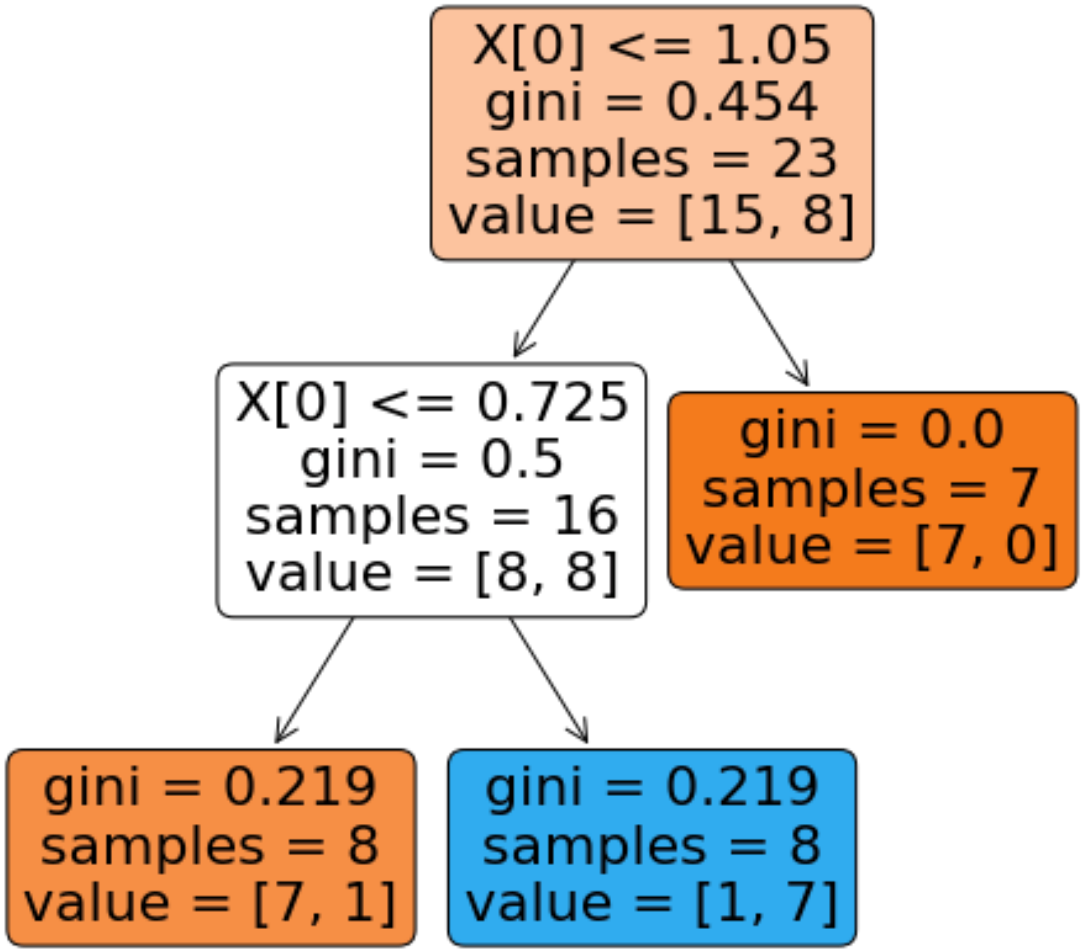
Alternative to Using Stopping Conditions

- What is the major issue with pre-specifying a stopping condition?
  - **you may stop too early or stop too late.**
- How can we fix this issue?
  - **choose several stopping criterion (set minimal Gain(*R*) at various levels) and cross-validate which is the best.**
- What is an alternative approach to this issue?
  - **Don’t stop. Instead prune back!**

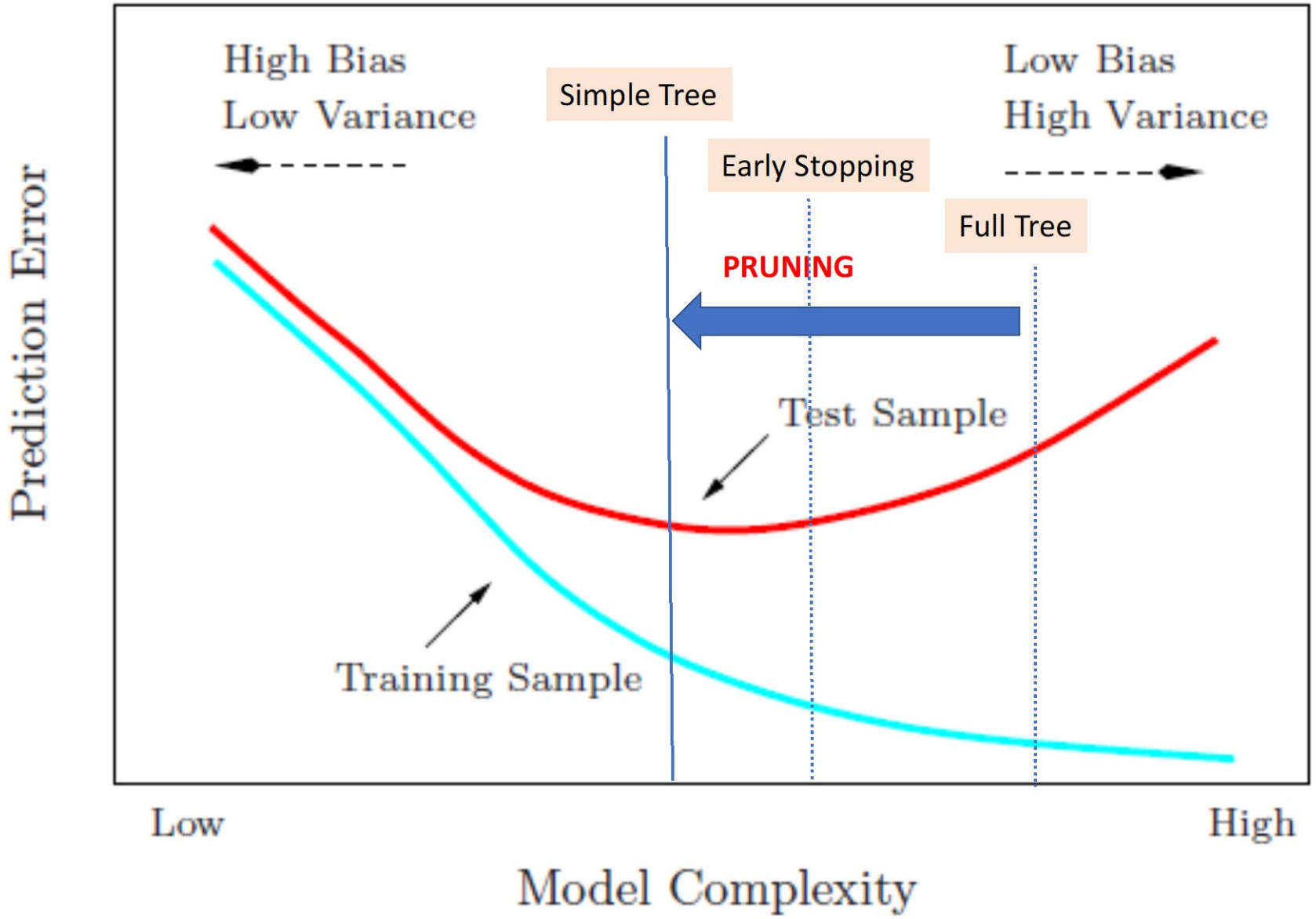
Motivation for Pruning



Motivation for Pruning



Motivation for Pruning



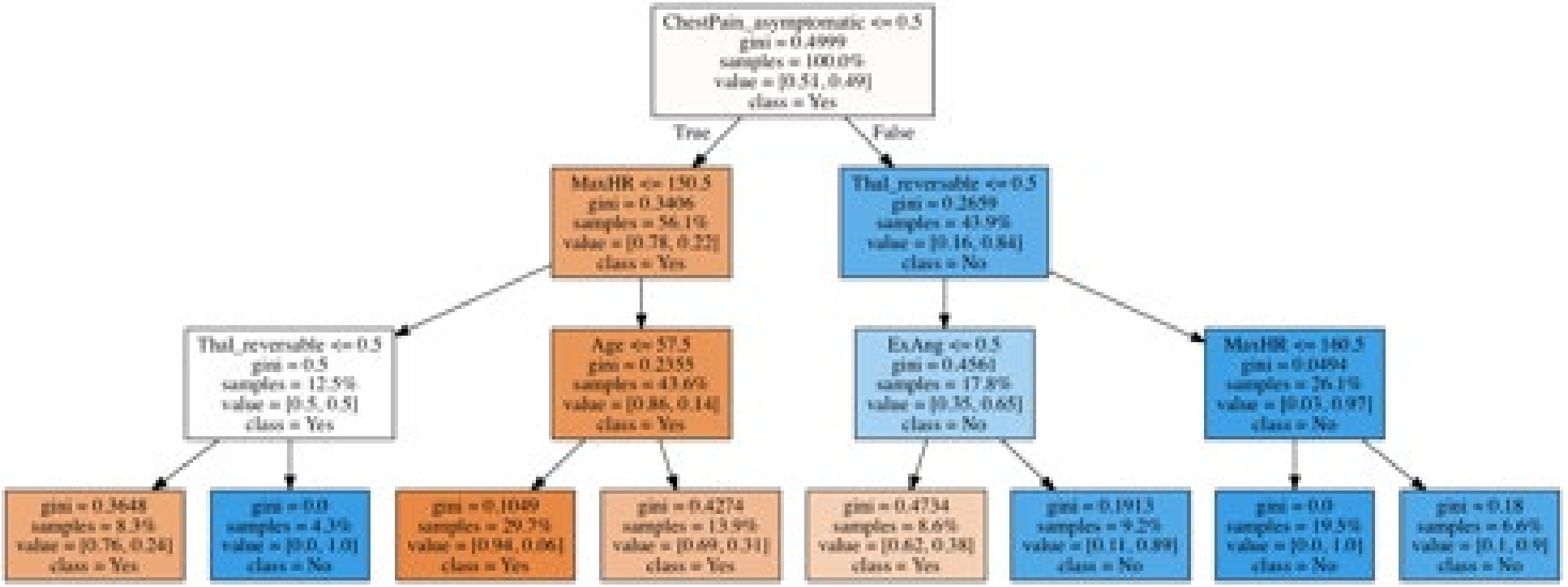
Pruning

- Rather than preventing a complex tree from growing, we can obtain a simpler tree by '**pruning**' a complex one.
- There are many method of pruning, a common one is **cost complexity pruning**, where we select from an array of smaller subtrees of the full model that optimizes a balance of performance and efficiency.
  - That is, we measure

$$C(T) = Error(T) + \alpha|T|$$

  - where T is a decision (sub) tree, $|T|$ is the number of leaves in the tree and α is the parameter for penalizing model complexity.

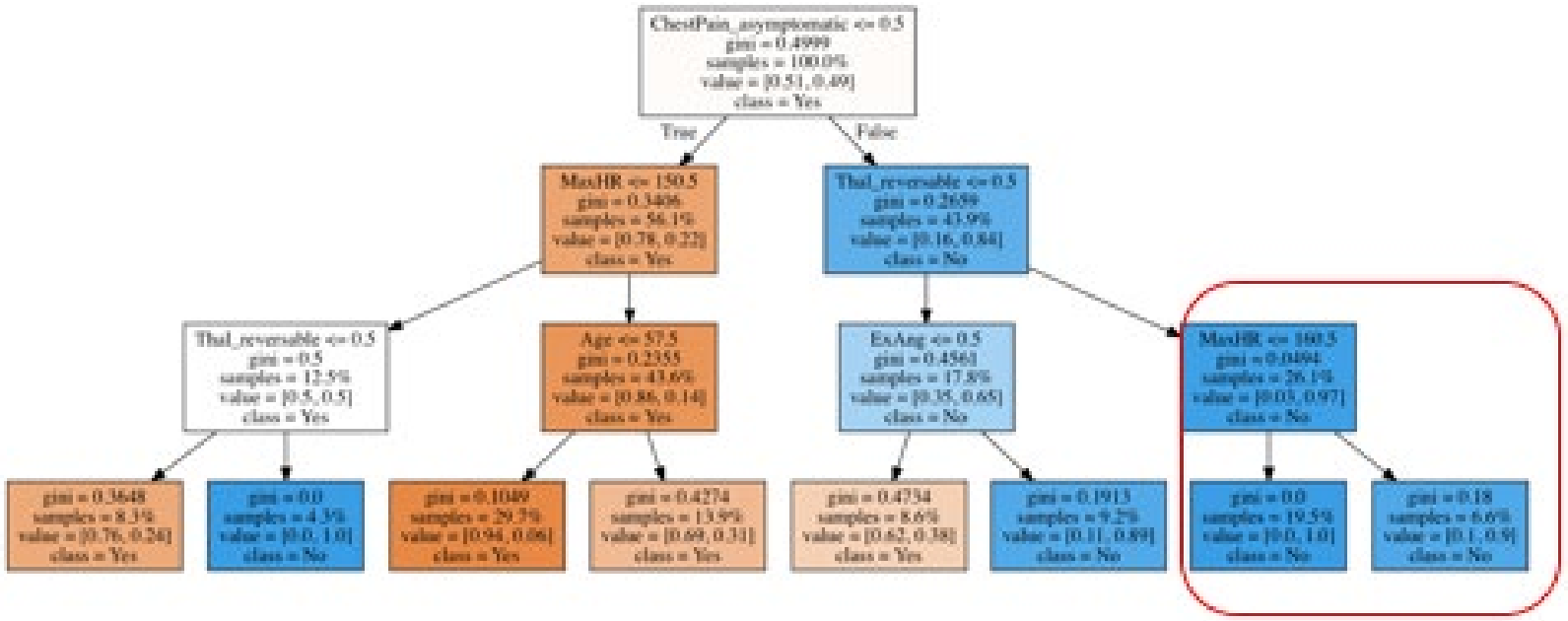
Pruning



$$\alpha = 0.2$$

Tree	Error	Num Leaves	Total

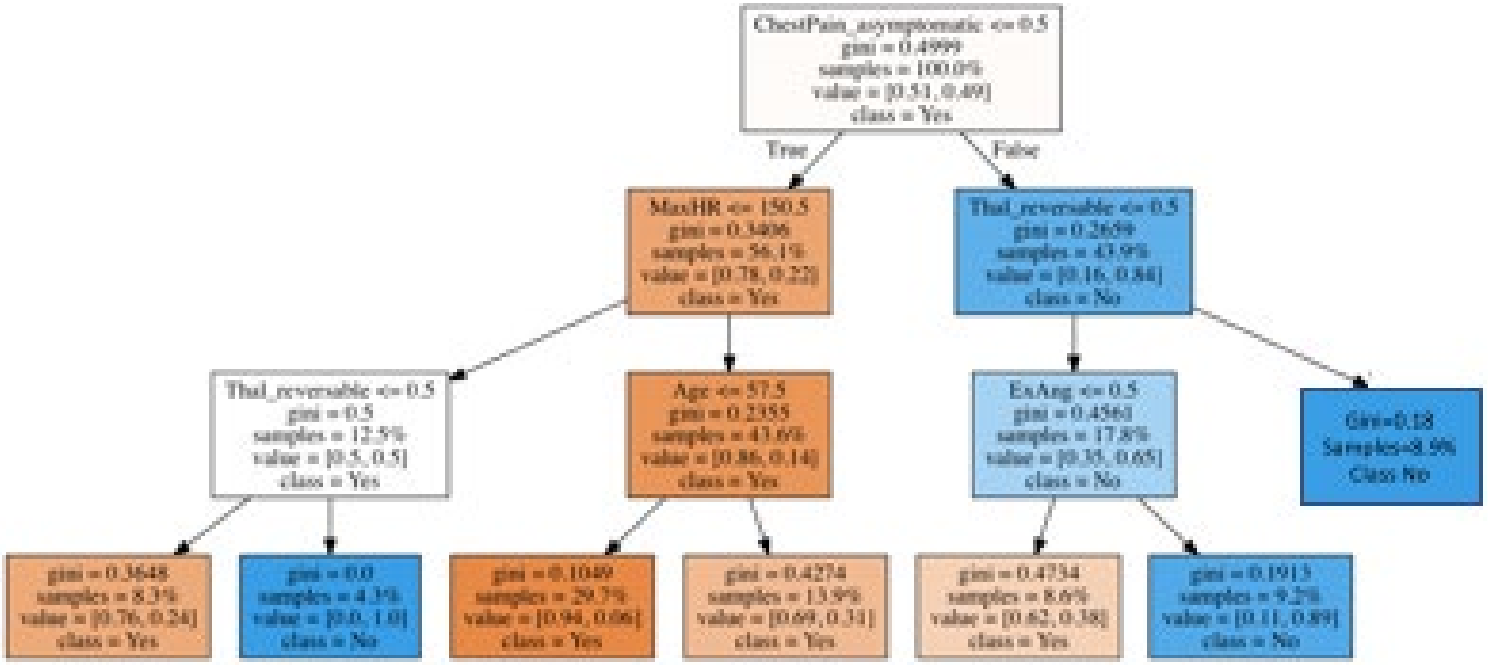
Pruning



$\alpha = 0.2$

Tree	Error	Num Leaves	Total
T	0.32	8	1.92

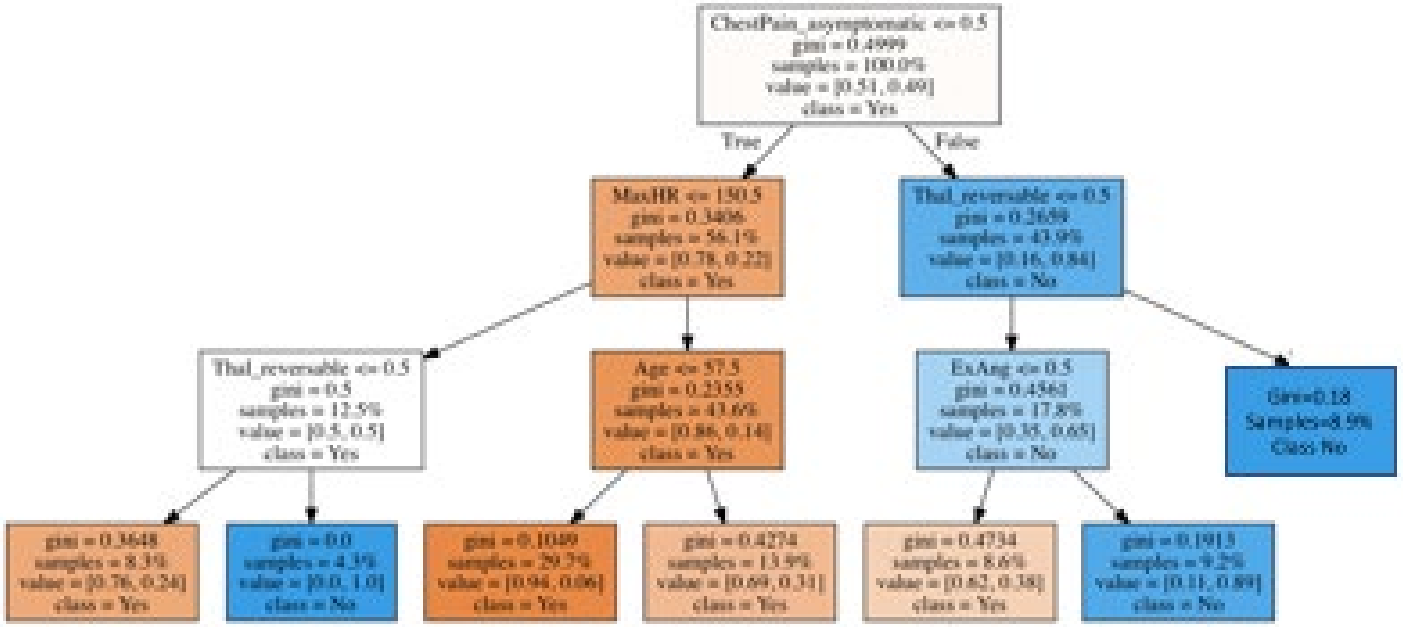
Pruning



$\alpha = 0.2$

Tree	Error	Num Leaves	Total
T	0.32	8	1.92
Tsmall	0.33	7	1.73

Pruning



$\alpha = 0.2$

Tree	Error	Num Leaves	Total
T	0.32	8	1.92
Tsmall	0.33	7	1.73

Smaller tree has larger error but less cost complexity score

slightly

Pruning

$$C(T) = Error(T) + \alpha|T|$$

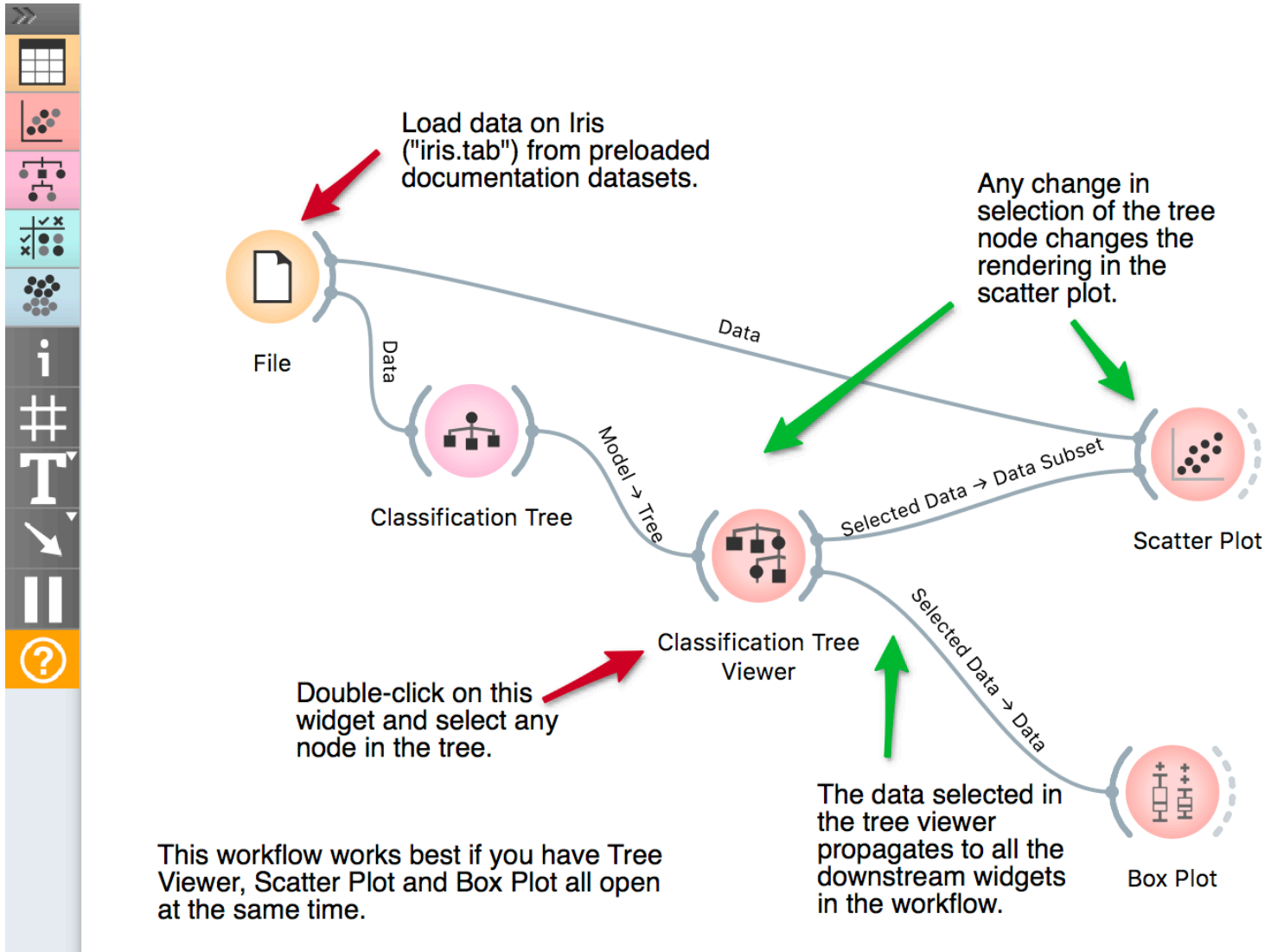
1. Fix α.

2. Find best tree for a given α and based on cost complexity C.

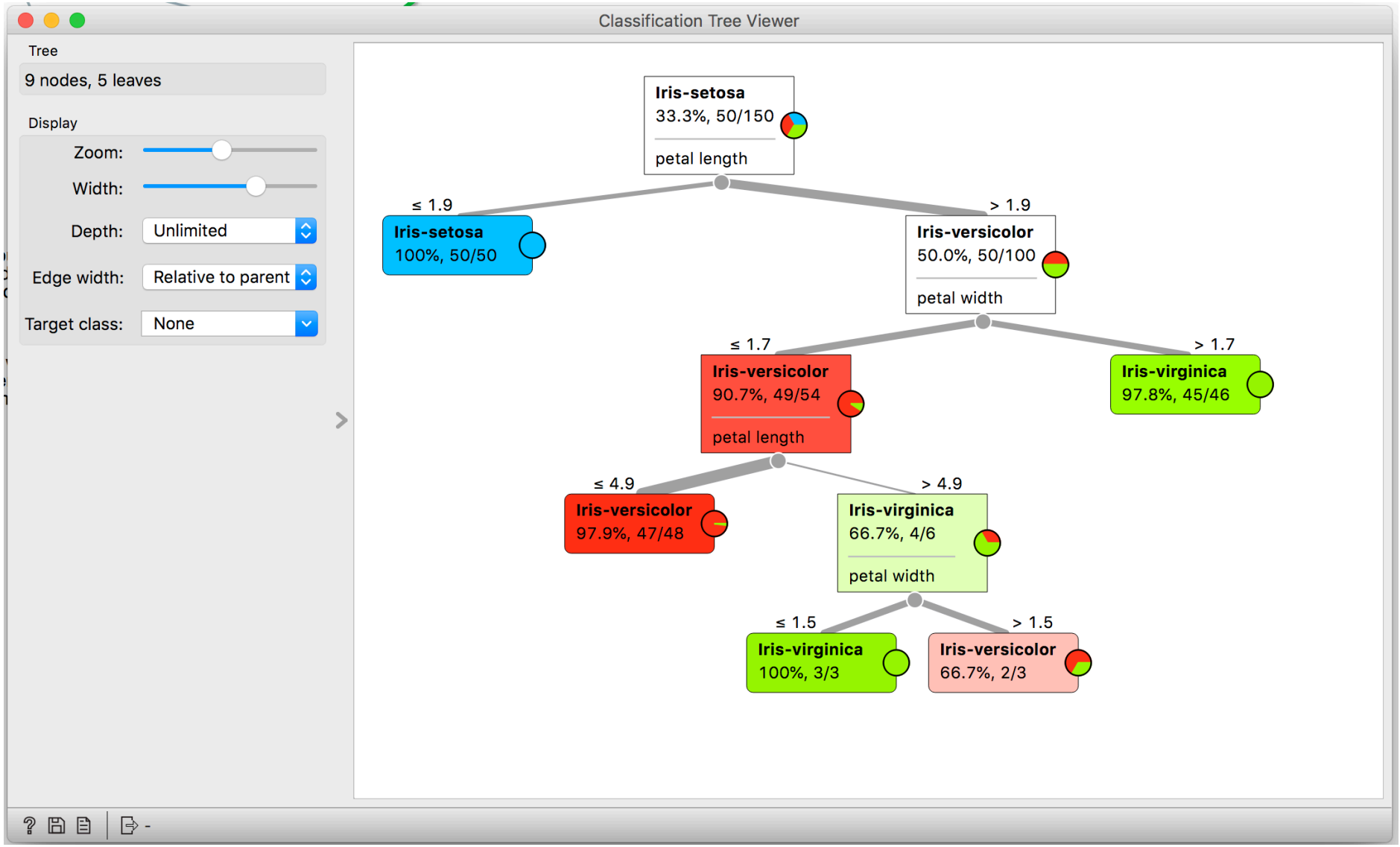
3. Find best α using Cross-Validation (what should be the error measure?)

Orange Data Mining (https://orangedatamining.com)

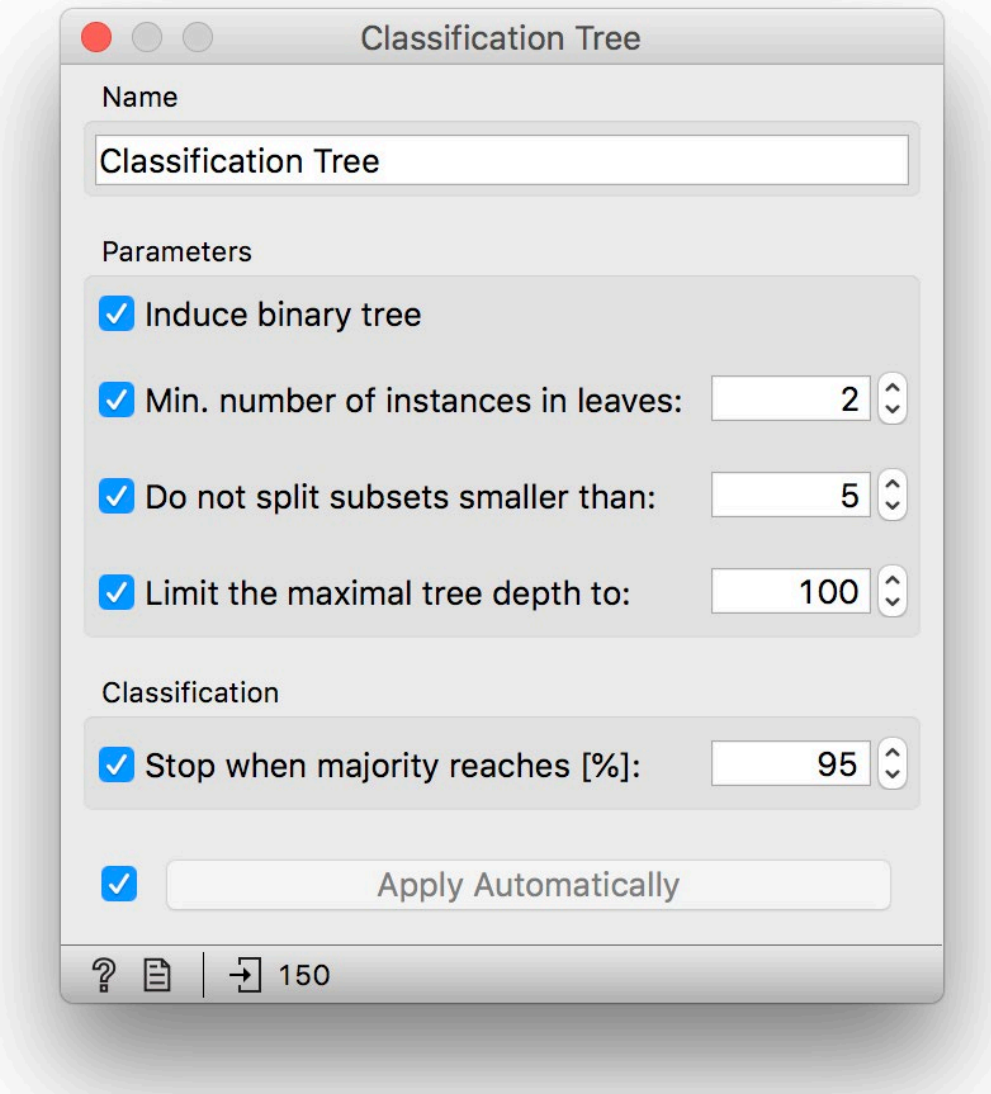
Open-source machine learning and data visualization.



Classification Tree Viewer in Orange



Classification Tree Parameters



An example with sklearn

```
from sklearn.tree import DecisionTreeClassifier
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split #--------------splitting data into test and train
from sklearn.tree import DecisionTreeClassifier #-----------Building decision tree model

wine_df = pd.read_csv('winequality-red.csv',sep=';')
```

An example with sklearn (2)

```
In [3]: wine_df.describe().transpose().round(2)
```

Out[3]:

	count	mean	std	min	25%	50%	75%	max
fixed acidity	1599.0	8.32	1.74	4.60	7.10	7.90	9.20	15.90
volatile acidity	1599.0	0.53	0.18	0.12	0.39	0.52	0.64	1.58
citric acid	1599.0	0.27	0.19	0.00	0.09	0.26	0.42	1.00
residual sugar	1599.0	2.54	1.41	0.90	1.90	2.20	2.60	15.50
chlorides	1599.0	0.09	0.05	0.01	0.07	0.08	0.09	0.61
free sulfur dioxide	1599.0	15.87	10.46	1.00	7.00	14.00	21.00	72.00
total sulfur dioxide	1599.0	46.47	32.90	6.00	22.00	38.00	62.00	289.00
density	1599.0	1.00	0.00	0.99	1.00	1.00	1.00	1.00
pH	1599.0	3.31	0.15	2.74	3.21	3.31	3.40	4.01
sulphates	1599.0	0.66	0.17	0.33	0.55	0.62	0.73	2.00
alcohol	1599.0	10.42	1.07	8.40	9.50	10.20	11.10	14.90
quality	1599.0	5.64	0.81	3.00	5.00	6.00	6.00	8.00

An example with sklearn (3)

In [5]:
```
    # splitting data into training and test set for independent attributes
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test =train_test_split(wine_df.drop('quality',axis=1), wine_df['quality'], test_size=.3,
                                                   random_state=22)
X_train.shape,X_test.shape
```

Out[5]:
```
((1119, 11), (480, 11))
```

In [6]:
```
clf_pruned = DecisionTreeClassifier(criterion = "gini", random_state = 100,
                                    max_depth=3, min_samples_leaf=5)
clf_pruned.fit(X_train, y_train)
```

Out[6]:
```
DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=3,
            max_features=None, max_leaf_nodes=None,
            min_impurity_decrease=0.0, min_impurity_split=None,
            min_samples_leaf=5, min_samples_split=2,
            min_weight_fraction_leaf=0.0, presort=False, random_state=100,
            splitter='best')
```

DecisionTreeClassifier Parameters

The following parameters can be tuned to improve the model output (Scikit Learn, 2019):

- **criterion** – Gini impurity is used to decide the variables based on which root node and following decision nodes should be split
- **class_weight** – None; All classes are assigned weight 1
- **max_depth** – 3; Pruning is done. When “None”, it signifies that nodes will be expanded till all leaves are homogeneous
- **max_features** – None; All features or independent variables are considered while deciding split of a node
- **max_leaf_nodes** – None;
- **min_impurity_decrease** – 0.0; A node is split only when the split ensures a decrease in the impurity of greater than or equal to zero
- **min_impurity_split** – None;
- **min_samples_leaf** – 1; Minimum number of samples required for a leaf to exists
- **min_samples_split** – 2; If min_samples_leaf =1, it signifies that the right and the left node should have 1 sample each, i.e. the parent node or the root node should have at least two samples
- **splitter** – ‘best’; Strategy used to choose the split at each node. Best ensure that all features are considered while deciding the split

An example with sklearn (4)

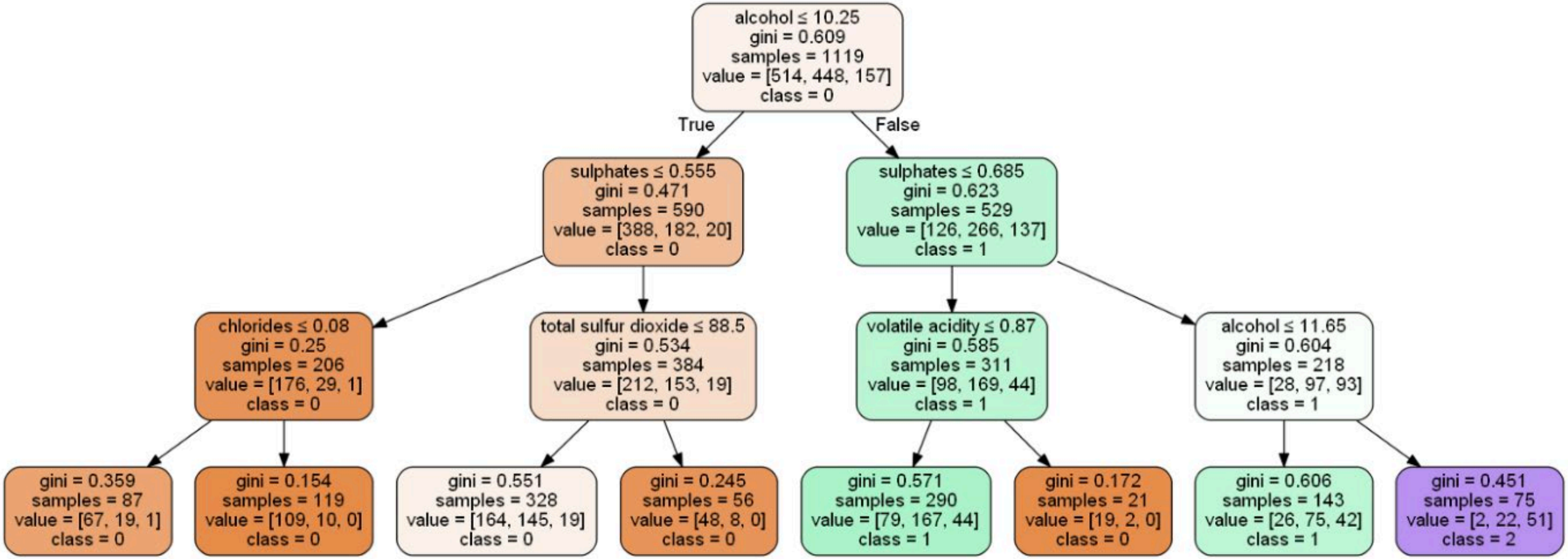


Image illustrating the decision tree model when the tree is pruned at depth =3